

The Effect of Invitation Message Style on Survey Response Rates and Times Analysis

```
library(tidyverse) # This loads dplyr and the %>% pipe
```

Warning: package 'ggplot2' was built under R version 4.4.3

```
— Attaching core tidyverse packages — tidyverse 2.0.0 —
✓ dplyr      1.1.4      ✓ readr      2.1.5
✓ forcats    1.0.0      ✓ stringr    1.5.1
✓ ggplot2    4.0.1      ✓ tibble     3.2.1
✓ lubridate  1.9.3      ✓ tidyr      1.3.1
✓ purrr      1.0.2

— Conflicts — tidyverse_conflicts() —
X dplyr::filter() masks stats::filter()
X dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(MASS)
```

Adjuntando el paquete: 'MASS'

The following object is masked from 'package:dplyr':

select

```
library(pwr)
```

Warning: package 'pwr' was built under R version 4.4.3

DATA PREPROCESSING

We load the complete dataset

```
data_clean <- data.frame(read.table("Results_ICA1.csv", header=TRUE, sep=","))
```

Transform columns into factors

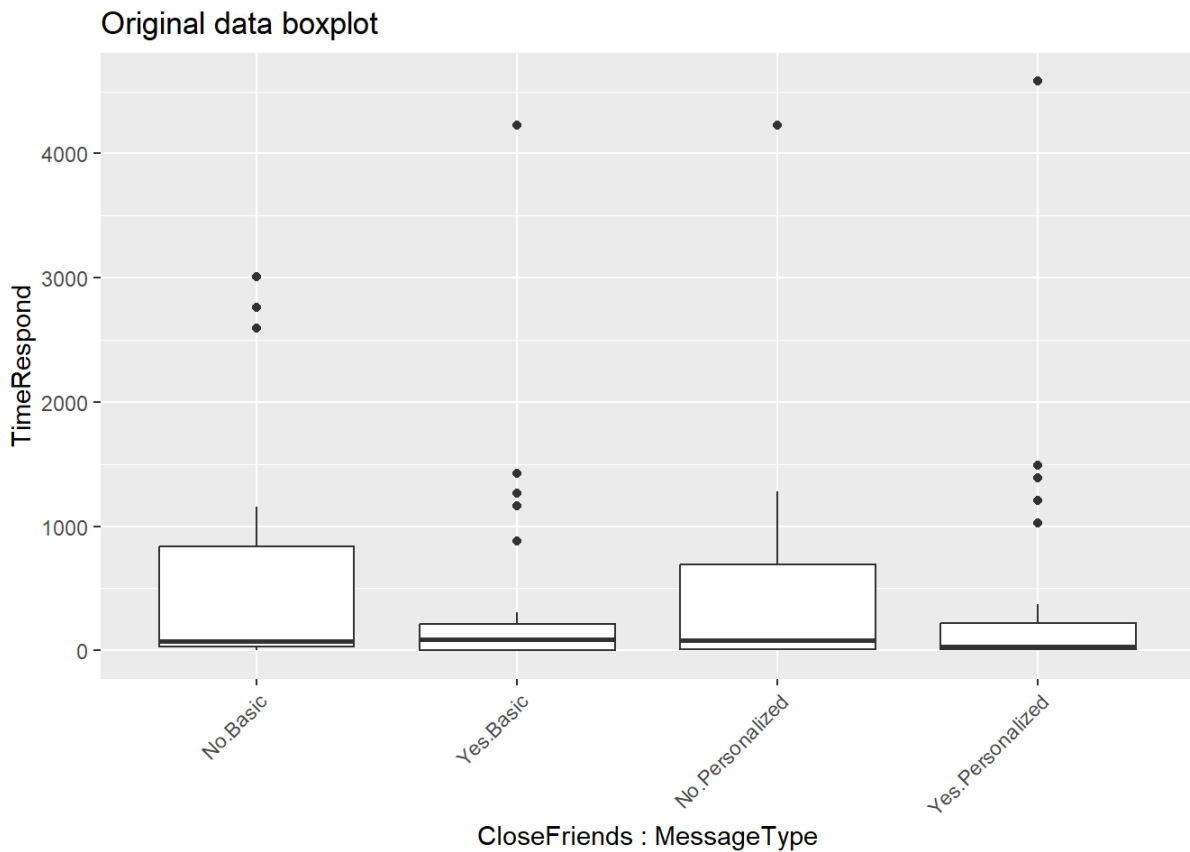
```
data_clean$CloseFriends <- relevel(as.factor(data_clean$CloseFriends), ref = "No")
data_clean$MessageType <- relevel(as.factor(data_clean$MessageType), ref = "Basic")
```

EXPLORATORY ANALYSIS

Boxplots

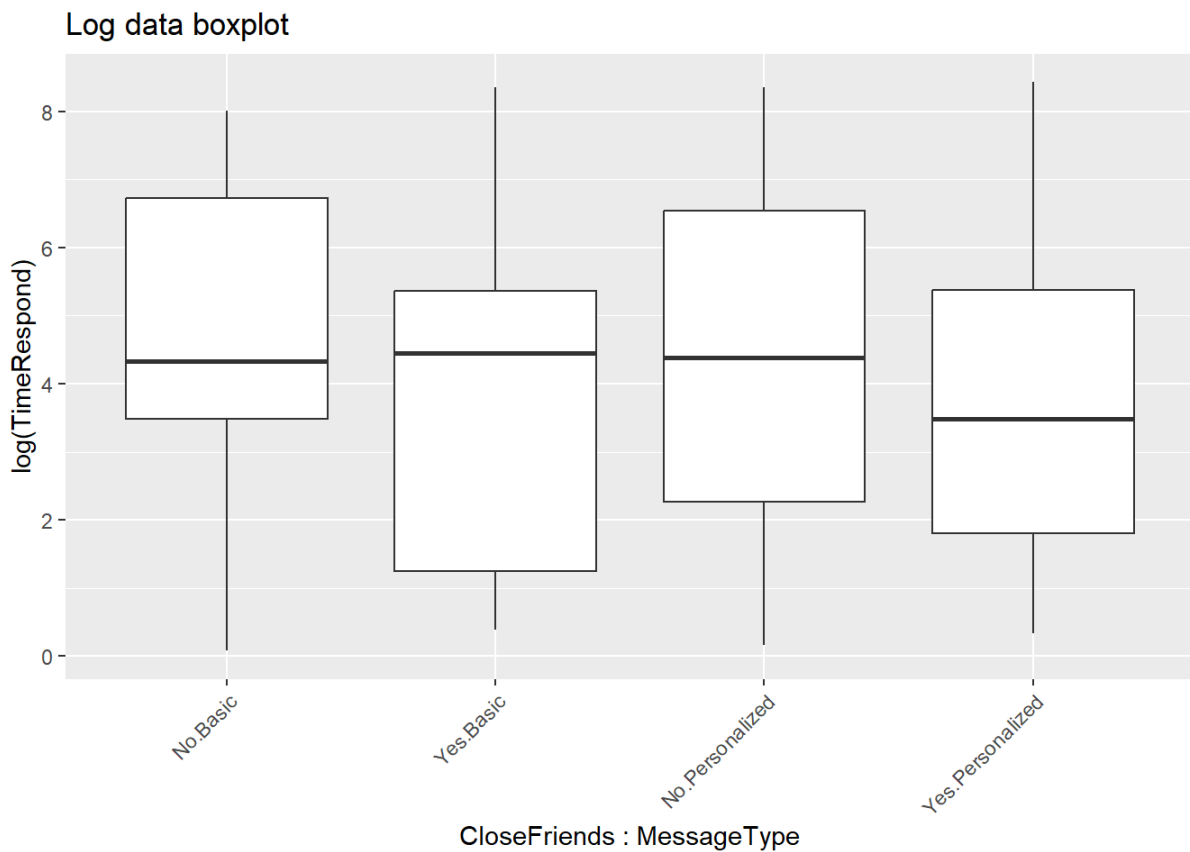
```
ggplot(data_clean, aes(x = interaction(CloseFriends, MessageType), y = TimeRespond)) +
  geom_boxplot() +
  labs(title = "Original data boxplot", x = "CloseFriends : MessageType") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Warning: Removed 29 rows containing non-finite outside the scale range
(`stat\_boxplot()`).



```
ggplot(data_clean, aes(x = interaction(CloseFriends, MessageType), y = log(TimeRespond))) +  
  geom_boxplot() +  
  labs(title = "Log data boxplot", x = "CloseFriends : MessageType") +  
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Warning: Removed 29 rows containing non-finite outside the scale range
(`stat\_boxplot()`).



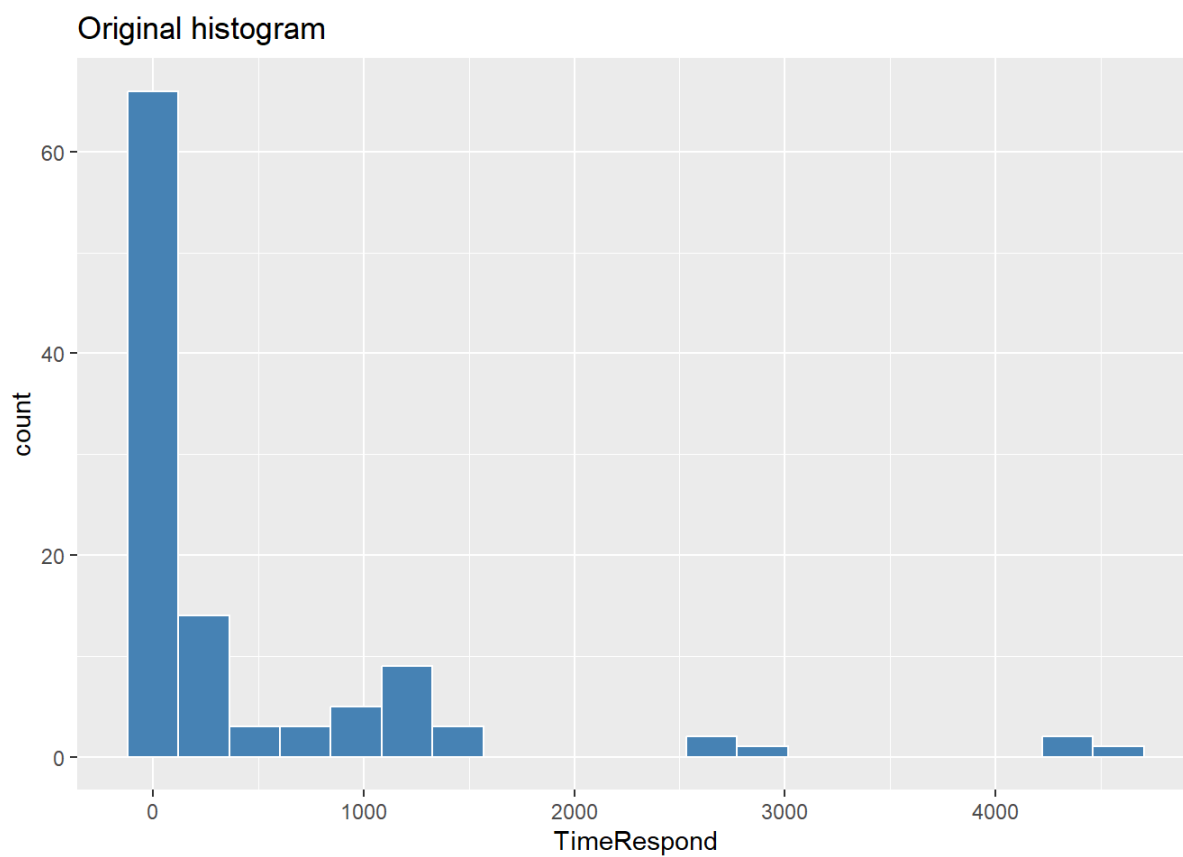
Dataframe per factor combination

```
# dataframe for each factor combination
datos_CP_clean <- data_clean %>%
  filter(
    CloseFriends == "Yes",
    MessageType == "Personalized"
  )
datos_NP_clean <- data_clean %>%
  filter(
    CloseFriends == "No",
    MessageType == "Personalized"
  )
datos_CB_clean <- data_clean %>%
  filter(
    CloseFriends == "Yes",
    MessageType == "Basic"
  )
datos_NB_clean <- data_clean %>%
  filter(
    CloseFriends == "No",
    MessageType == "Basic"
  )
```

Histograms

```
# total histogram
ggplot(data_clean, aes(x = TimeRespond)) +
  geom_histogram(fill = "steelblue", color = "white", bins = 20) +
  ggtitle("Original histogram")
```

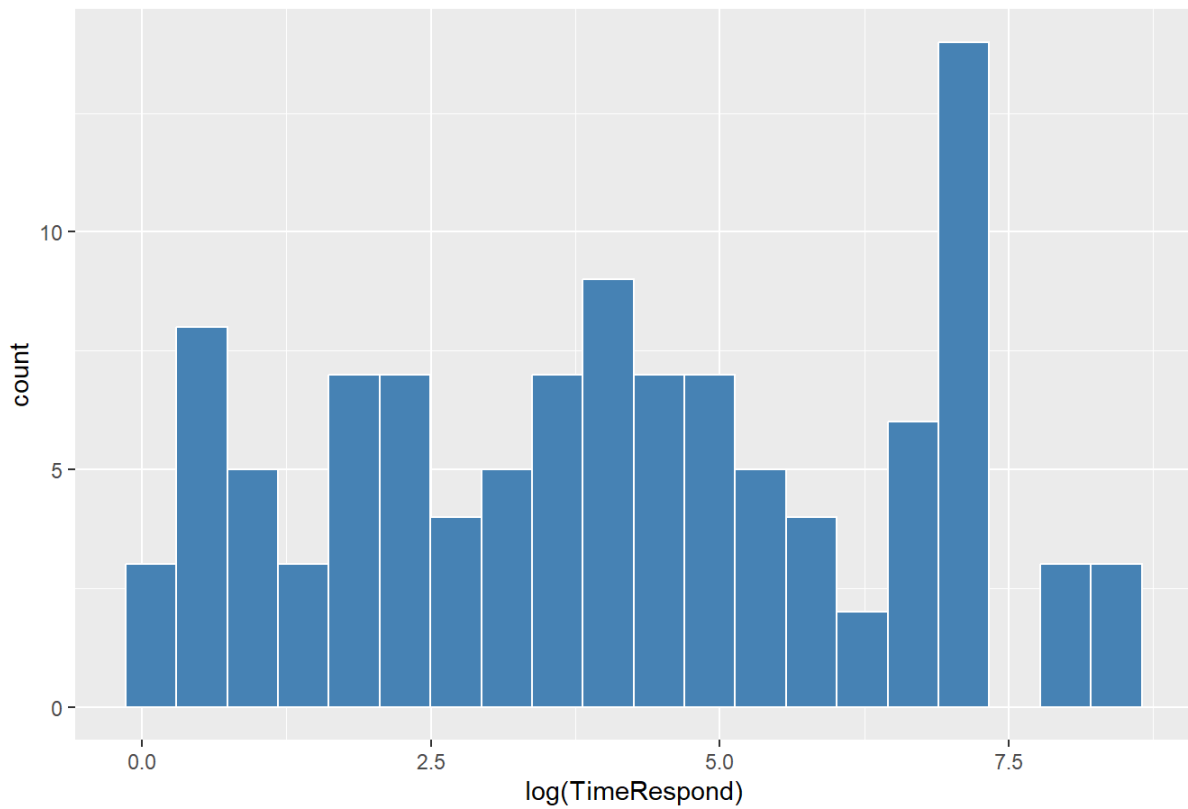
Warning: Removed 29 rows containing non-finite outside the scale range
(`stat\_bin()`).



```
ggplot(data_clean, aes(x = log(TimeRespond))) +  
  geom_histogram(fill = "steelblue", color = "white", bins = 20) +  
  ggtitle("Log histogram")
```

Warning: Removed 29 rows containing non-finite outside the scale range
(`stat\_bin()`).

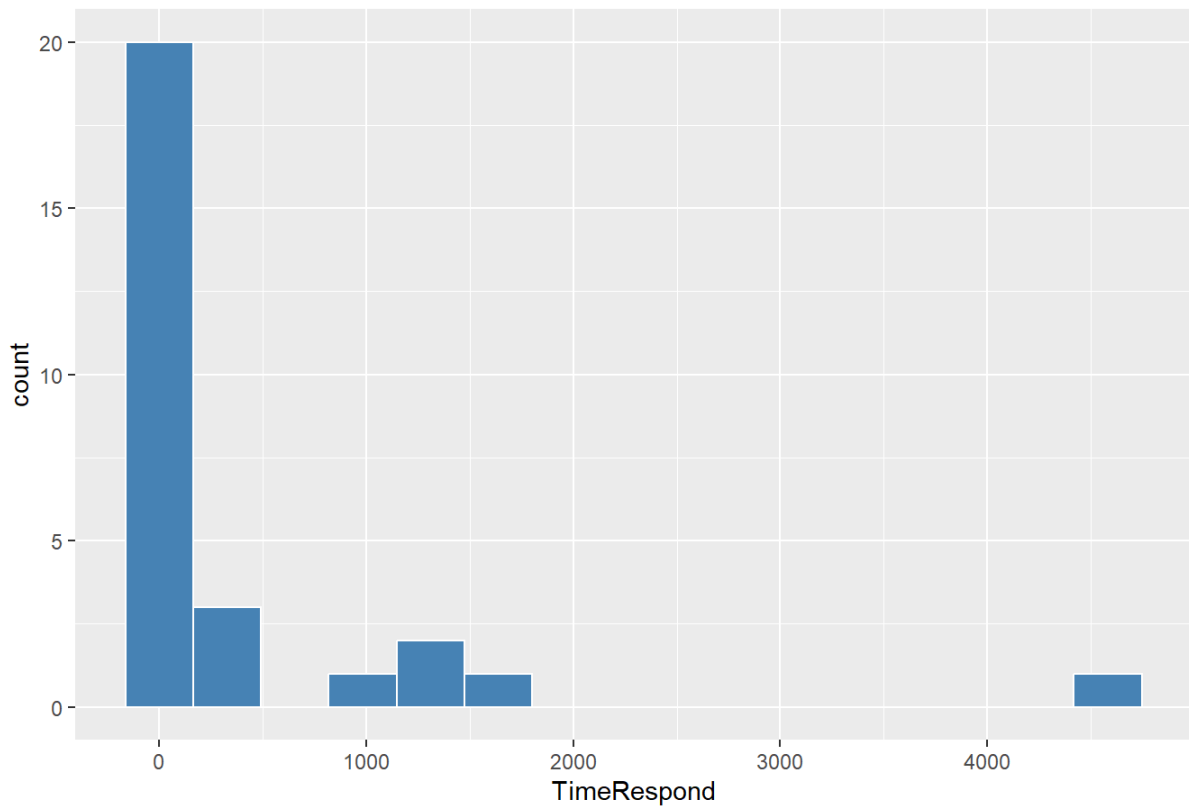
Log histogram



```
# histograms per factor combination
ggplot(datos_CP_clean, aes(x = TimeRespond)) +
  geom_histogram(fill = "steelblue", color = "white", bins = 15) +
  ggtitle("Close Friends + Personalized histogram")
```

Warning: Removed 5 rows containing non-finite outside the scale range (``stat_bin()``).

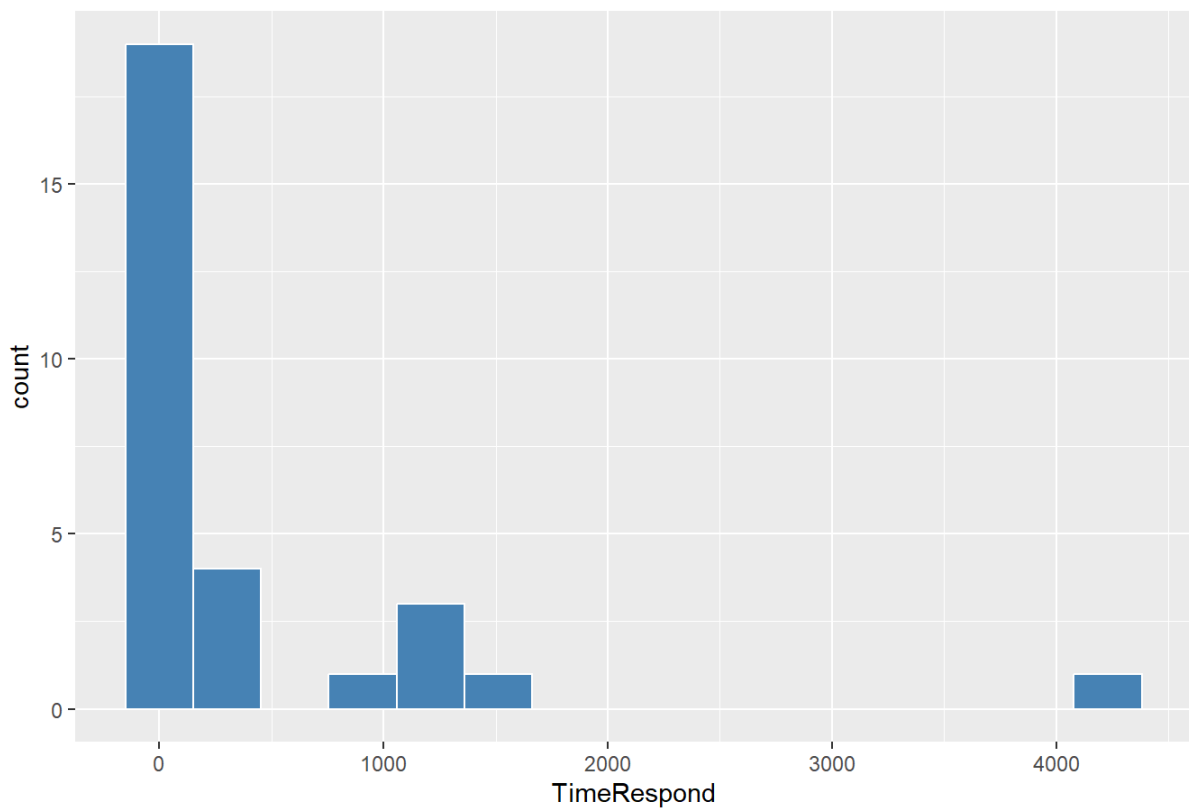
Close Friends + Personalized histogram



```
ggplot(datos_CB_clean, aes(x = TimeRespond)) +  
  geom_histogram(fill = "steelblue", color = "white", bins = 15) +  
  ggtitle("Close Friends + Basic histogram")
```

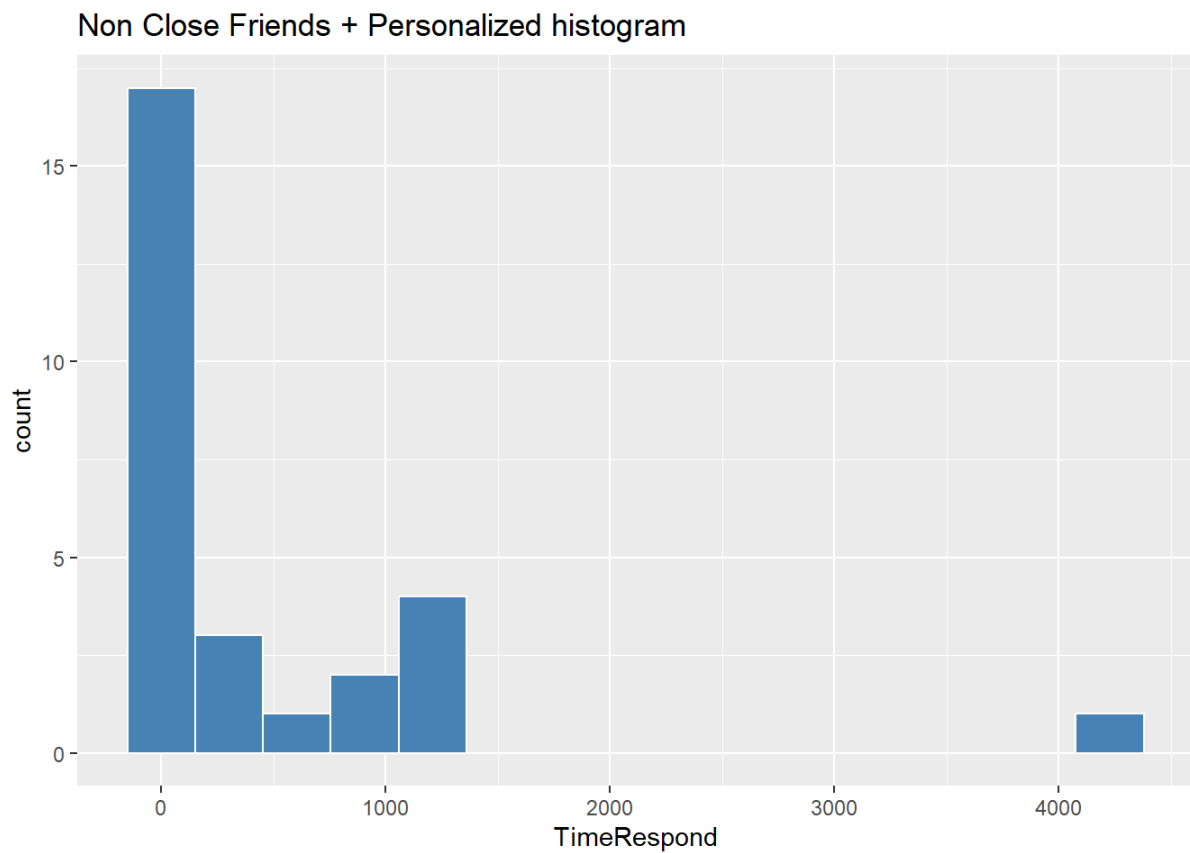
Warning: Removed 4 rows containing non-finite outside the scale range
(`stat\_bin()`).

Close Friends + Basic histogram



```
ggplot(datos_NP_clean, aes(x = TimeRespond)) +
  geom_histogram(fill = "steelblue", color = "white", bins = 15) +
  ggtitle("Non Close Friends + Personalized histogram")
```

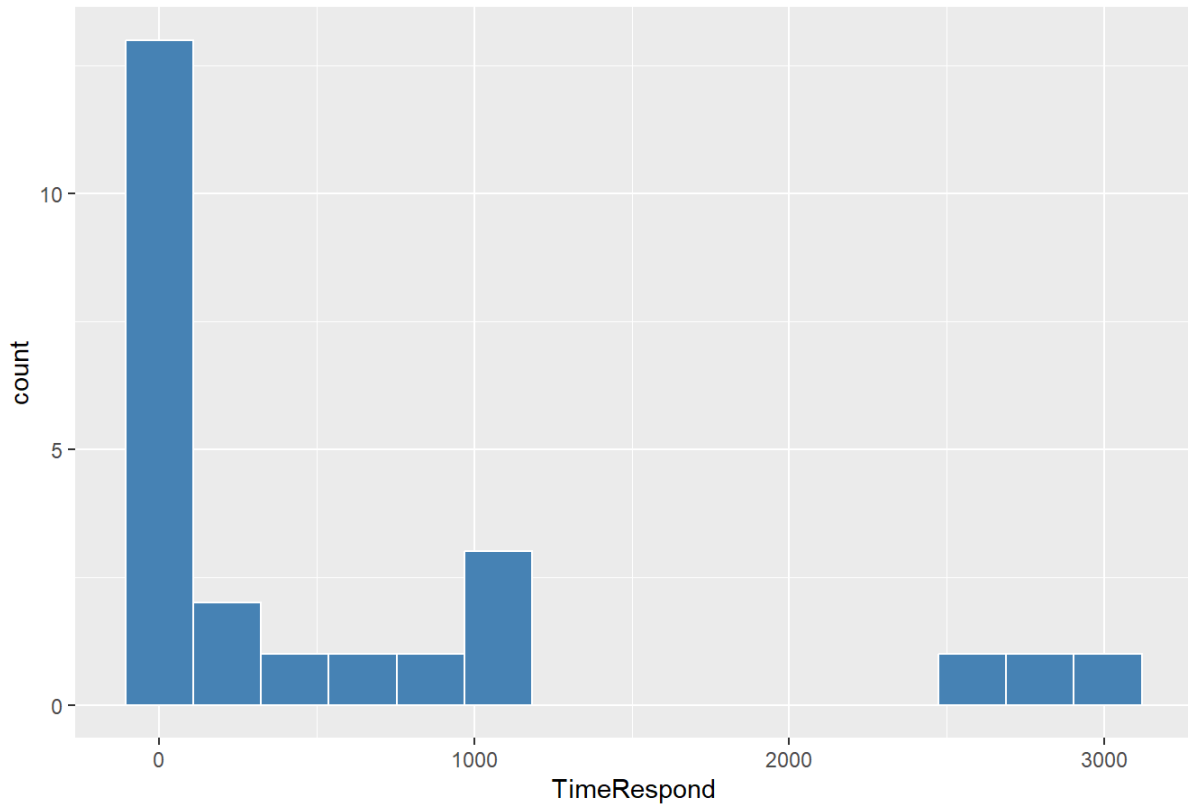
Warning: Removed 8 rows containing non-finite outside the scale range
(`stat\_bin()`).



```
ggplot(datos_NB_clean, aes(x = TimeRespond)) +
  geom_histogram(fill = "steelblue", color = "white", bins = 15) +
  ggtitle("Non Close Friends + Basic histogram")
```

Warning: Removed 12 rows containing non-finite outside the scale range
(`stat\_bin()`).

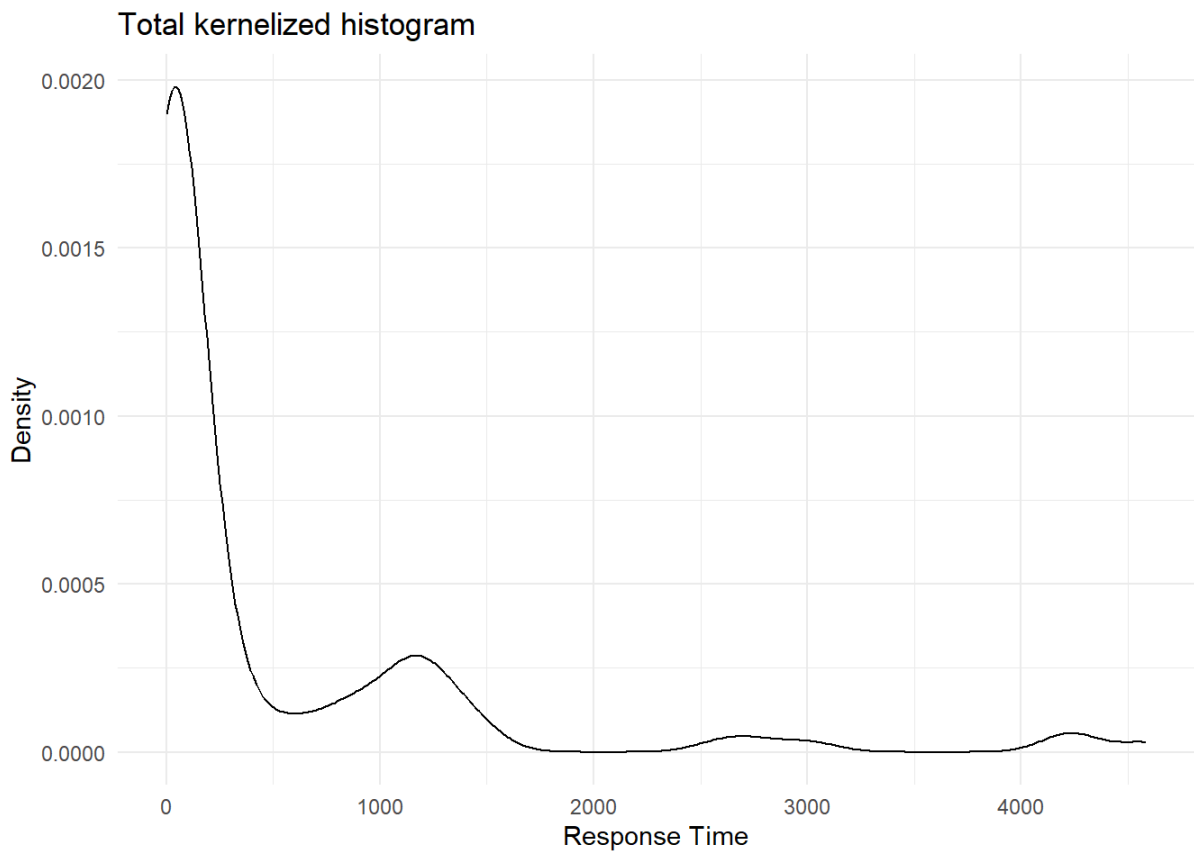
Non Close Friends + Basic histogram



Kernelized histograms

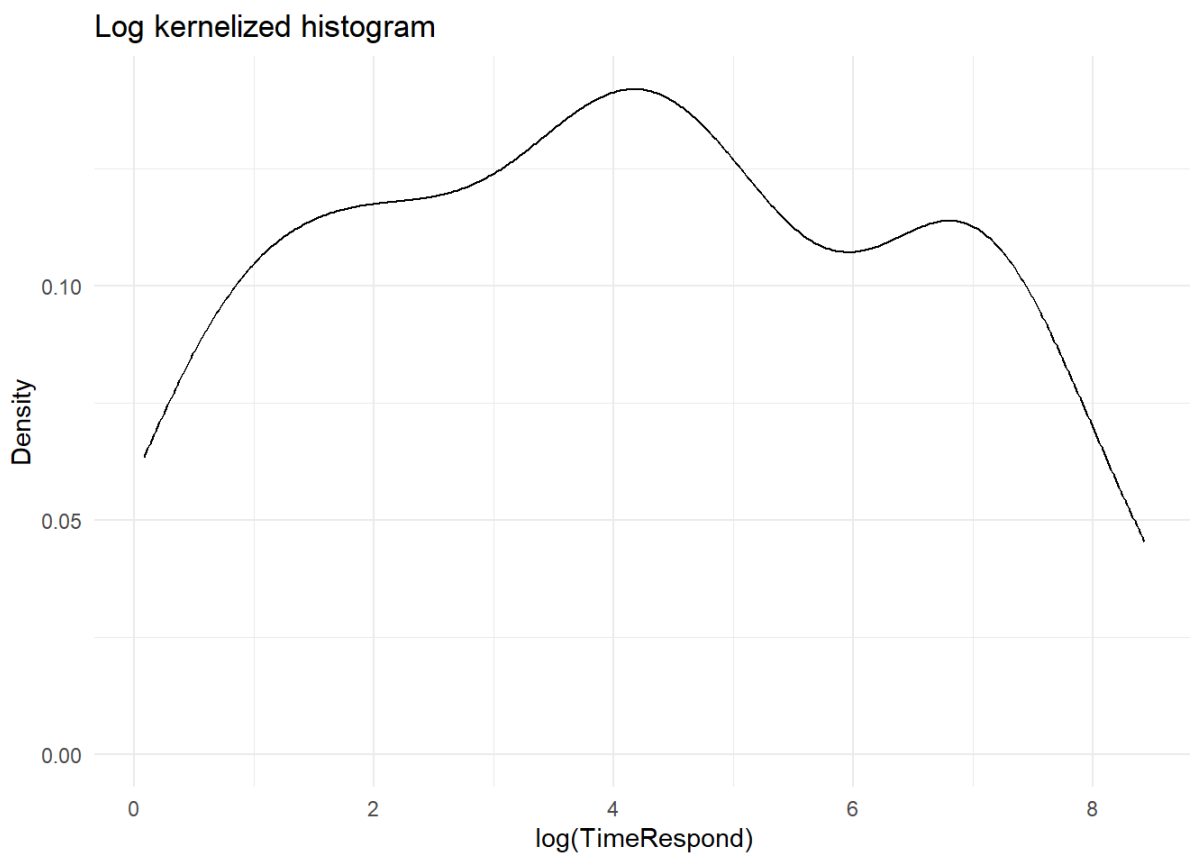
```
# kernelized kistogram total
ggplot(data_clean, aes(x = TimeRespond)) +
  geom_density(alpha = 0.5) + # alpha makes the fill transparent so you can see overlaps
  labs(
    title = "Kernel Density of TimeRespond by Factor Combination",
    x = "Response Time",
    y = "Density",
    fill = "Factor Combination"
  ) +
  theme_minimal() +
  ggtitle("Total kernelized histogram")
```

Warning: Removed 29 rows containing non-finite outside the scale range
(`stat\_density()`).



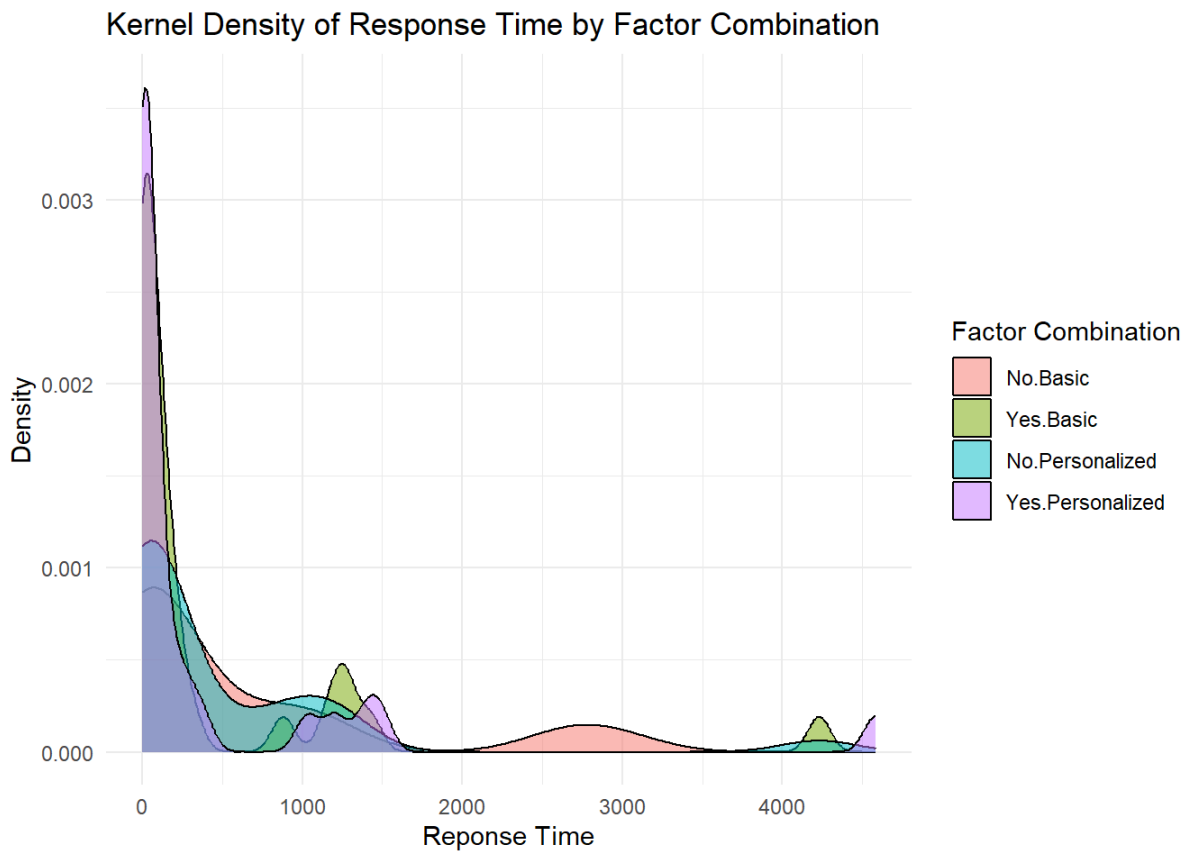
```
# kernelized kistogram log
ggplot(data_clean, aes(x = log(TimeRespond))) +
  geom_density(alpha = 0.5) + # alpha makes the fill transparent so you can see overlaps
  labs(
    title = "Kernel Density of log(TimeRespond) by Factor Combination",
    x = "log(TimeRespond)",
    y = "Density",
    fill = "Factor Combination"
  ) +
  theme_minimal() +
  ggtitle("Log kernelized histogram")
```

Warning: Removed 29 rows containing non-finite outside the scale range (``stat_density()``).



```
# combined kernelized histogram
ggplot(data_clean, aes(x = TimeRespond, fill = interaction(CloseFriends, MessageType))) +
  geom_density(alpha = 0.5) + # alpha makes the fill transparent so you can see overlaps
  labs(
    title = "Kernel Density of Response Time by Factor Combination",
    x = "Reponse Time",
    y = "Density",
    fill = "Factor Combination"
  ) +
  theme_minimal()
```

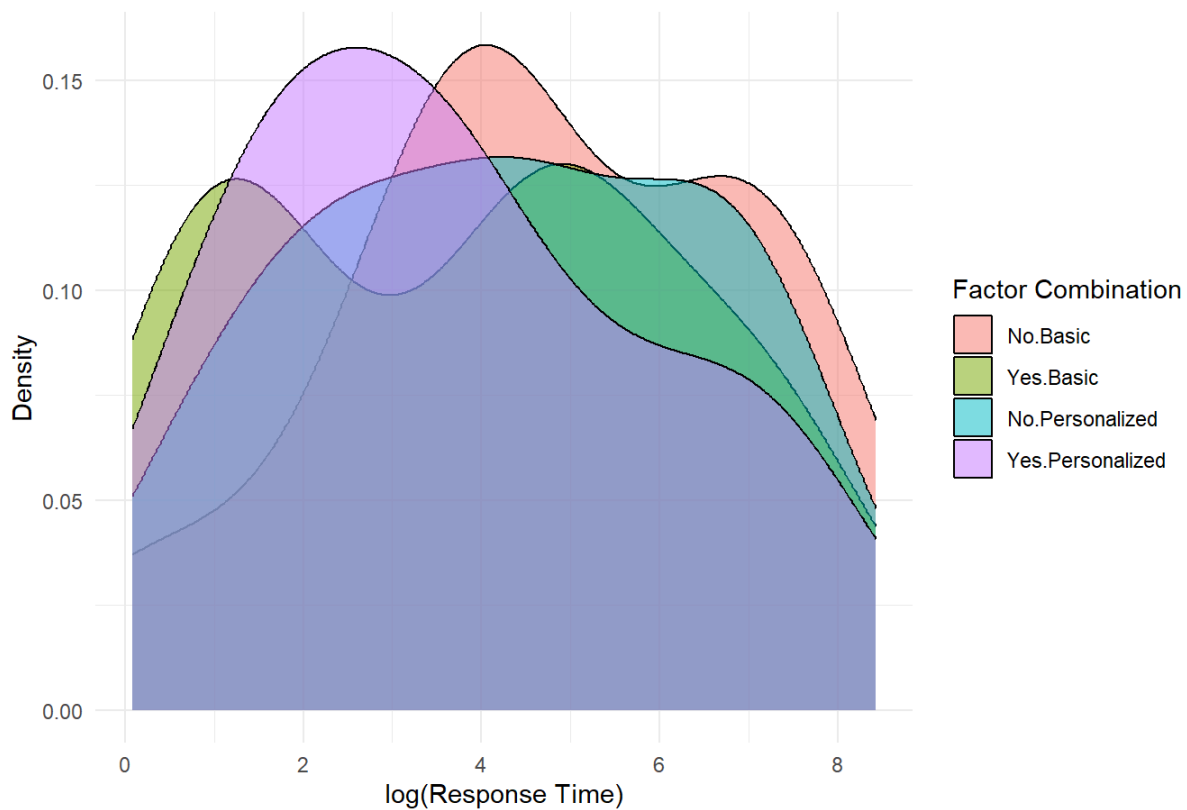
Warning: Removed 29 rows containing non-finite outside the scale range
(`stat\_density()`).



```
ggplot(data_clean, aes(x = log(TimeRespond), fill = interaction(CloseFriends, MessageType))) +
  geom_density(alpha = 0.5) + # alpha makes the fill transparent so you can see overlaps
  labs(
    title = "Kernel Density of log(Response Time) by Factor Combination",
    x = "log(Response Time)",
    y = "Density",
    fill = "Factor Combination"
  ) +
  theme_minimal()
```

Warning: Removed 29 rows containing non-finite outside the scale range (``stat_density()``).

Kernel Density of log(Response Time) by Factor Combination



Count number of observations per factor combination

```
data_clean %>%  
  count(CloseFriends, MessageType)
```

	CloseFriends	MessageType	n
1	No	Basic	36
2	No	Personalized	36
3	Yes	Basic	33
4	Yes	Personalized	33

Delete N/A

```
datos_compressed_na <- data_clean %>%  
  drop_na(TimeRespond)
```

```
datos_compressed_na %>%  
  count(CloseFriends, MessageType)
```

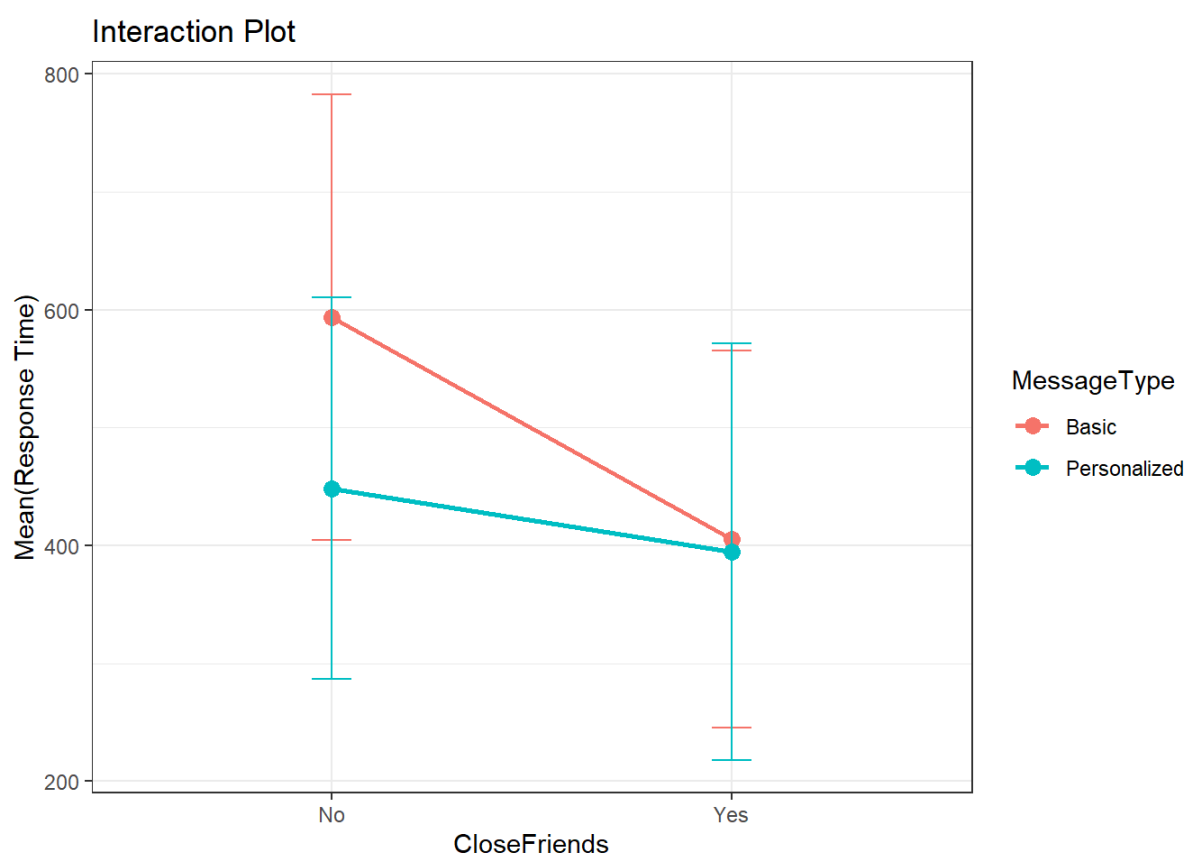
	CloseFriends	MessageType	n
1	No	Basic	24
2	No	Personalized	28
3	Yes	Basic	29
4	Yes	Personalized	28

Interaction plot

Original data

```
summary_df <- datos_compressed_na %>%
  group_by(CloseFriends, MessageType) %>%
  summarise(
    mean_response = mean(TimeRespond),
    se = sd(TimeRespond) / sqrt(n()),
    .groups = 'drop'
  )
```

```
ggplot(summary_df, aes(x = CloseFriends, y = mean_response, color = MessageType, group = MessageType)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean_response - se, ymax = mean_response + se), width = 0.1) +
  labs(title = "Interaction Plot",
       x = "CloseFriends",
       y = "Mean(Response Time)") +
  theme_bw()
```

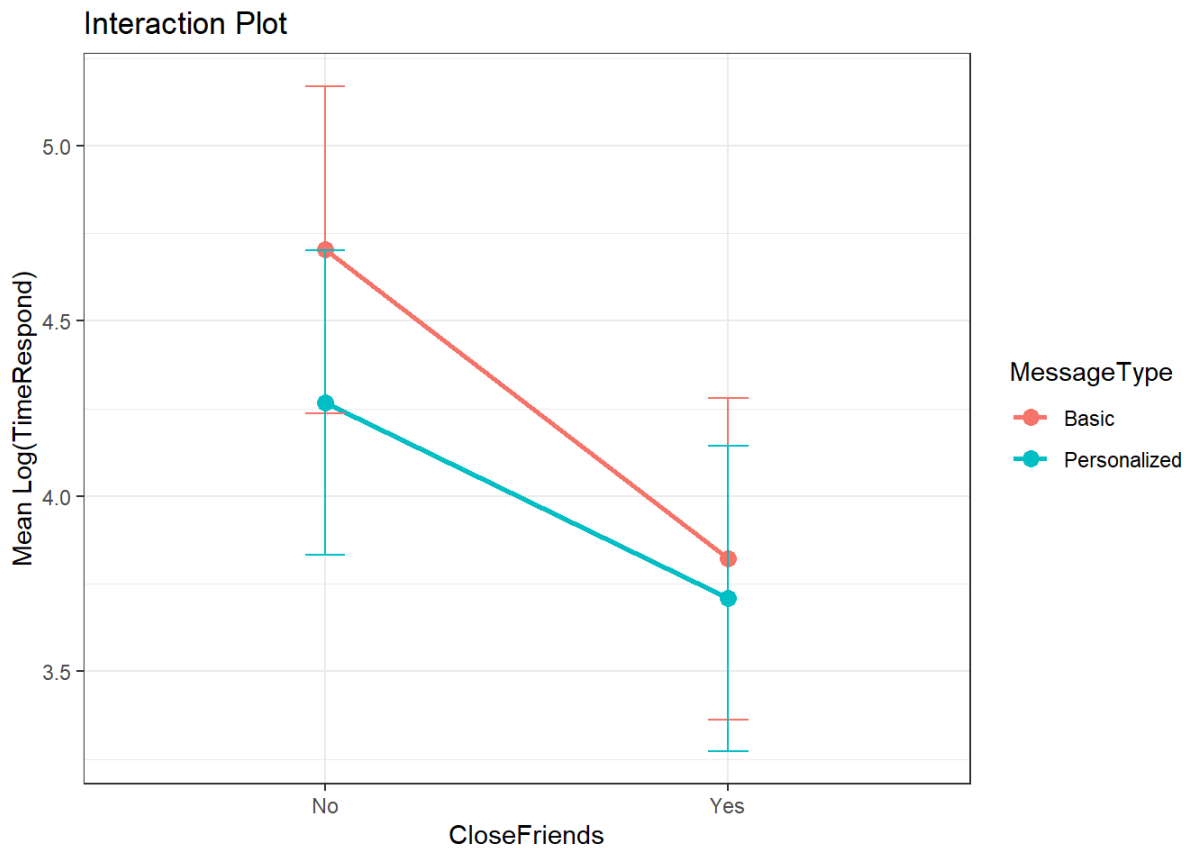


Data after logarithmic transform

```
summary_df_log <- datos_compressed_na %>%
  group_by(CloseFriends, MessageType) %>%
  summarise(
    mean_response_log = mean(log(TimeRespond)),
    se = sd(log(TimeRespond)) / sqrt(n()),
    .groups = 'drop'
  )
```

```
ggplot(summary_df_log, aes(x = CloseFriends, y = mean_response_log, color = MessageType, group = MessageType)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean_response_log - se, ymax = mean_response_log + se), width = 0.1) +
```

```
labs(title = "Interaction Plot",
     x = "CloseFriends",
     y = "Mean Log(TimeRespond)") +
theme_bw()
```



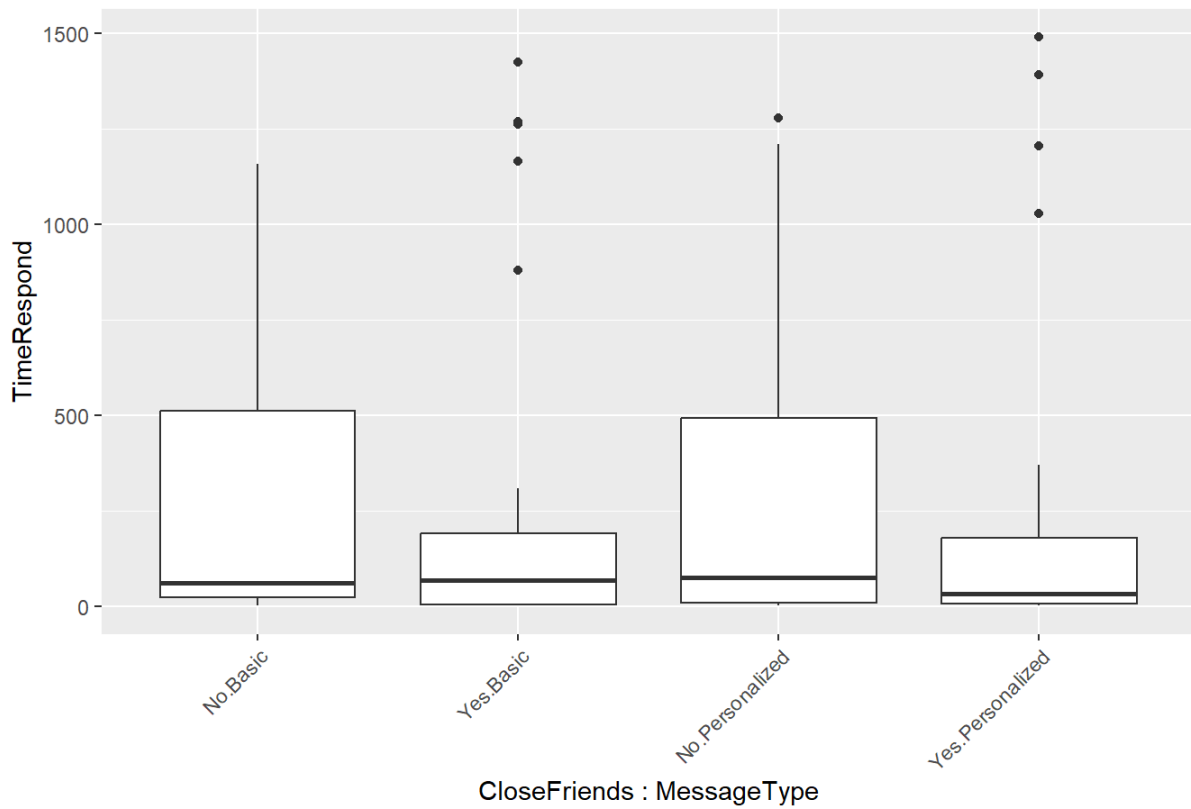
OUTLIER ELIMINATION

We delete all observations where TimeRespond > 2000 (can be seen as anomalies looking at the histogram - people who didn't answer in the first two days).

```
# this also deleted N/A
datos_compressed_outliers <- data_clean %>%
  filter(
    TimeRespond < 2000
  )
```

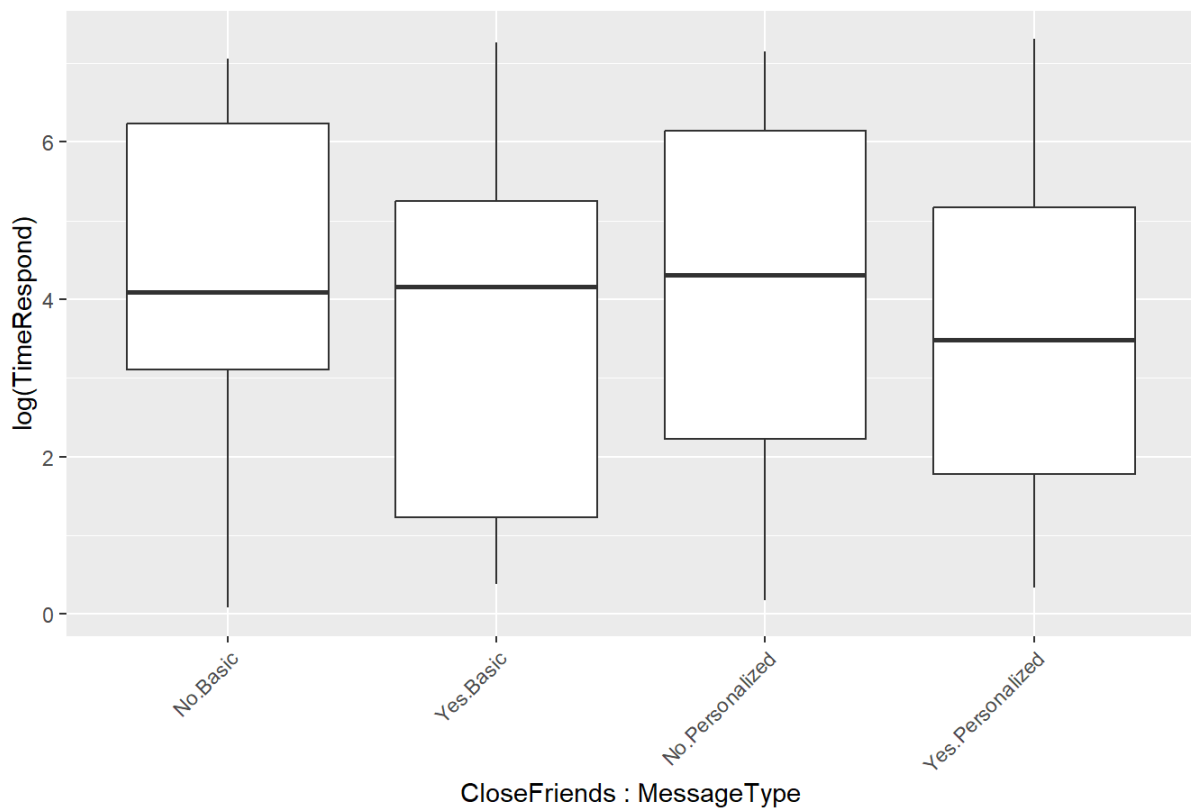
```
ggplot(datos_compressed_outliers, aes(x = interaction(CloseFriends, MessageType), y = TimeRespond))
  geom_boxplot() +
  labs(title = "Data without outliers boxplot", x = "CloseFriends : MessageType") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Data without outliers boxplot



```
ggplot(datos_compressed_outliers, aes(x = interaction(CloseFriends, MessageType), y = log(TimeRespo
geom_boxplot() +
labs(title = "Log data without outliers boxplot", x = "CloseFriends : MessageType") +
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Log data without outliers boxplot



Count number of observations per factor combination

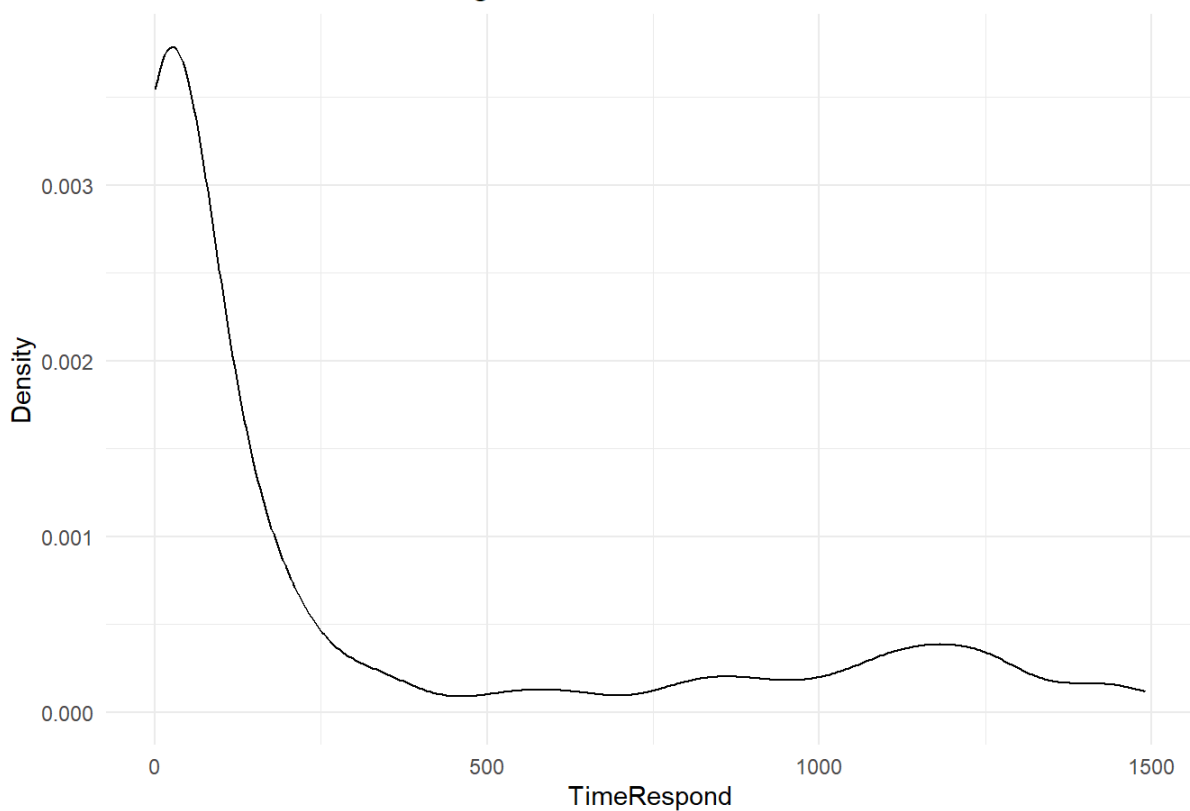
```
# Simple count
datos_compressed_outliers %>%
  count(CloseFriends, MessageType)
```

	CloseFriends	MessageType	n
1	No	Basic	21
2	No	Personalized	27
3	Yes	Basic	28
4	Yes	Personalized	27

Kernelized histograms

```
# kernelized kistogram total
ggplot(datos_compressed_outliers, aes(x = TimeRespond)) +
  geom_density(alpha = 0.5) + # alpha makes the fill transparent so you can see overlaps
  labs(
    title = "Kernel Density of TimeRespond by Factor Combination",
    x = "TimeRespond",
    y = "Density",
    fill = "Factor Combination"
  ) +
  theme_minimal() +
  ggtitle("No outliers kernelized histogram")
```

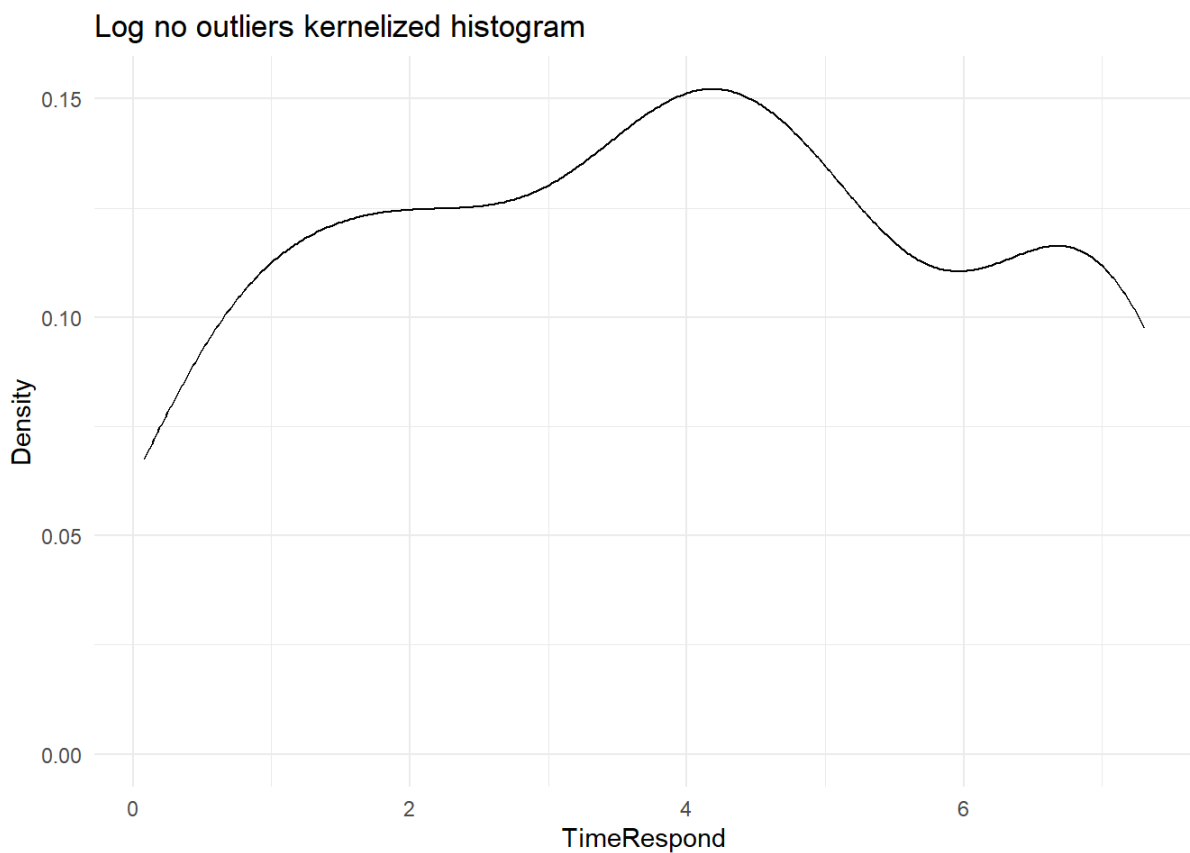
No outliers kernelized histogram



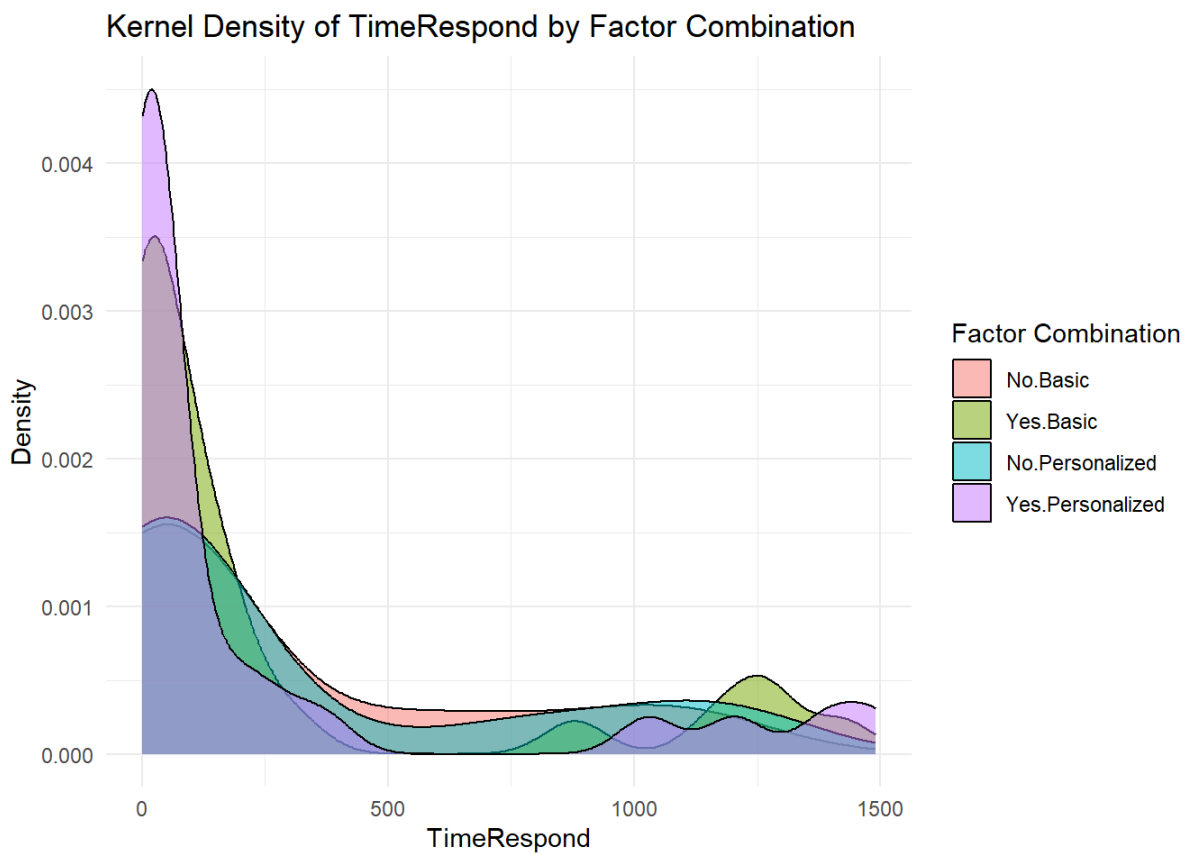
```
# kernelized kistogram log
ggplot(datos_compressed_outliers, aes(x = log(TimeRespond))) +
  geom_density(alpha = 0.5) + # alpha makes the fill transparent so you can see overlaps
  labs(
    title = "Kernel Density of log(TimeRespond) by Factor Combination",
```



```
x = "TimeRespond",
y = "Density",
fill = "Factor Combination"
) +
theme_minimal() +
ggtitle("Log no outliers kernelized histogram")
```

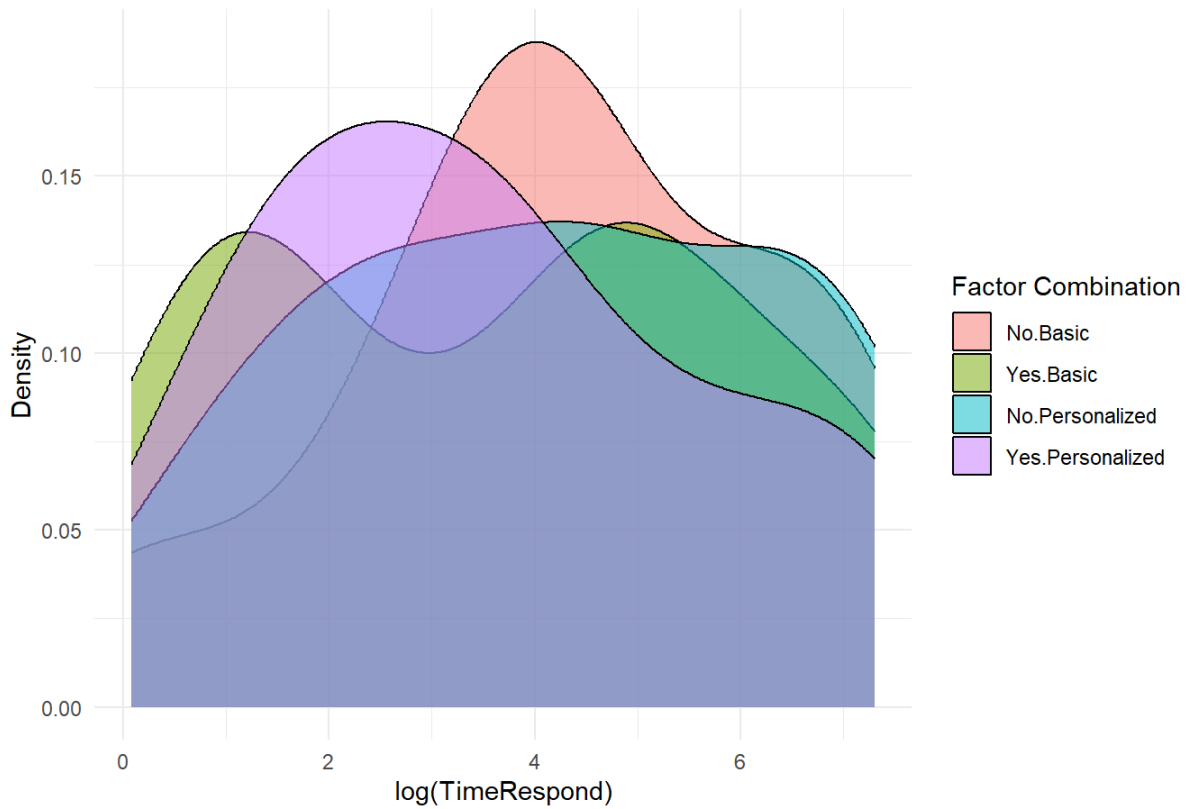


```
# combined kernelized histogram
ggplot(datos_compressed_outliers, aes(x = TimeRespond, fill = interaction(CloseFriends, MessageType)
  geom_density(alpha = 0.5) + # alpha makes the fill transparent so you can see overlaps
  labs(
    title = "Kernel Density of TimeRespond by Factor Combination",
    x = "TimeRespond",
    y = "Density",
    fill = "Factor Combination"
  ) +
  theme_minimal()
```



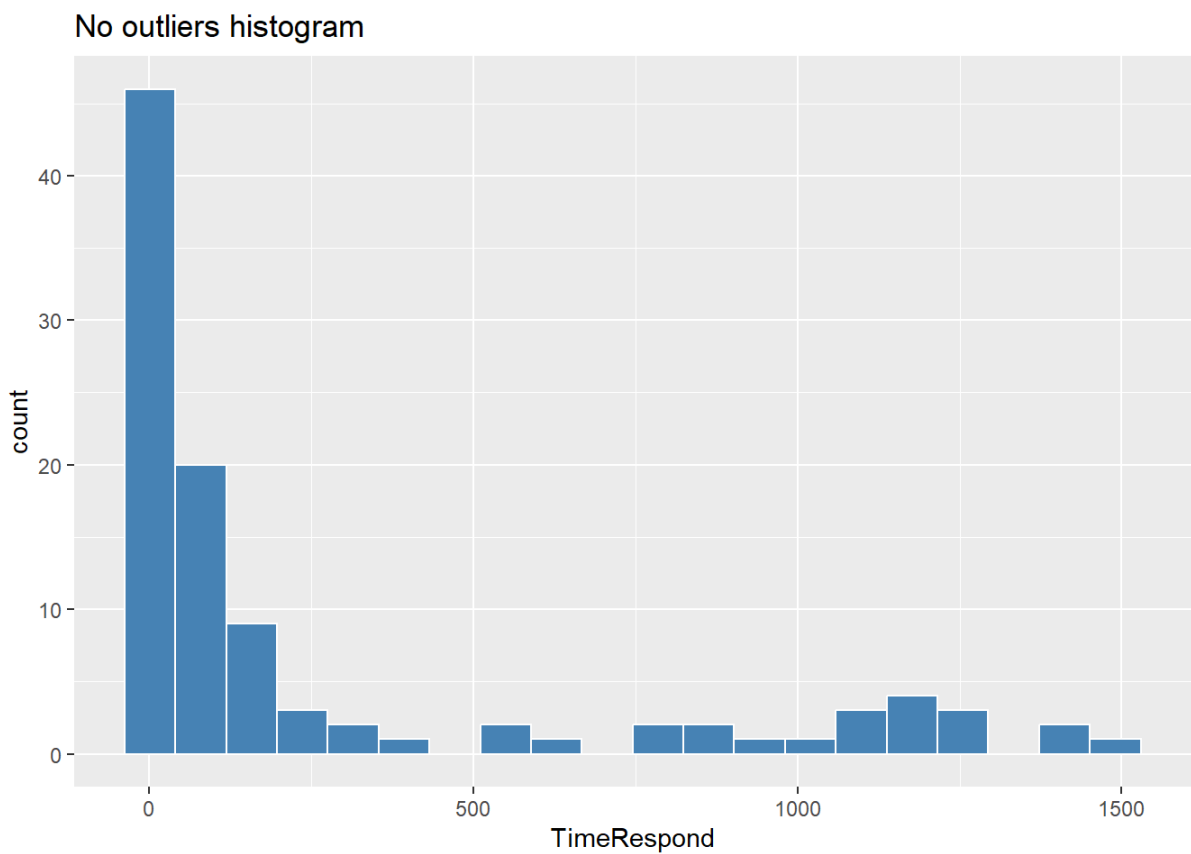
```
ggplot(datos_compressed_outliers, aes(x = log(TimeRespond), fill = interaction(CloseFriends, Message))) +  
  geom_density(alpha = 0.5) + # alpha makes the fill transparent so you can see overlaps  
  labs(  
    title = "Kernel Density of log(TimeRespond) by Factor Combination",  
    x = "log(TimeRespond)",  
    y = "Density",  
    fill = "Factor Combination"  
  ) +  
  theme_minimal()
```

Kernel Density of log(TimeRespond) by Factor Combination

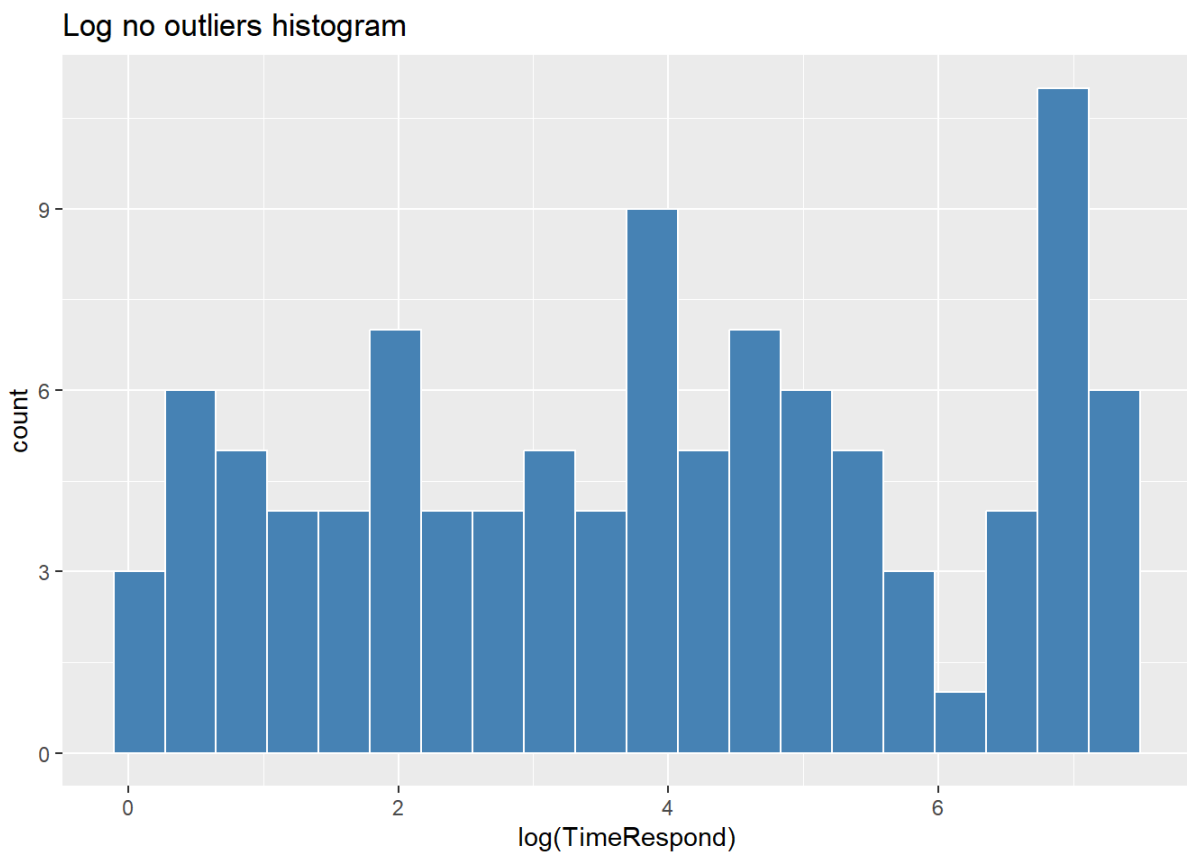


Histograms

```
# total histogram
ggplot(datos_compressed_outliers, aes(x = TimeRespond)) +
  geom_histogram(fill = "steelblue", color = "white", bins = 20) +
  ggtitle("No outliers histogram")
```



```
ggplot(datos_compressed_outliers, aes(x = log(TimeRespond))) +
  geom_histogram(fill = "steelblue", color = "white", bins = 20) +
  ggtitle("Log no outliers histogram")
```



SAMPLE SIZE

```
# Calculate power for a single effect (Main Effect or Interaction)
analysis <- pwr.f2.test(u = 1, f2 = 0.0625, sig.level = 0.05, power = 0.80)
print(analysis)
```

Multiple regression power calculation

```
u = 1
v = 125.5312
f2 = 0.0625
sig.level = 0.05
power = 0.8
```

```
# Calculate total N needed for the whole 2x2 design
# N = v + u + 1
total_n <- ceiling(analysis$v + analysis$u + 1)
cat("Required Total Sample Size:", total_n)
```

Required Total Sample Size: 128

PERCENTAGE OF ANSWERS PER FACTOR COMBINATION

Chi-SQ test for CloseFriends

Does being a close friend have any effect on the probability of answering?

```
table_CloseFriends <- table(data_clean$CloseFriends, !is.na(data_clean$TimeRespond))
table_CloseFriends
```

	FALSE	TRUE
No	20	52
Yes	9	57

We can see that a higher percentage of the close friends answered. Is this difference significant?

```
chisq.test(table_CloseFriends)
```

Pearson's Chi-squared test with Yates' continuity correction

```
data: table_CloseFriends
X-squared = 3.3405, df = 1, p-value = 0.06759
```

Not significant.

Chi-SQ test for MessageType

```
table_MessageType <- table(data_clean$MessageType, !is.na(data_clean$TimeRespond))
table_MessageType
```

	FALSE	TRUE
Basic	16	53
Personalized	13	56

We can also see that a higher percentage of the personalized messages answered. Is this difference significant?

```
chisq.test(table_MessageType)
```

Pearson's Chi-squared test with Yates' continuity correction

```
data: table_MessageType
X-squared = 0.17463, df = 1, p-value = 0.676
```

Also not significant.

Logistic Regression

```
logit <- glm(!is.na(data_clean$TimeRespond) ~ CloseFriends * MessageType,
             data = data_clean,
             family = binomial(link = "logit"))
summary(logit)
```

Call:

```
glm(formula = !is.na(data_clean$TimeRespond) ~ CloseFriends *
```

```
MessageType, family = binomial(link = "logit"), data = data_clean)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	0.6931	0.3536	1.961	0.0499 *
CloseFriendsYes	1.2879	0.6399	2.013	0.0442 *
MessageTypePersonalized	0.5596	0.5345	1.047	0.2951
CloseFriendsYes:MessageTypePersonalized	-0.8179	0.8977	-0.911	0.3623

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 141.91 on 137 degrees of freedom
Residual deviance: 136.42 on 134 degrees of freedom
AIC: 144.42

Number of Fisher Scoring iterations: 4

```
# Combined Odds Ratios and 95% Wald CIs  
exp(cbind(OR = coef(logit), confint.default(logit)))
```

	OR	2.5 %	97.5 %
(Intercept)	2.0000000	1.0001954	3.999218
CloseFriendsYes	3.6250000	1.0342358	12.705637
MessageTypePersonalized	1.7500000	0.6138353	4.989123
CloseFriendsYes:MessageTypePersonalized	0.4413793	0.0759746	2.564221

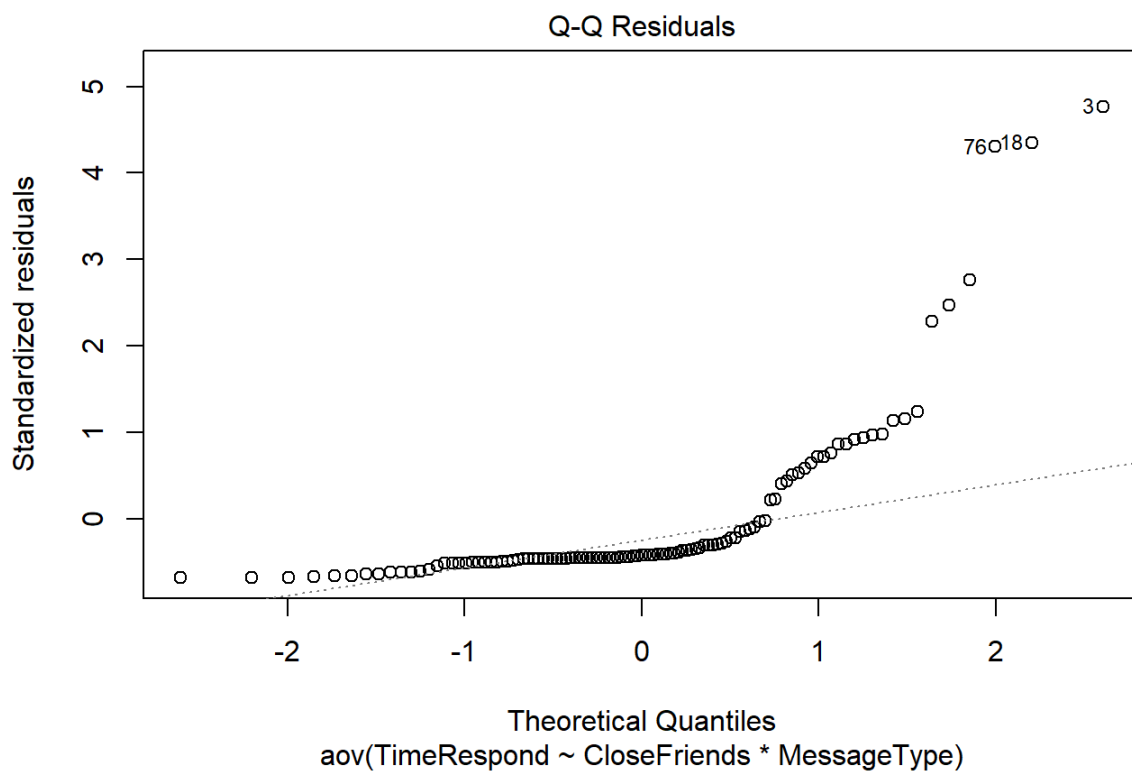
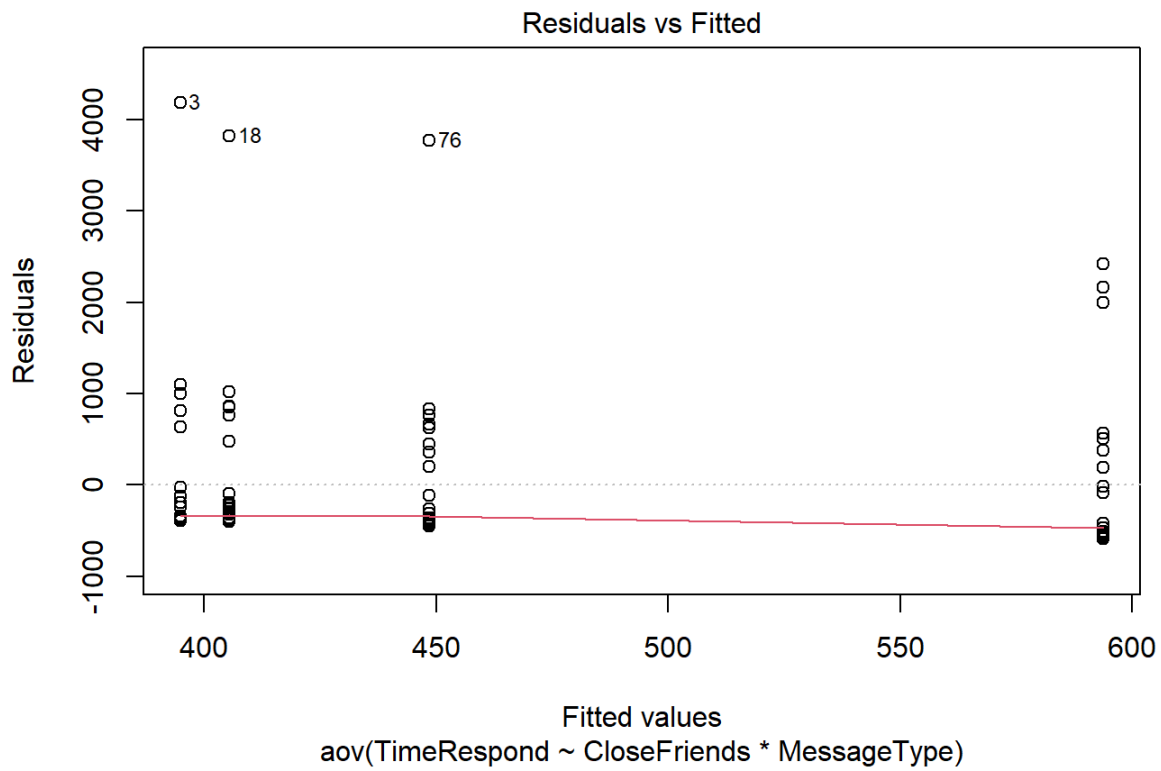
2^2 FACTORIAL EXPERIMENT

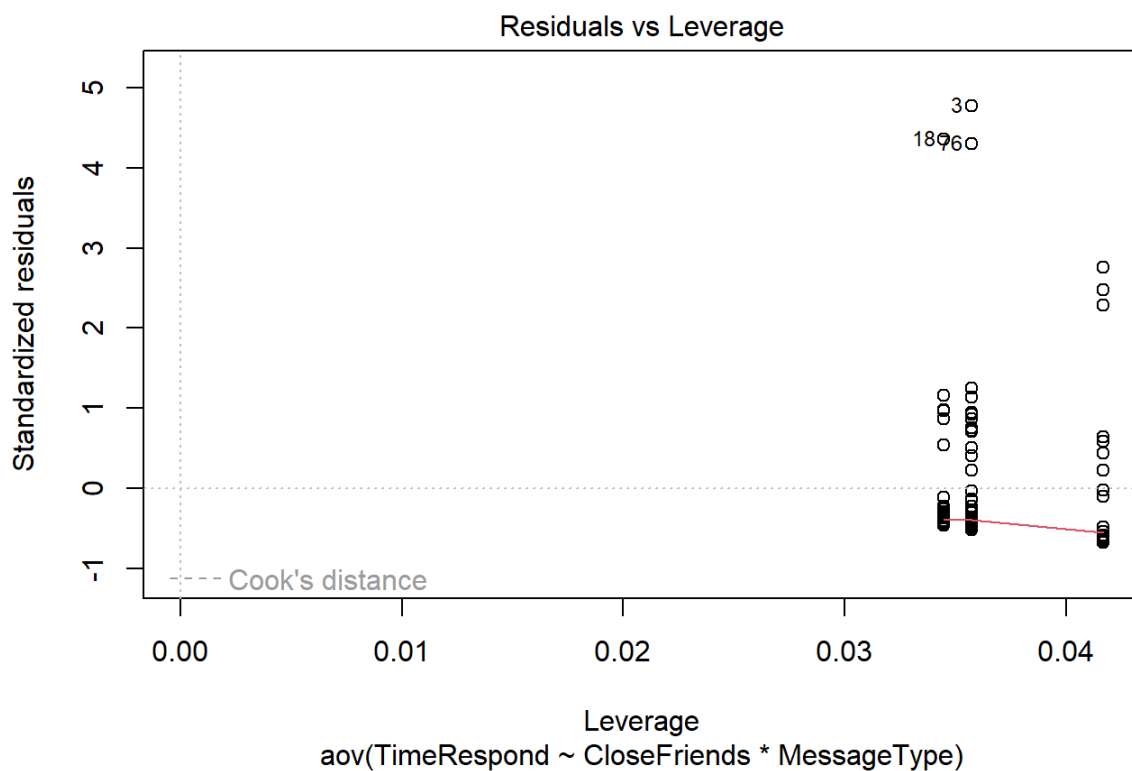
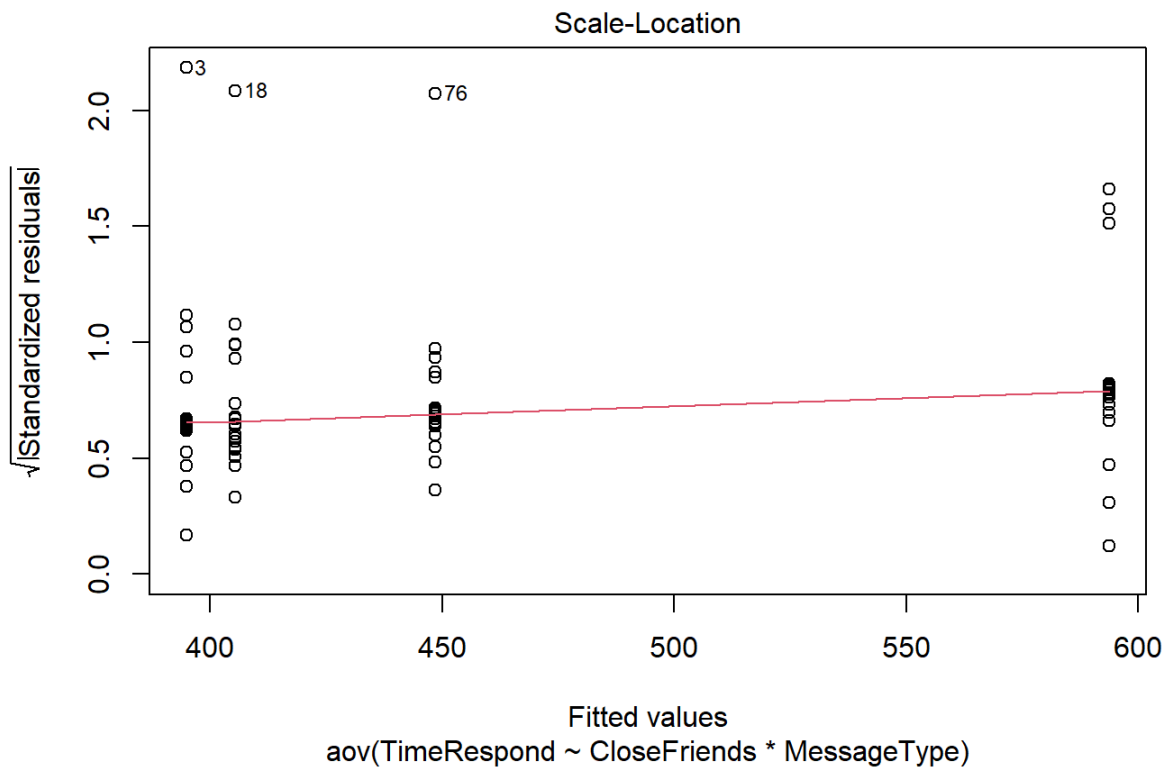
Original dataset

```
options(contrasts=c("contr.sum", "contr.sum"))  
fit_anova_clean <- aov(TimeRespond ~ CloseFriends * MessageType, data=datos_compressed_na)  
summary(fit_anova_clean)
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
CloseFriends	1	360989	360989	0.452	0.503
MessageType	1	151208	151208	0.189	0.665
CloseFriends:MessageType	1	122694	122694	0.153	0.696
Residuals	105	83928623	799320		

```
plot(fit_anova_clean)
```





Residuals seem independent, but they are nowhere near being normal. This was expected due to the nature of time data. We need to apply some transformation to make the data normal.

Logarithmic transform

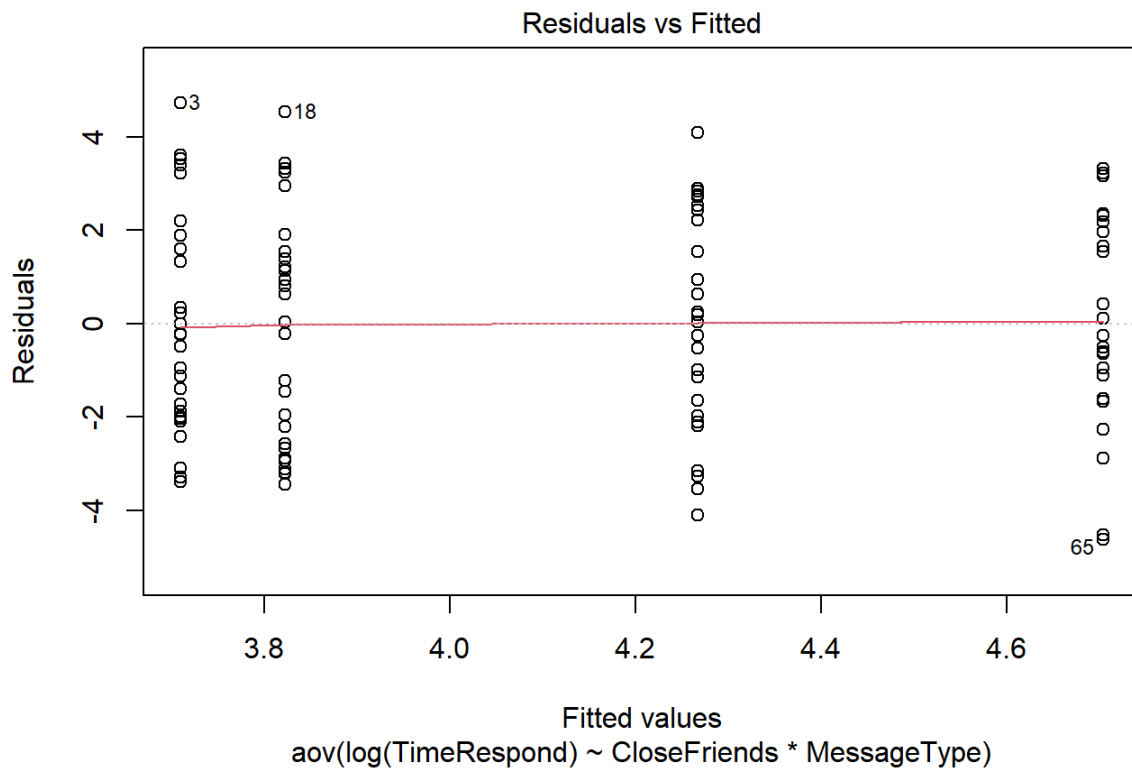
```
options(contrasts=c("contr.sum", "contr.sum"))
fit_anova_clean_log <- aov(log(TimeRespond) ~ CloseFriends * MessageType, data=datos_compressed_na)
```

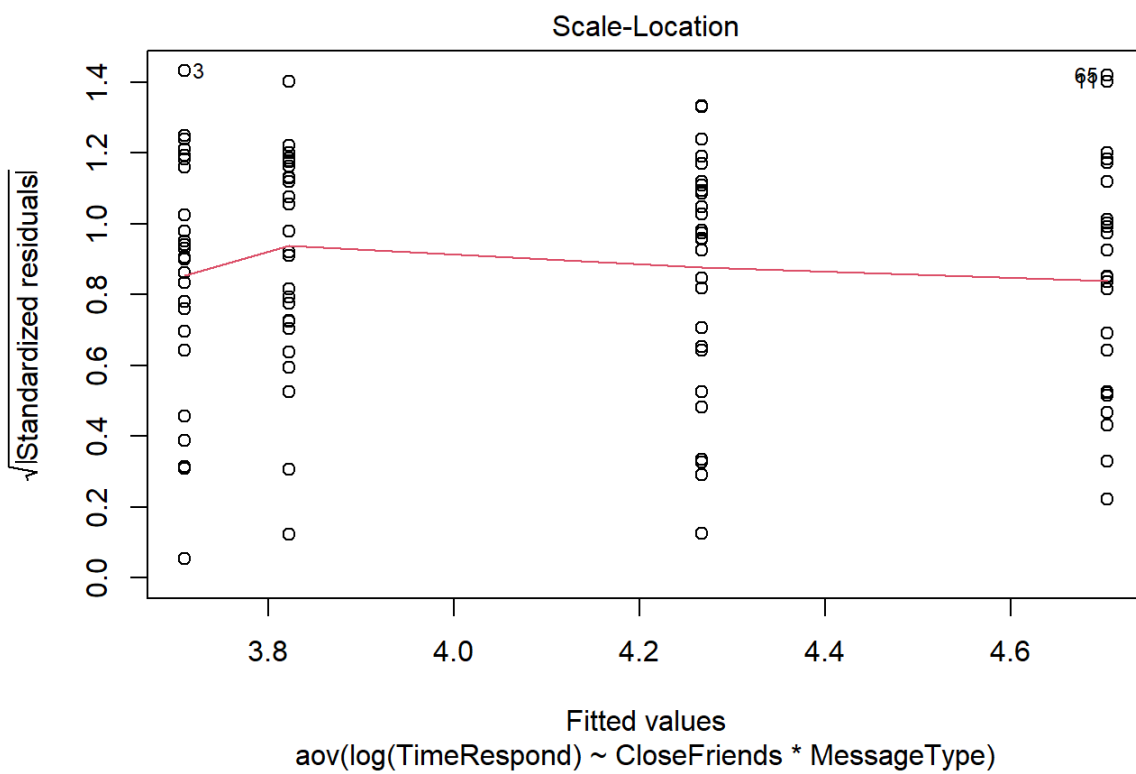
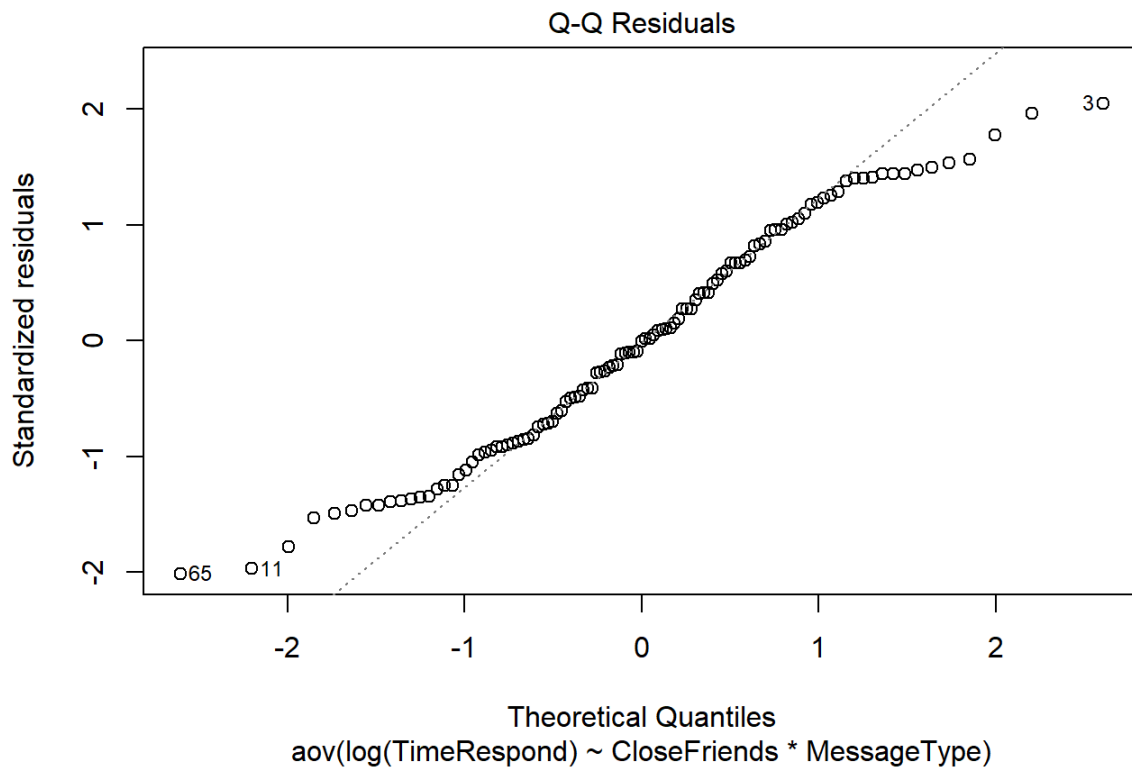


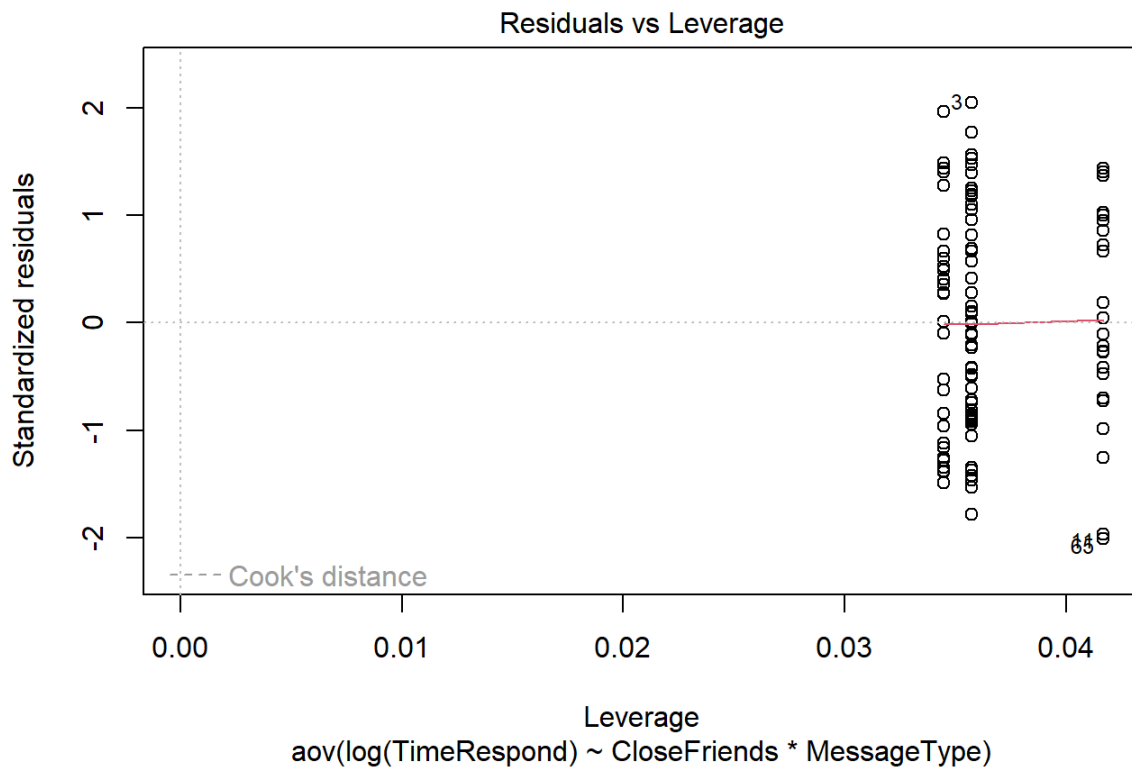
```
summary(fit_anova_clean_log)
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
CloseFriends	1	13.4	13.419	2.439	0.121
MessageType	1	1.9	1.934	0.351	0.555
CloseFriends:MessageType	1	0.7	0.713	0.130	0.720
Residuals	105	577.8	5.503		

```
plot(fit_anova_clean_log)
```







Some improvement, but still not normal. We perform Shapiro-Wilk test to confirm it.

```
shapiro.test(fit_anova_clean_log$residuals)
```

Shapiro-Wilk normality test

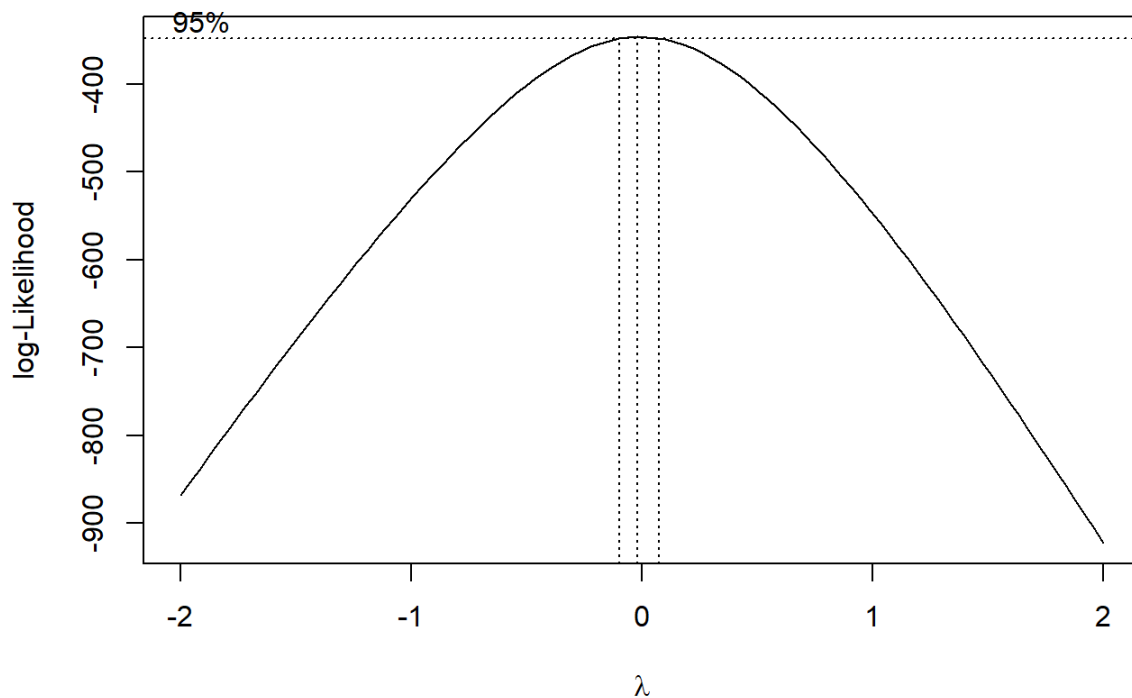
```
data: fit_anova_clean_log$residuals
W = 0.9719, p-value = 0.02085
```

We verify that the data is not normal.

Box-Cox transform

Find the optimal lambda for the boxcox transform. If lambda=0, we apply a logarithmic transform; else $y' = (y^{\lambda}-1)/\lambda$.

```
bc <- boxcox(TimeRespond ~ CloseFriends * MessageType, data=datos_compressed_na)
```



```
lambda <- bc$x[which.max(bc$y)]
```

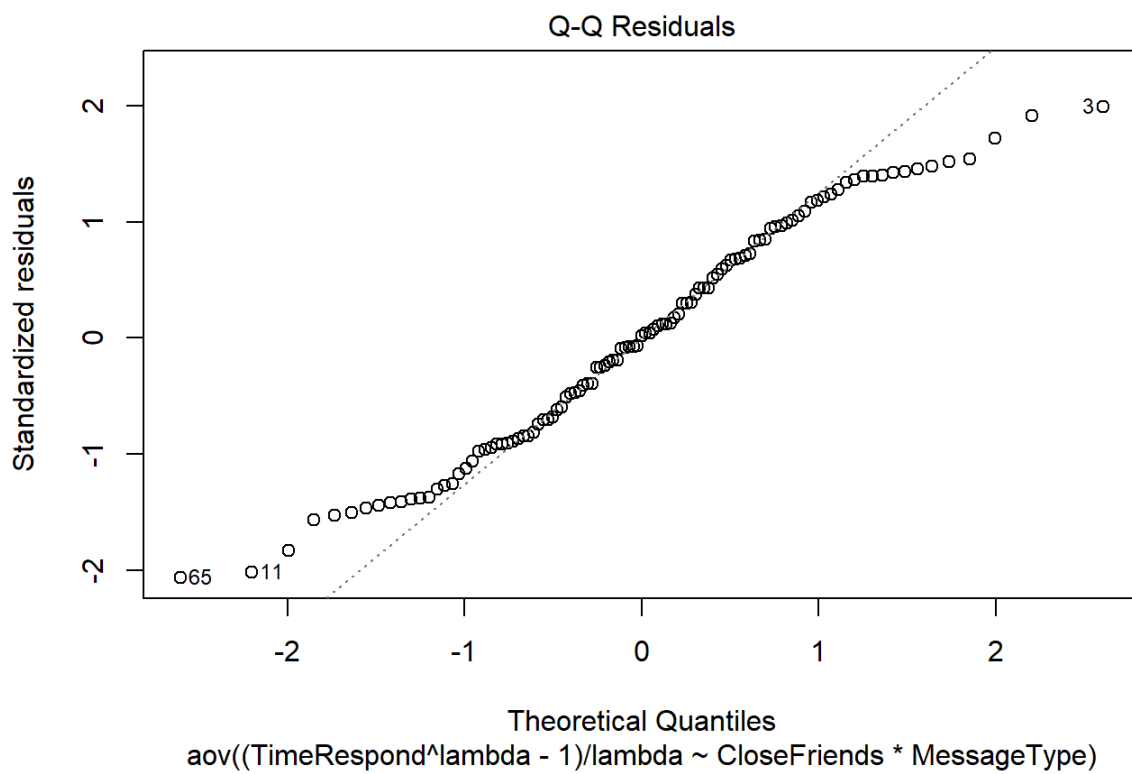
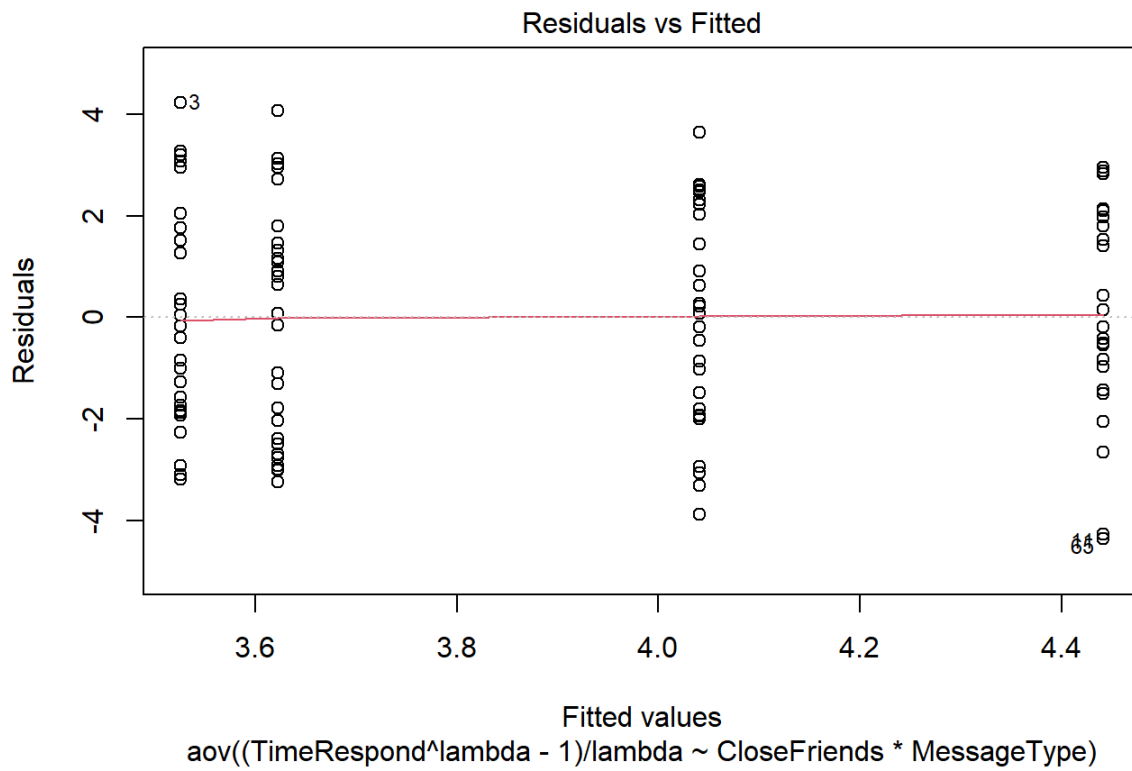
Ideal lambda is almost zero, so the ideal transformation should be a logarithm (which we've already tried and proven it did not bring normality). We will try the power transformation with the value of lambda given as well.

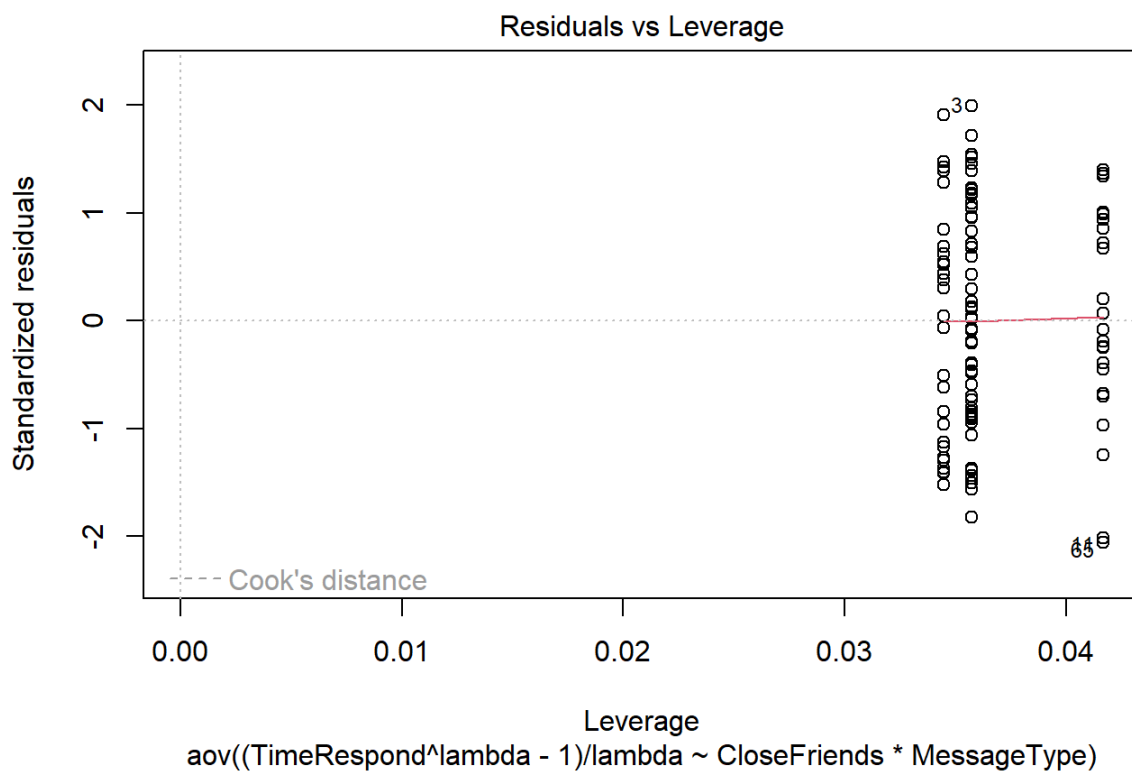
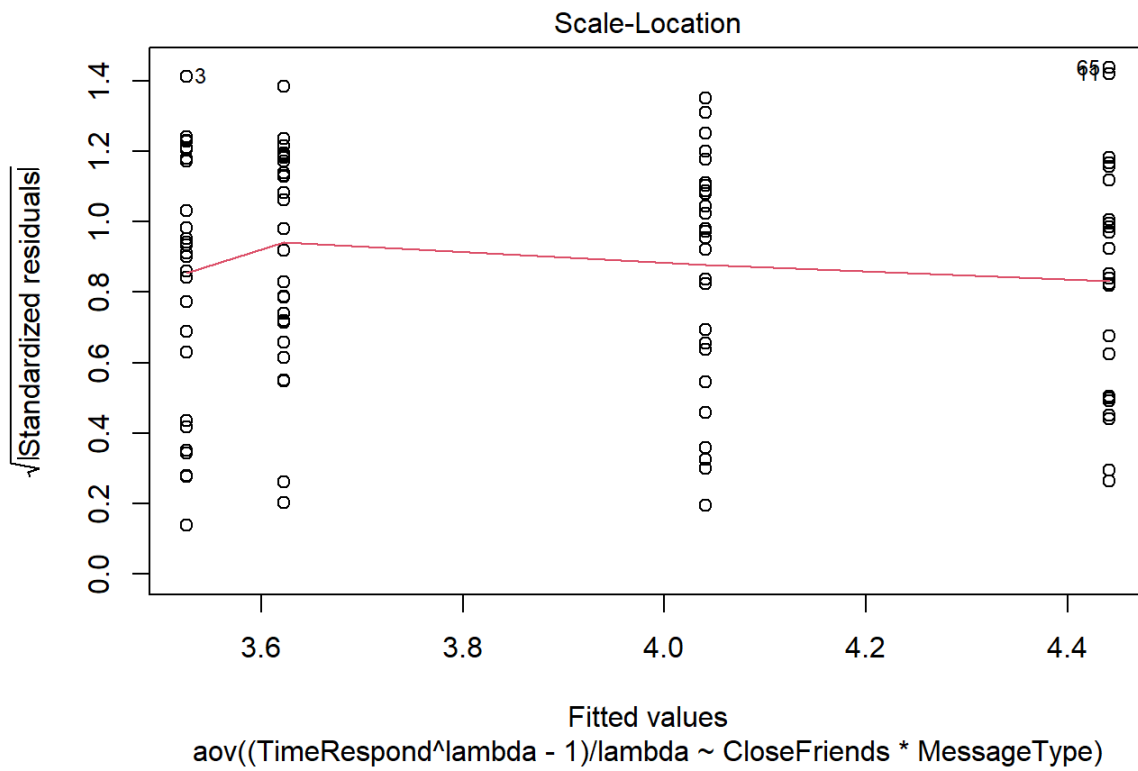
```
fit_anova_clean_boxcox <- aov((TimeRespond^lambda-1)/lambda ~ CloseFriends * MessageType, data=dato)
summary(fit_anova_clean_boxcox)
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
CloseFriends	1	11.5	11.517	2.470	0.119
MessageType	1	1.6	1.578	0.338	0.562
CloseFriends:MessageType	1	0.6	0.624	0.134	0.715
Residuals	105	489.6	4.663		

We check normality assumptions:

```
plot(fit_anova_clean_boxcox)
```





```
shapiro.test(fit_anova_clean_boxcox$residuals)
```

Shapiro-Wilk normality test

```
data: fit_anova_clean_boxcox$residuals
W = 0.97245, p-value = 0.02316
```

Data is still not normal.

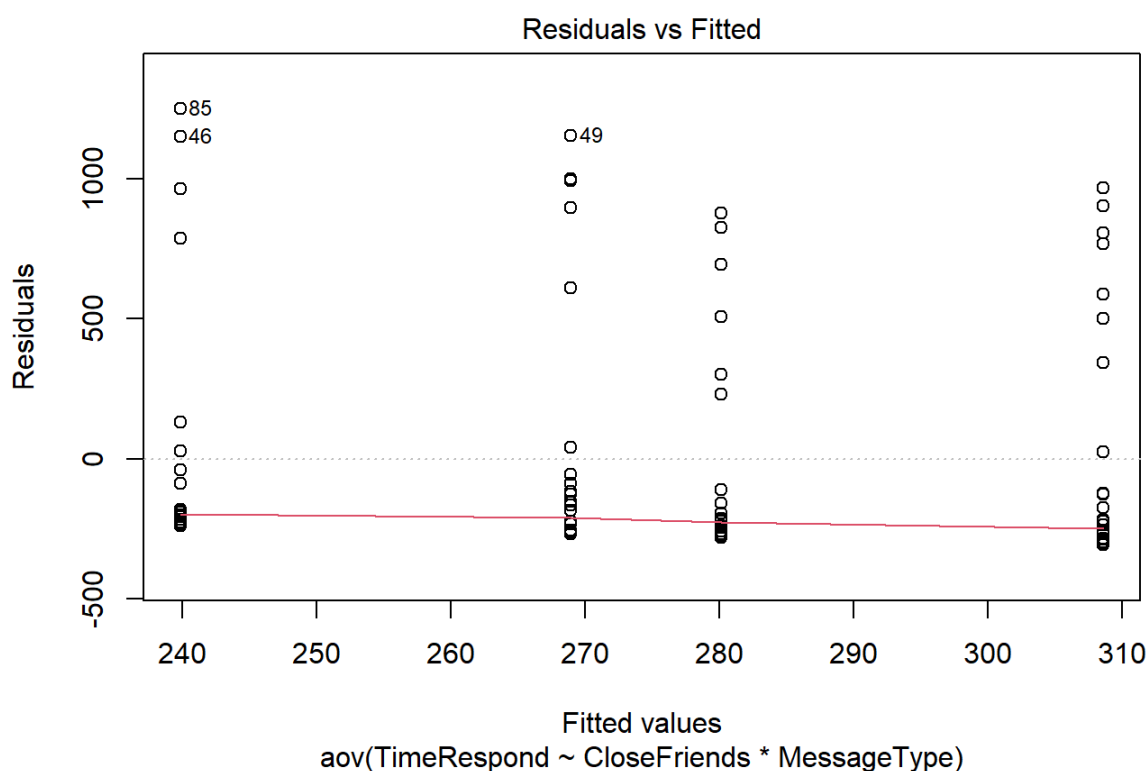
Dataset without outliers

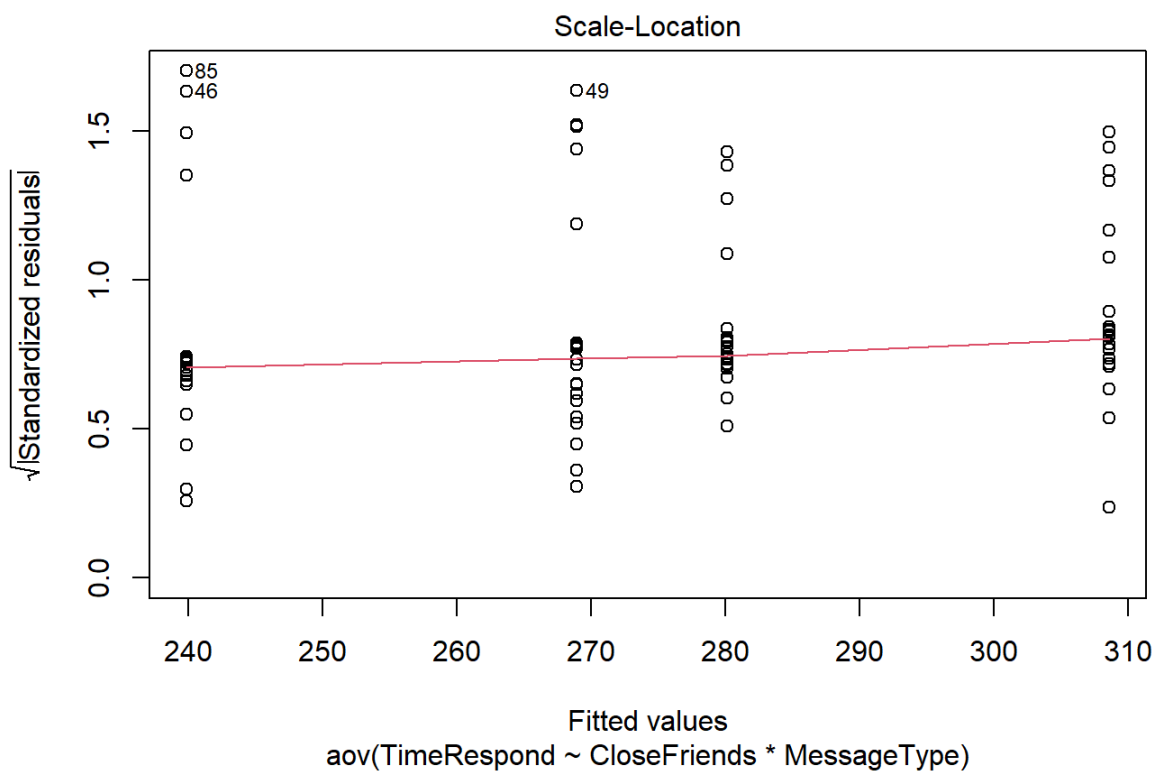
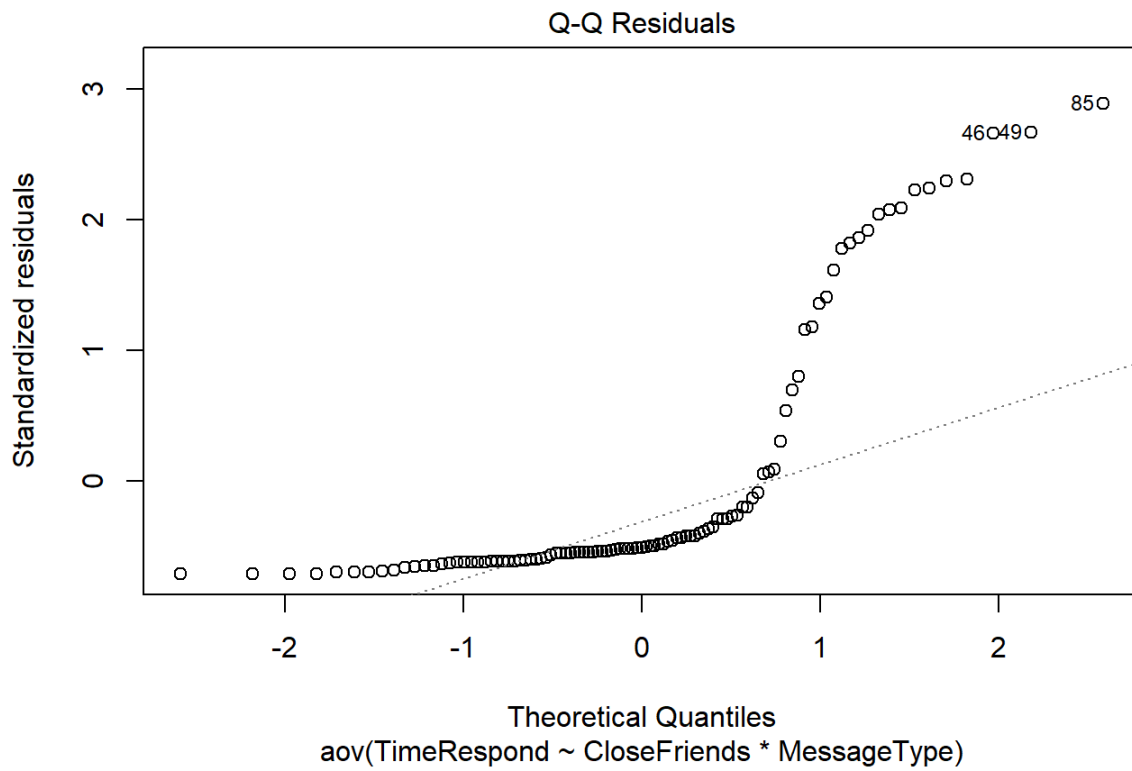
We'll repeat the analysis with the dataset without outliers.

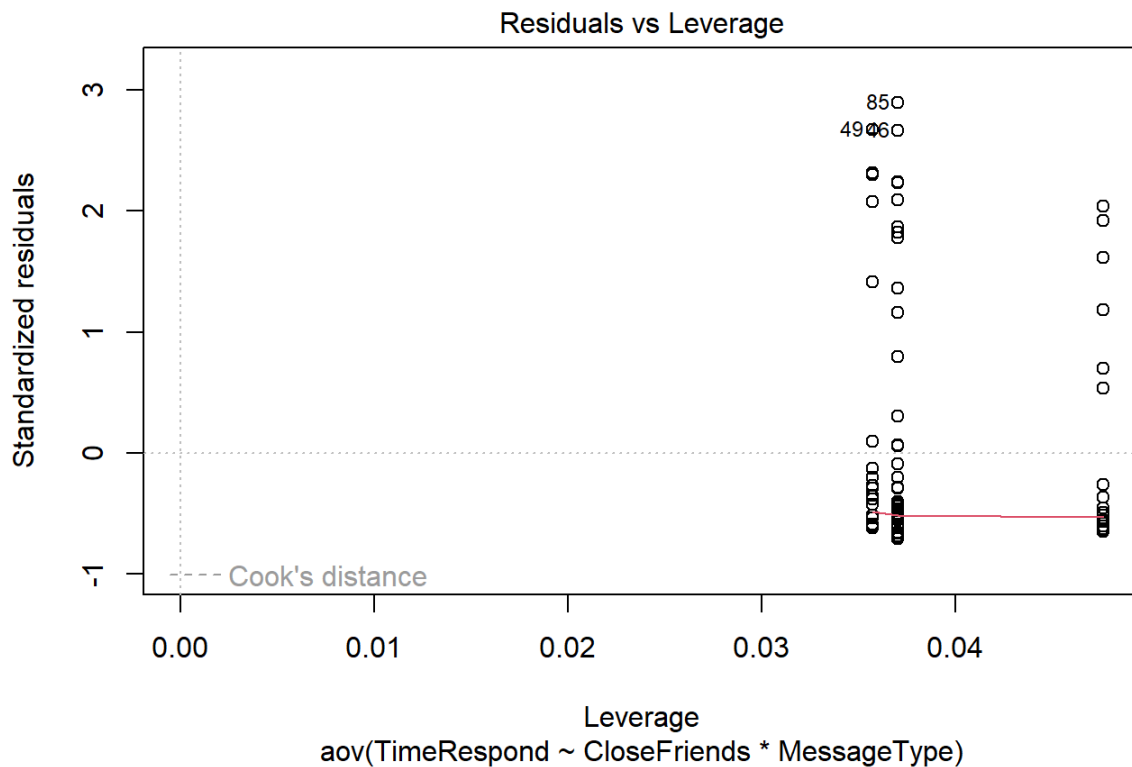
```
options(contrasts=c("contr.sum","contr.sum"))
fit_anova_outliers <- aov(TimeRespond ~ CloseFriends * MessageType, data=datos_compressed_outliers)
summary(fit_anova_outliers)
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
CloseFriends	1	44165	44165	0.228	0.634
MessageType	1	161	161	0.001	0.977
CloseFriends:MessageType	1	21040	21040	0.109	0.743
Residuals	99	19189784	193836		

```
plot(fit_anova_outliers)
```







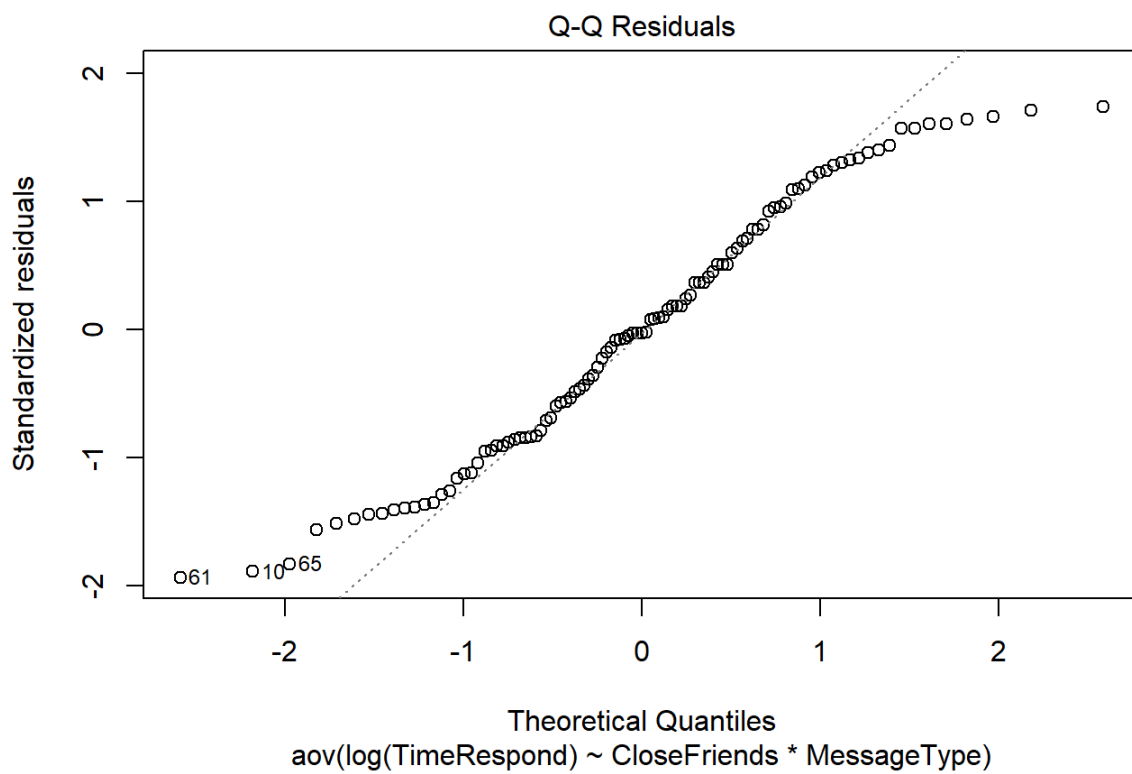
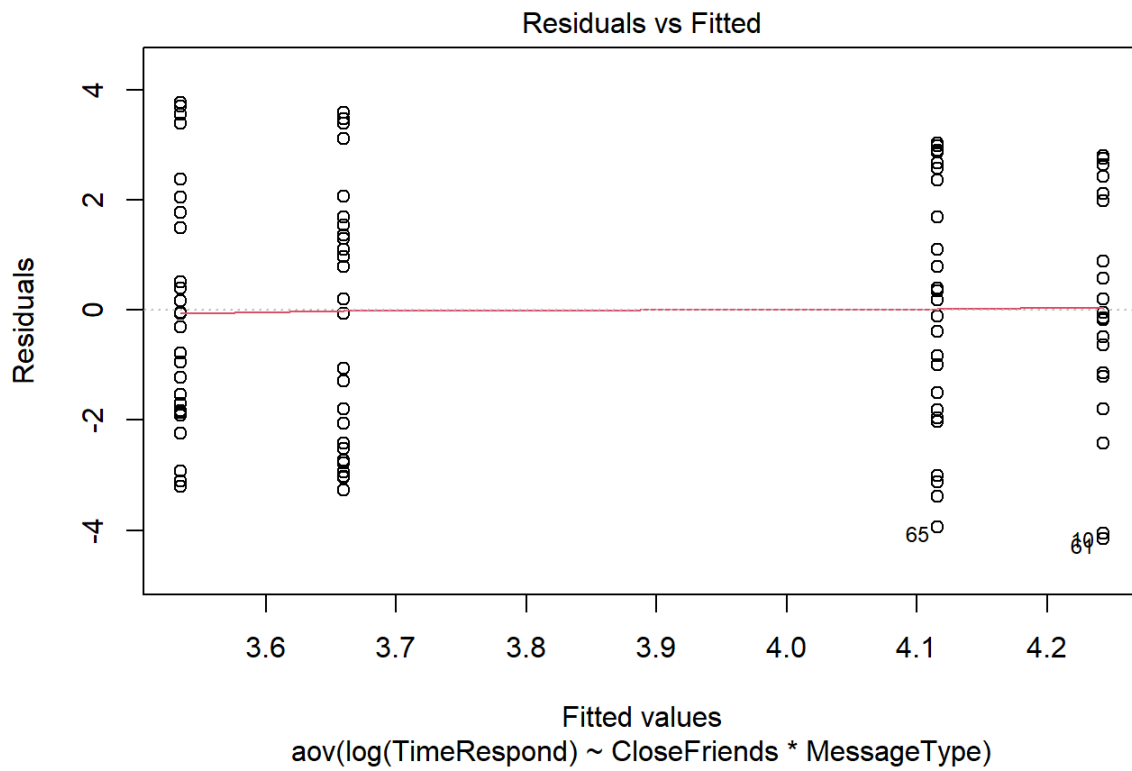
Obviously, the original data is still not normal. Let's apply the same transforms as before.

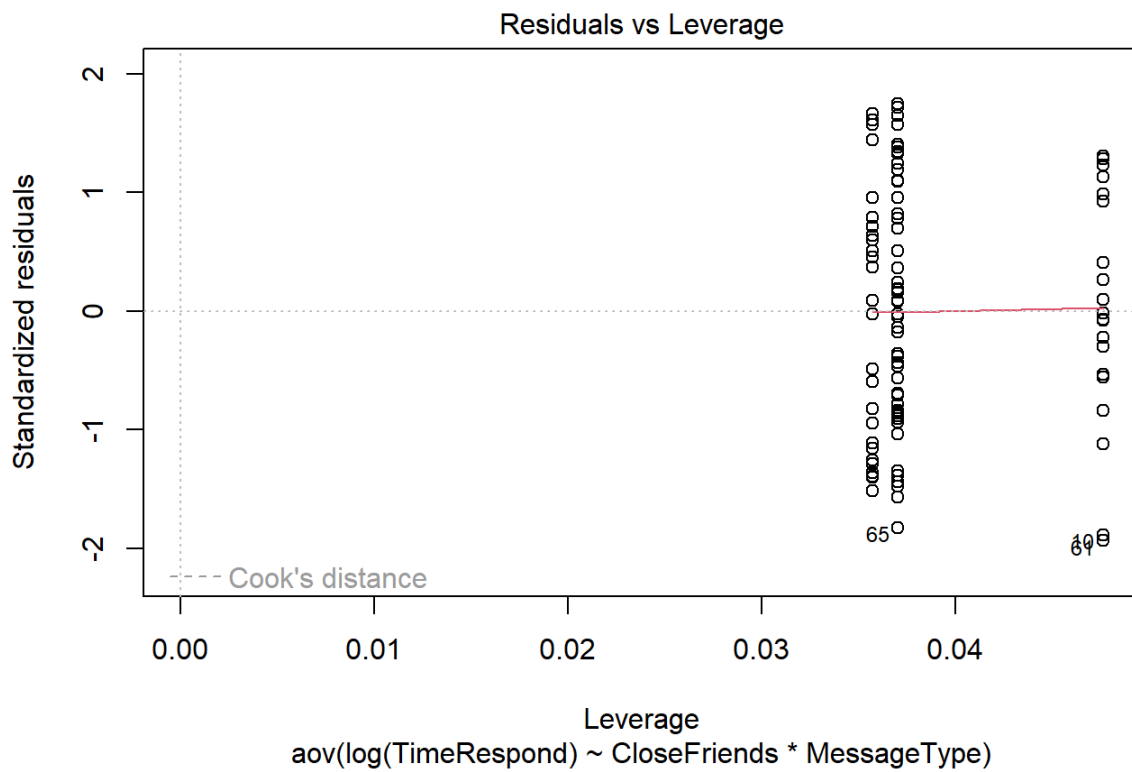
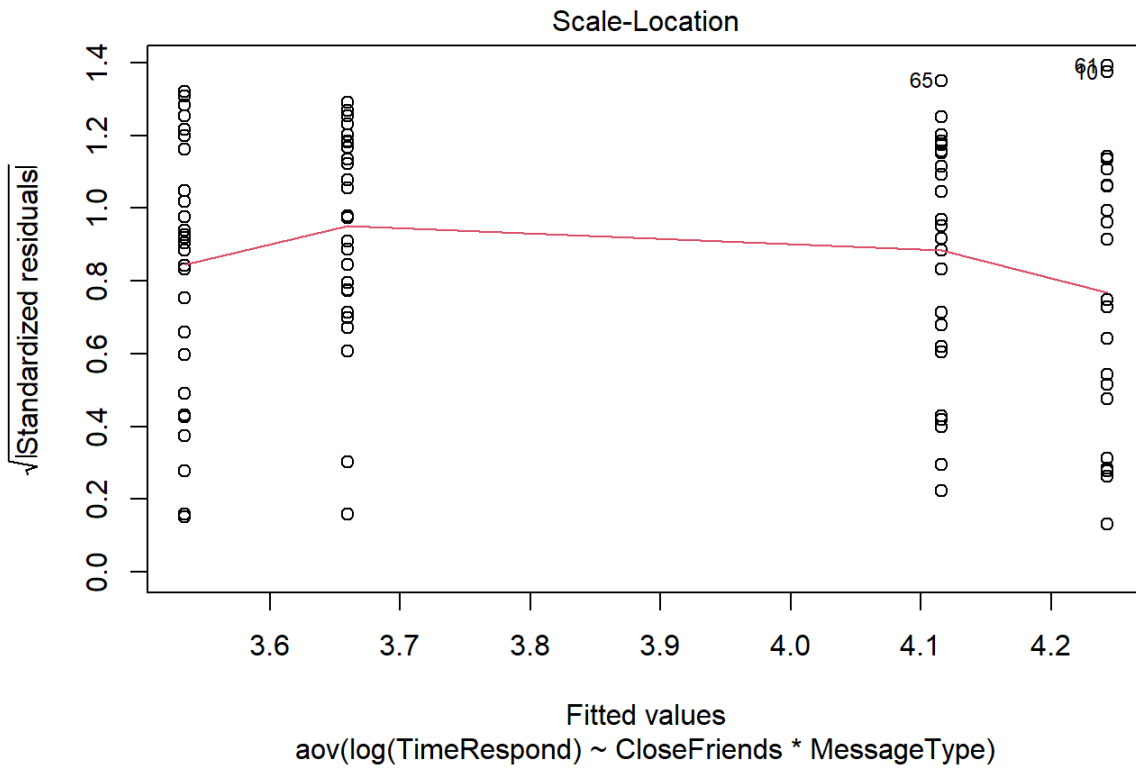
Logarithmic transformation

```
options(contrasts=c("contr.sum", "contr.sum"))
fit_anova_outliers_log <- aov(log(TimeRespond) ~ CloseFriends * MessageType, data=datos_compressed_)
summary(fit_anova_outliers_log)
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
CloseFriends	1	8.4	8.426	1.736	0.191
MessageType	1	0.4	0.408	0.084	0.773
CloseFriends:MessageType	1	0.0	0.000	0.000	0.999
Residuals	99	480.4	4.853		

```
plot(fit_anova_outliers_log)
```





```
shapiro.test(fit_anova_outliers_log$residuals)
```

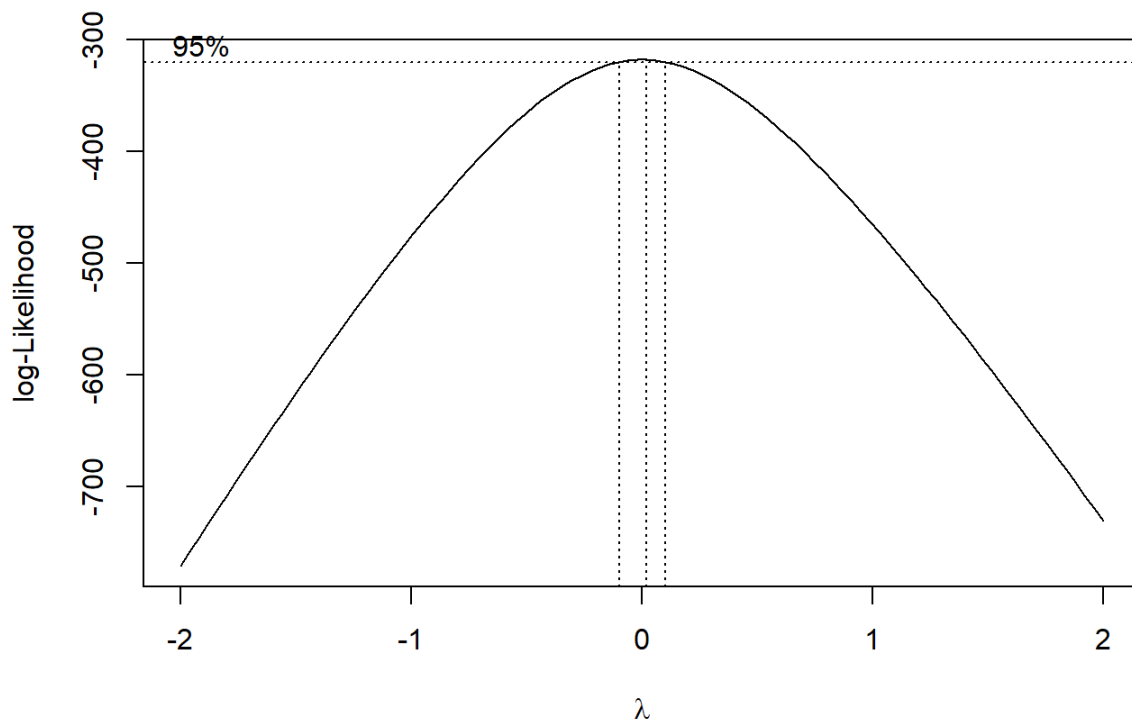
Shapiro-Wilk normality test

```
data: fit_anova_outliers_log$residuals  
W = 0.96467, p-value = 0.007475
```

Data is still not normal.

Box-Cox transformation

```
bc <- boxcox(TimeRespond ~ CloseFriends * MessageType, data=datos_compressed_outliers)
```



```
lambda <- bc$x[which.max(bc$y)]
```

Same as before, lambda is almost 0, so the ideal transformation is logarithmic.

Cannot find a way to transform the data into normality.

This makes sense, due to the nature of the data - it has two peaks. What if the study the people who answered in the first and in the second days separately?

DIVIDE INTO FIRST AND SECOND DAY

Now we study the people who answered the first day or not. We establish the border between first and second day at 500 minutes (which corresponds to 6am of the following day).

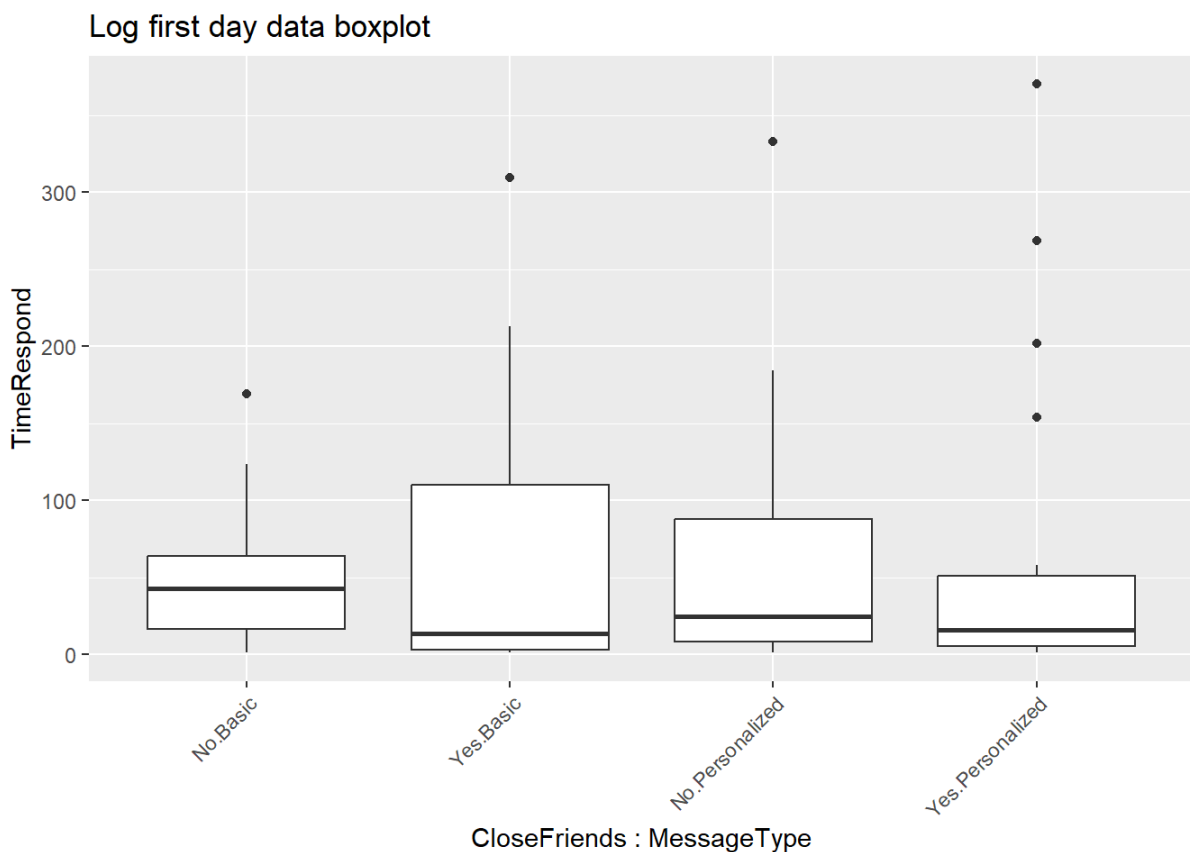
Preprocessing for first day data

```
datos_compressed_first_day <- data_clean %>%  
  filter(  
    TimeRespond < 500  
  )
```

```
datos_compressed_first_day %>%  
  count(CloseFriends, MessageType)
```

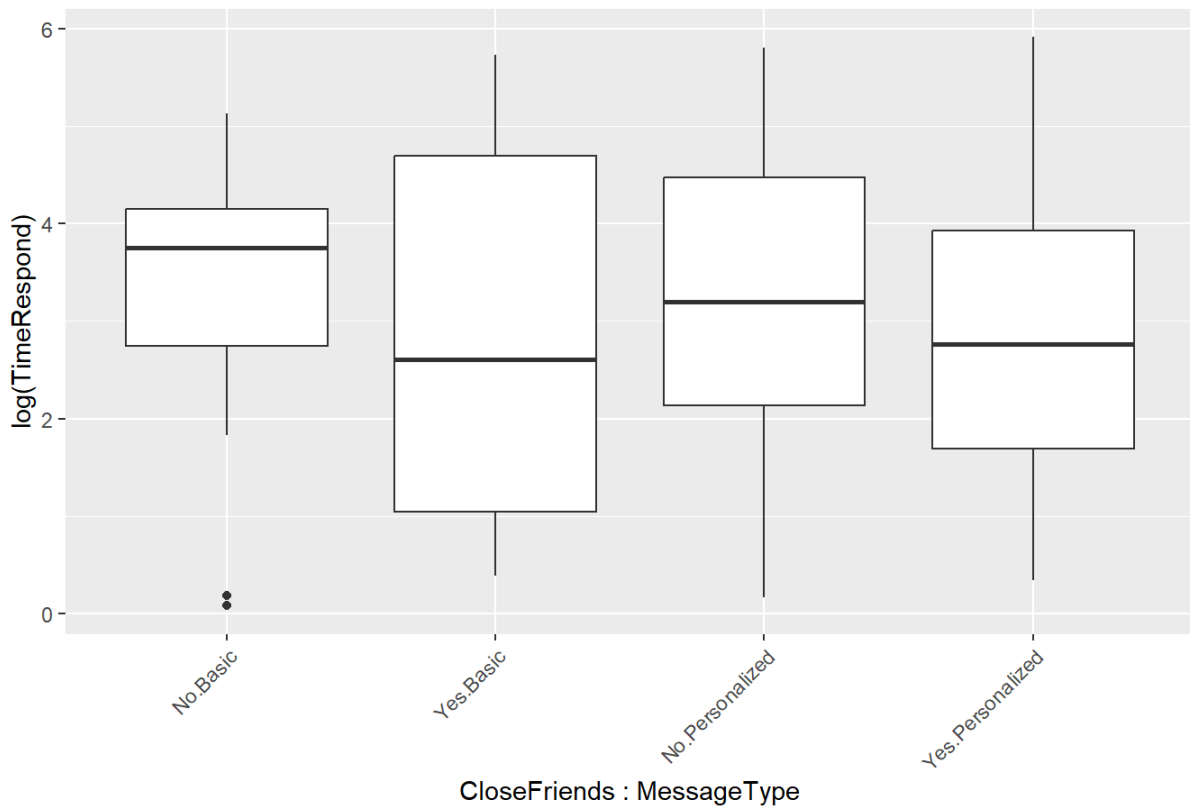
	CloseFriends	MessageType	n
1	No	Basic	15
2	No	Personalized	20
3	Yes	Basic	23
4	Yes	Personalized	23

```
ggplot(datos_compressed_first_day, aes(x = interaction(CloseFriends, MessageType), y = TimeRespond))  
  geom_boxplot() +  
  labs(title = "Log first day data boxplot", x = "CloseFriends : MessageType") +  
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



```
ggplot(datos_compressed_first_day, aes(x = interaction(CloseFriends, MessageType), y = log(TimeRespond)))  
  geom_boxplot() +  
  labs(title = "Log first day data boxplot", x = "CloseFriends : MessageType") +  
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

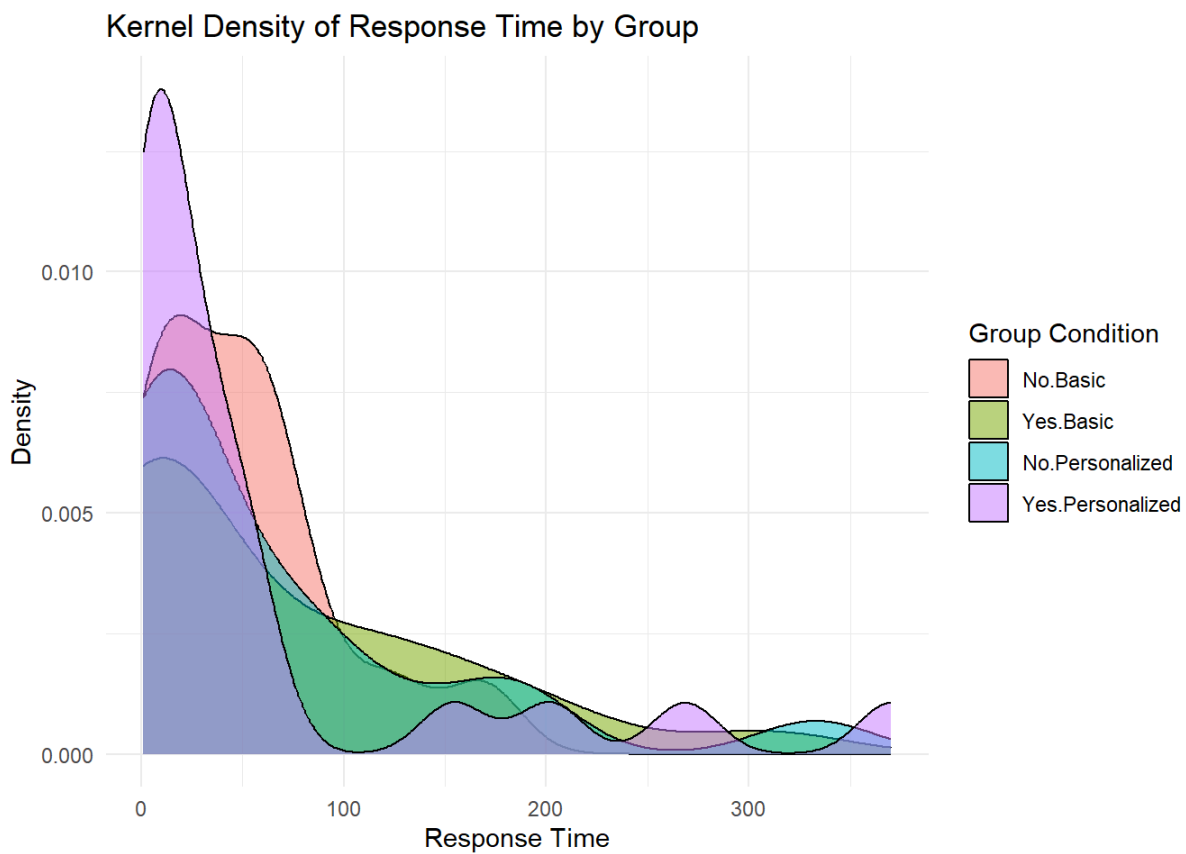
Log first day data boxplot



```
data_clean$FirstDay <- data_clean$TimeRespond < 500
data_clean$FirstDay[is.na(data_clean$FirstDay)] <- 0
```

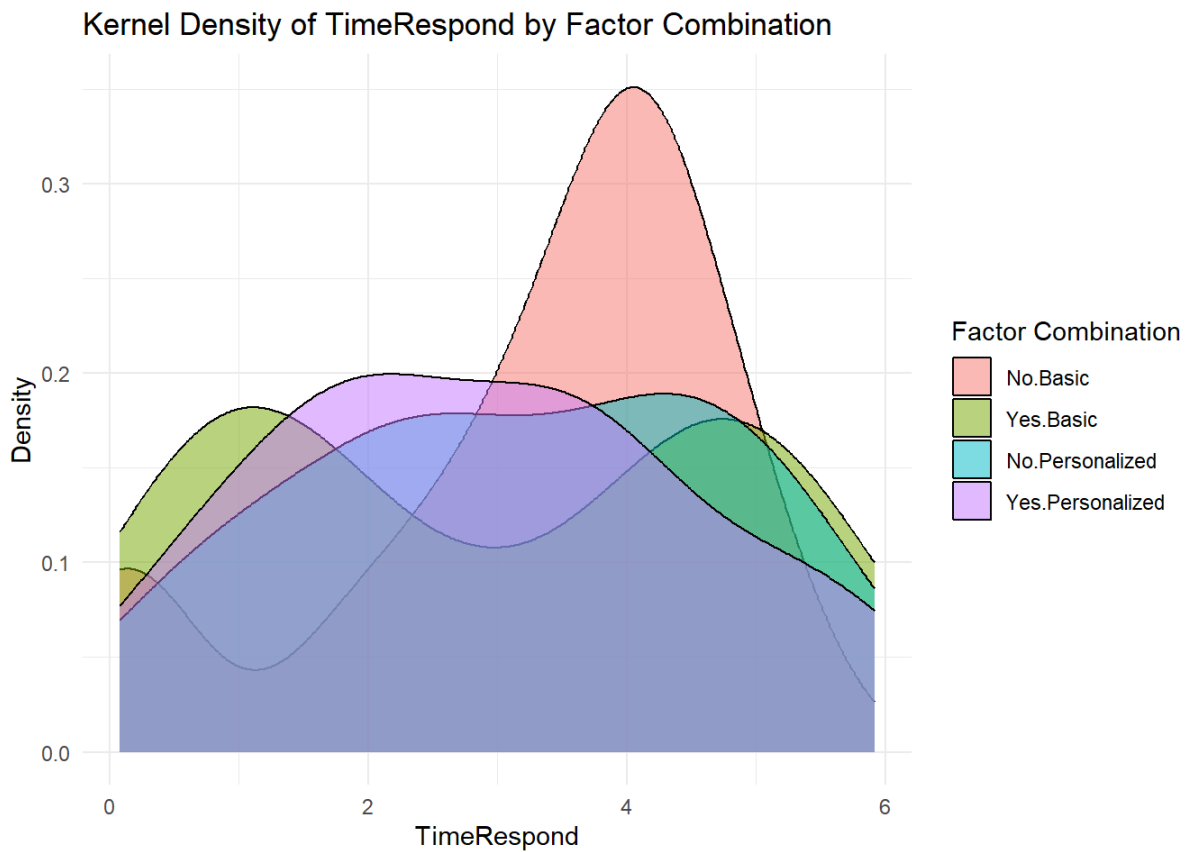
Histograms

```
ggplot(datos_compressed_first_day, aes(x = TimeRespond, fill = interaction(CloseFriends, MessageType))
  geom_density(alpha = 0.5) + # alpha makes the fill transparent so you can see overlaps
  labs(
    title = "Kernel Density of Response Time by Group",
    x = "Response Time",
    y = "Density",
    fill = "Group Condition"
  ) +
  theme_minimal()
```



Logarithmic histograms:

```
ggplot(datos_compressed_first_day, aes(x = log(TimeRespond), fill = interaction(CloseFriends, Messa
geom_density(alpha = 0.5) + # alpha makes the fill transparent so you can see overlaps
labs(
  title = "Kernel Density of TimeRespond by Factor Combination",
  x = "TimeRespond",
  y = "Density",
  fill = "Factor Combination"
) +
theme_minimal()
```



First-day Response Rate

Chi-SQ For CloseFriends

```
condition <- data_clean$TimeRespond < 500 &
  !is.na(data_clean$TimeRespond)
table_CloseFriends_FD <- table(data_clean$CloseFriends, condition)
table_CloseFriends_FD
```

```
condition
  FALSE TRUE
No     37  35
Yes    20  46
```

```
chisq.test(table_CloseFriends_FD)
```

Pearson's Chi-squared test with Yates' continuity correction

```
data: table_CloseFriends_FD
X-squared = 5.4753, df = 1, p-value = 0.01929
```

Significant difference.

Chi-SQ For MessageType

```
table_MessageType_FD <- table(data_clean$MessageType, condition)
table_MessageType_FD
```


	condition	
	FALSE	TRUE
Basic	31	38
Personalized	26	43

```
chisq.test(table_MessageType_FD)
```

Pearson's Chi-squared test with Yates' continuity correction

```
data: table_MessageType_FD
X-squared = 0.47823, df = 1, p-value = 0.4892
```

Not significant.

Logistic regression

We further analyze the first day response date using logistic regression.

```
logit_FD <- glm(FirstDay ~ CloseFriends * MessageType,
               data = data_clean,
               family = binomial(link = "logit"))
summary(logit_FD)
```

Call:

```
glm(formula = FirstDay ~ CloseFriends * MessageType, family = binomial(link = "logit"),
    data = data_clean)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	0.3881	0.1792	2.166	0.0303 *
CloseFriends1	-0.4448	0.1792	-2.482	0.0131 *
MessageType1	-0.1399	0.1792	-0.781	0.4349
CloseFriends1:MessageType1	-0.1399	0.1792	-0.781	0.4349

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 187.11 on 137 degrees of freedom
 Residual deviance: 179.33 on 134 degrees of freedom
 AIC: 187.33

Number of Fisher Scoring iterations: 4

```
# Combined Odds Ratios and 95% Wald CIs
exp(cbind(OR = coef(logit_FD), confint.default(logit_FD)))
```

	OR	2.5 %	97.5 %
(Intercept)	1.4742102	1.0376074	2.0945261
CloseFriends1	0.6409610	0.4511336	0.9106635
MessageType1	0.8694417	0.6119474	1.2352841
CloseFriends1:MessageType1	0.8694417	0.6119474	1.2352841

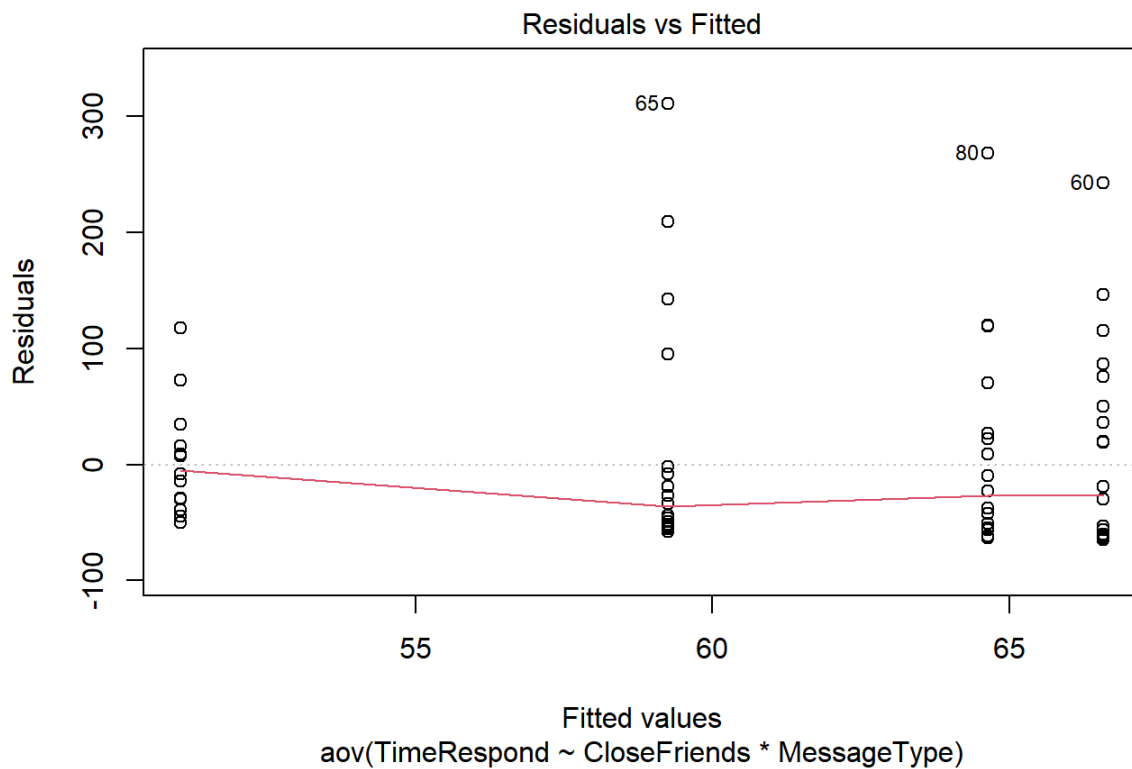
ANOVA for first day data

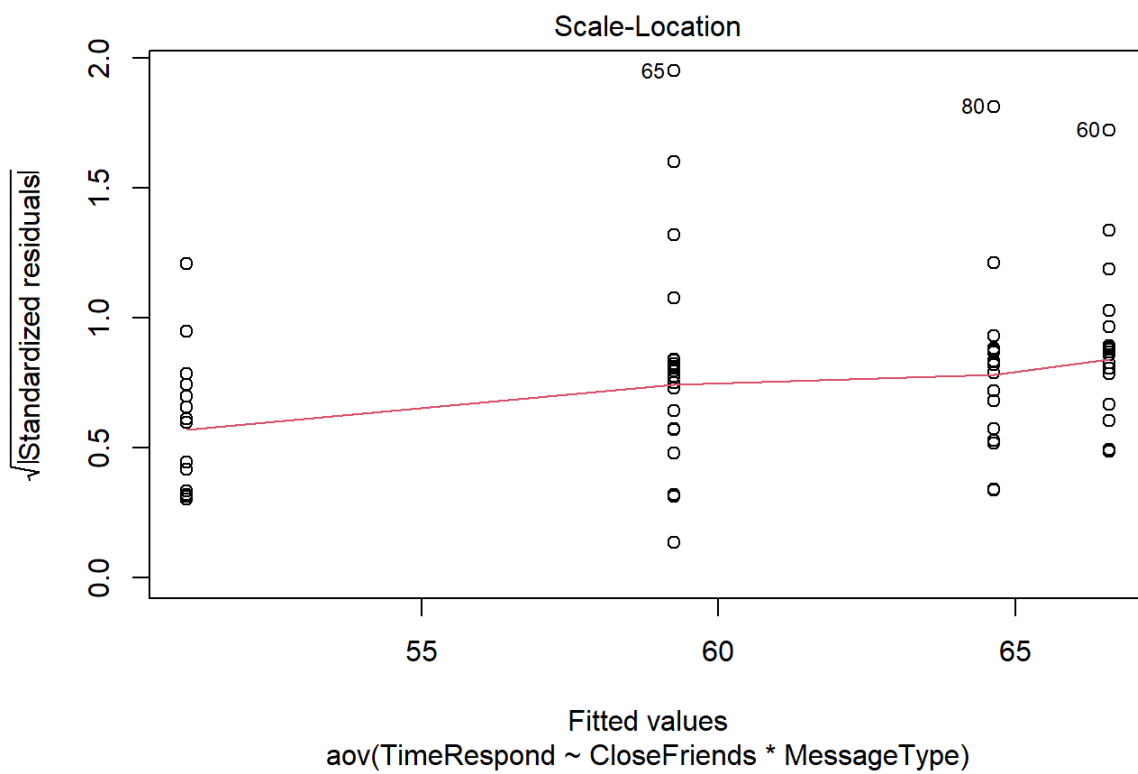
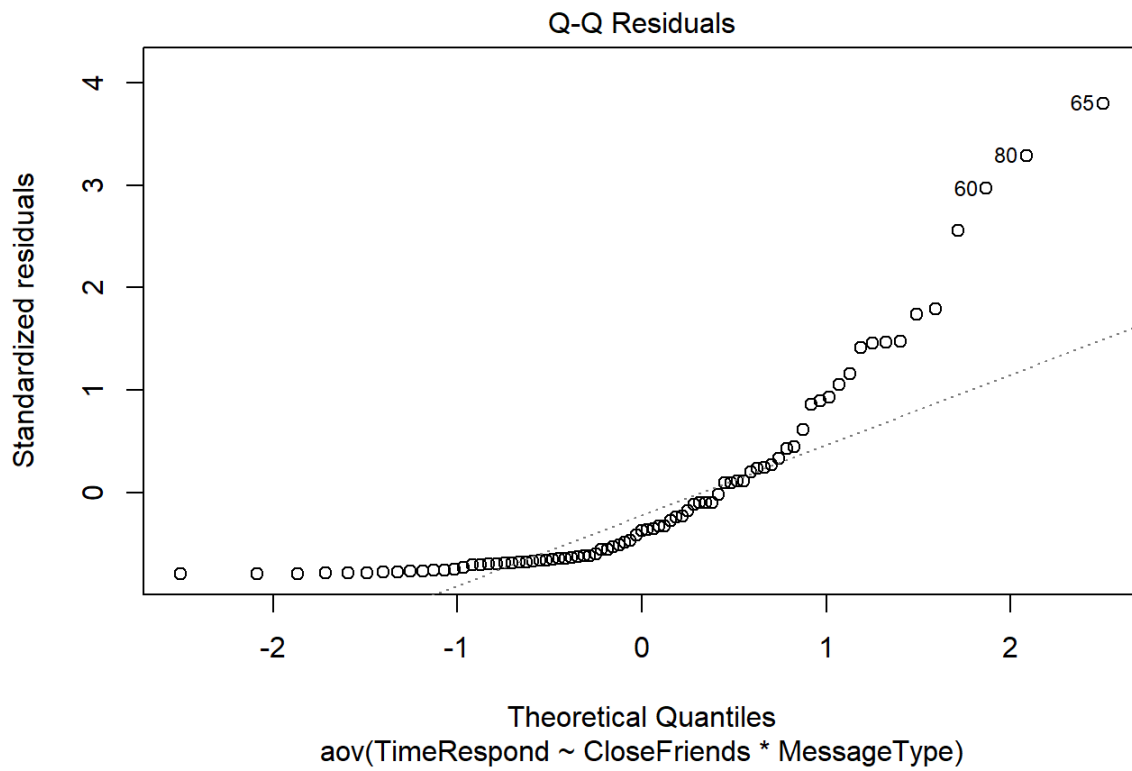
Data without transformations

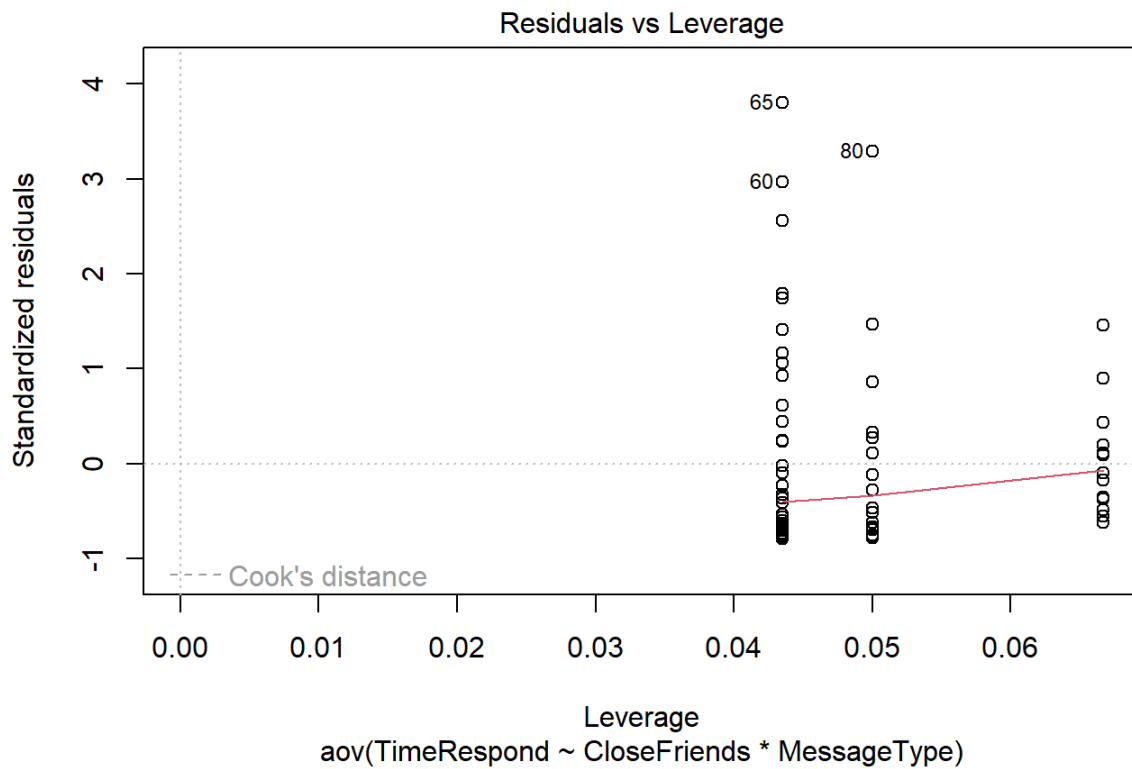
```
options(contrasts=c("contr.sum","contr.sum"))
fit_anova_first_day <- aov(TimeRespond ~ CloseFriends * MessageType, data=datos_compressed_first_da
summary(fit_anova_first_day)
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
CloseFriends	1	334	334	0.048	0.828
MessageType	1	52	52	0.007	0.932
CloseFriends:MessageType	1	2149	2149	0.307	0.581
Residuals	77	538307	6991		

```
plot(fit_anova_first_day)
```







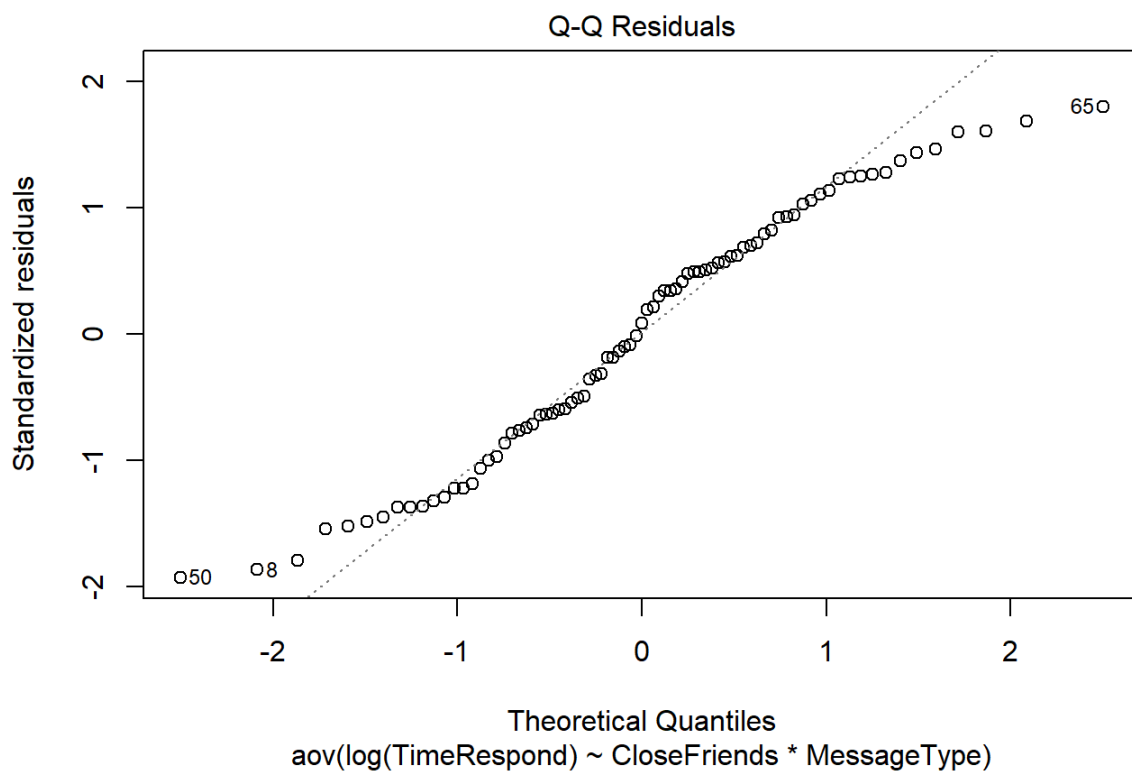
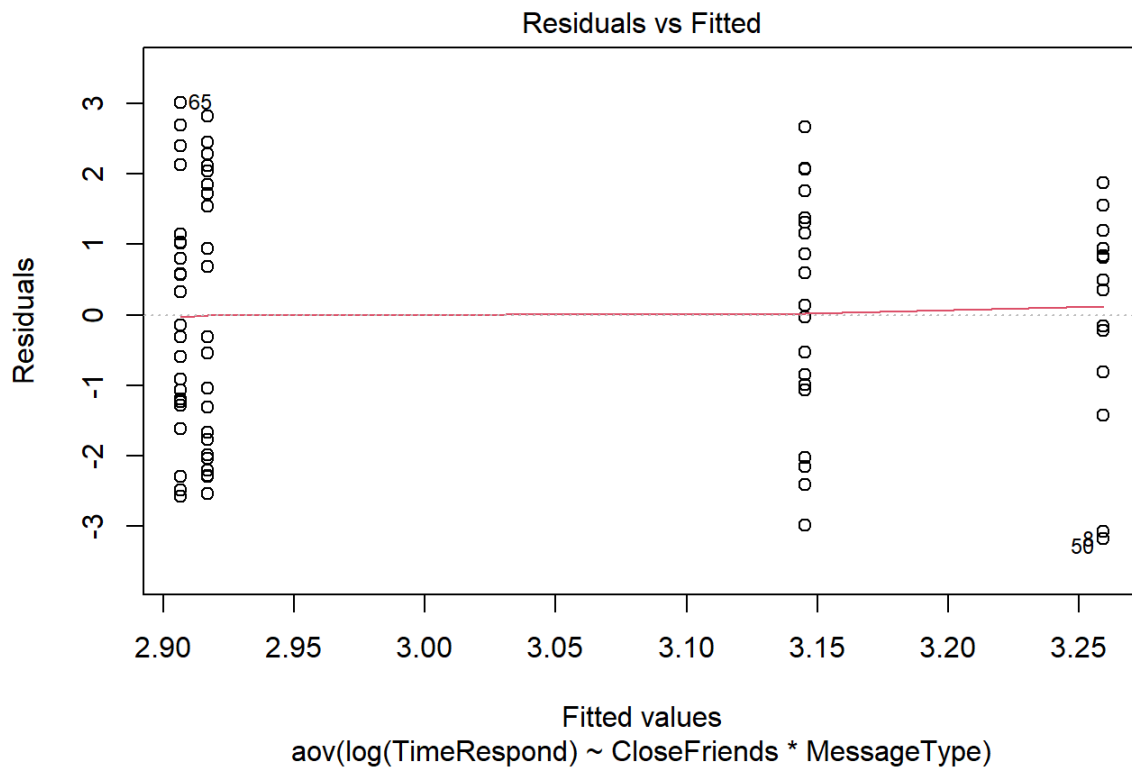
Still not normal.

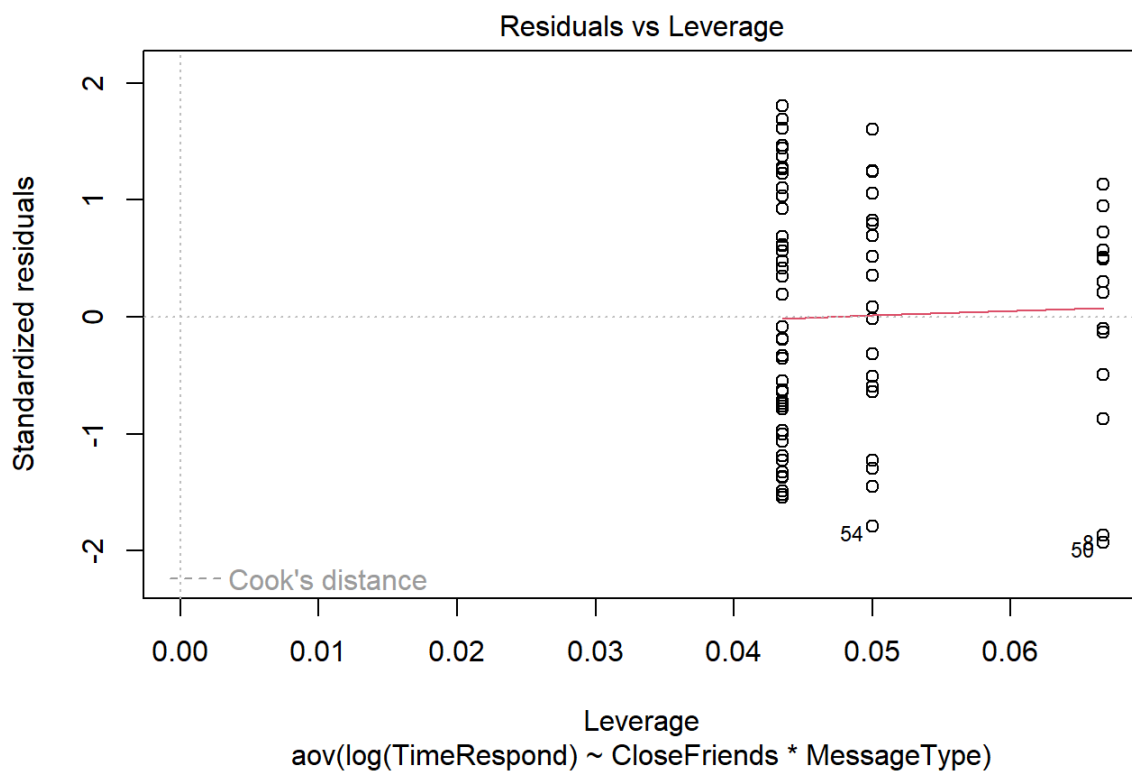
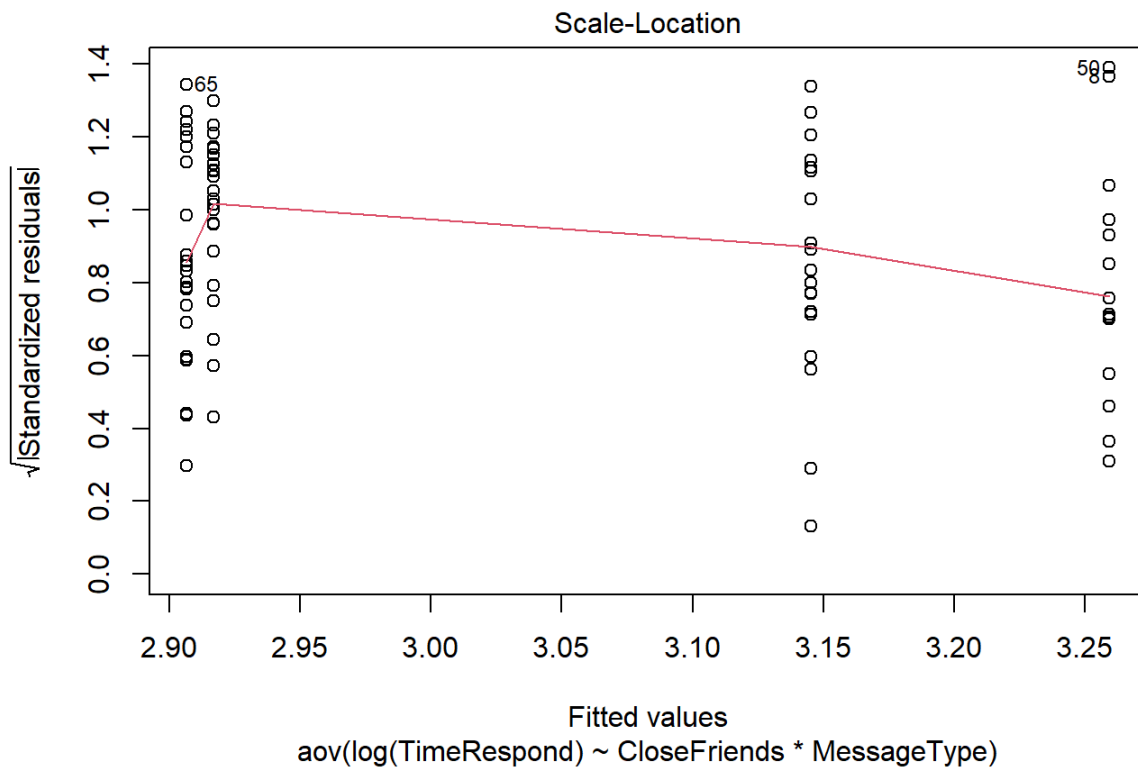
Logarithmic transform

```
options(contrasts=c("contr.sum", "contr.sum"))
fit_anova_first_day_log <- aov(log(TimeRespond) ~ CloseFriends * MessageType, data=datos_compressed)
summary(fit_anova_first_day_log)
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
CloseFriends	1	1.58	1.5844	0.546	0.462
MessageType	1	0.06	0.0599	0.021	0.886
CloseFriends:MessageType	1	0.05	0.0528	0.018	0.893
Residuals	77	223.50	2.9026		

```
plot(fit_anova_first_day_log)
```





```
shapiro.test(fit_anova_first_day_log$residuals)
```

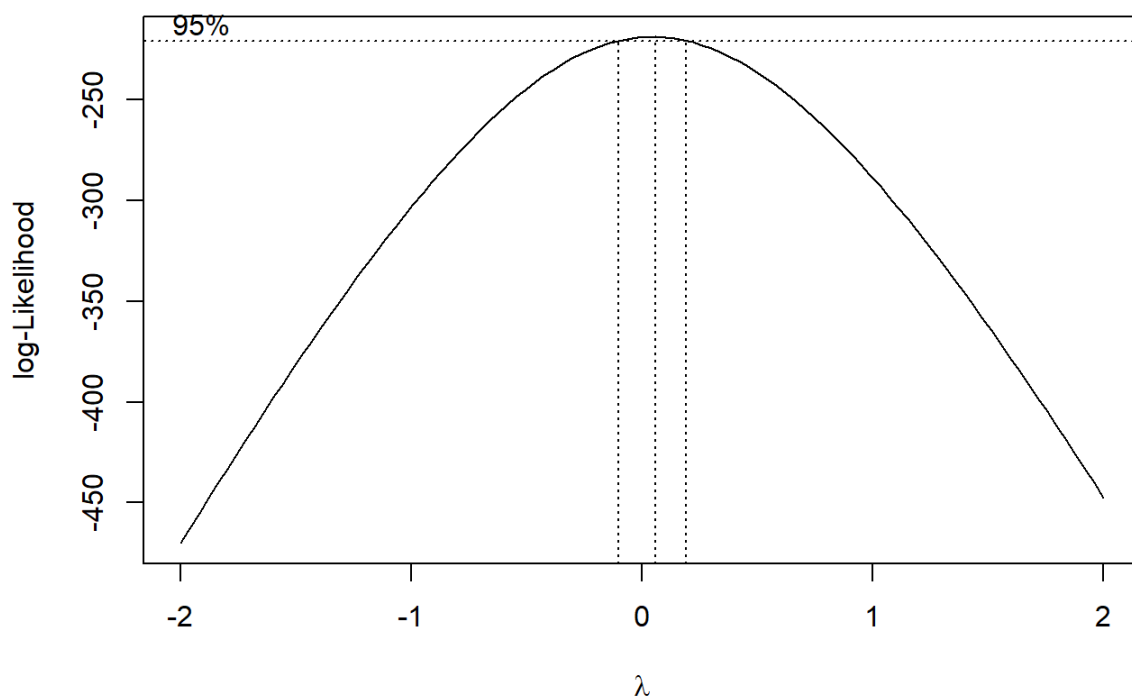
Shapiro-Wilk normality test

```
data: fit_anova_first_day_log$residuals
W = 0.96285, p-value = 0.01899
```

Still not normal.

BoxCox transform

```
bc <- boxcox(TimeRespond ~ CloseFriends * MessageType, data=datos_compressed_first_day)
```



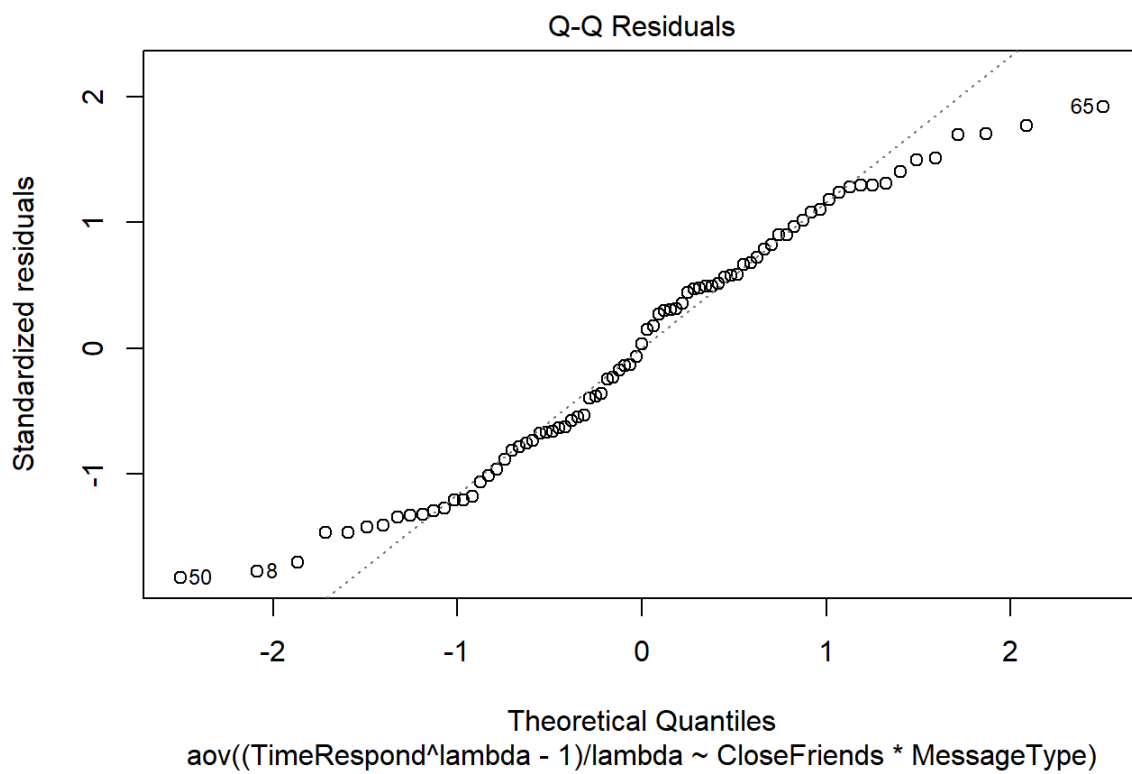
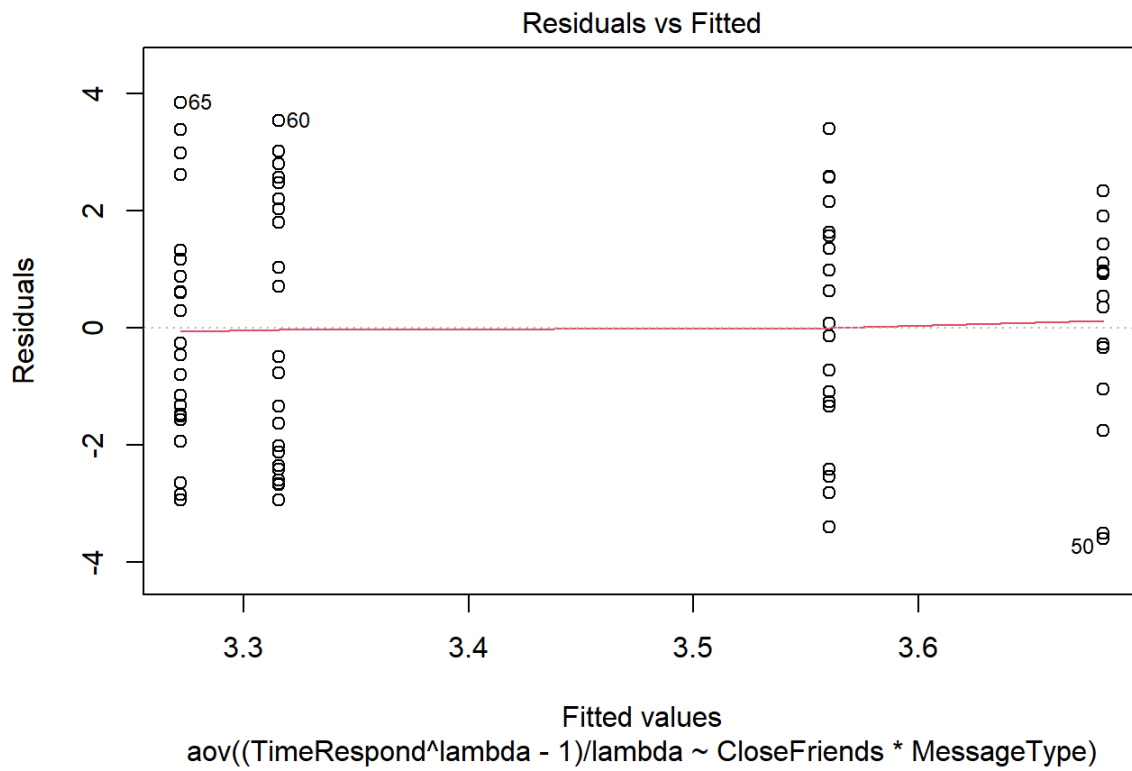
```
lambda <- bc$x[which.max(bc$y)]
```

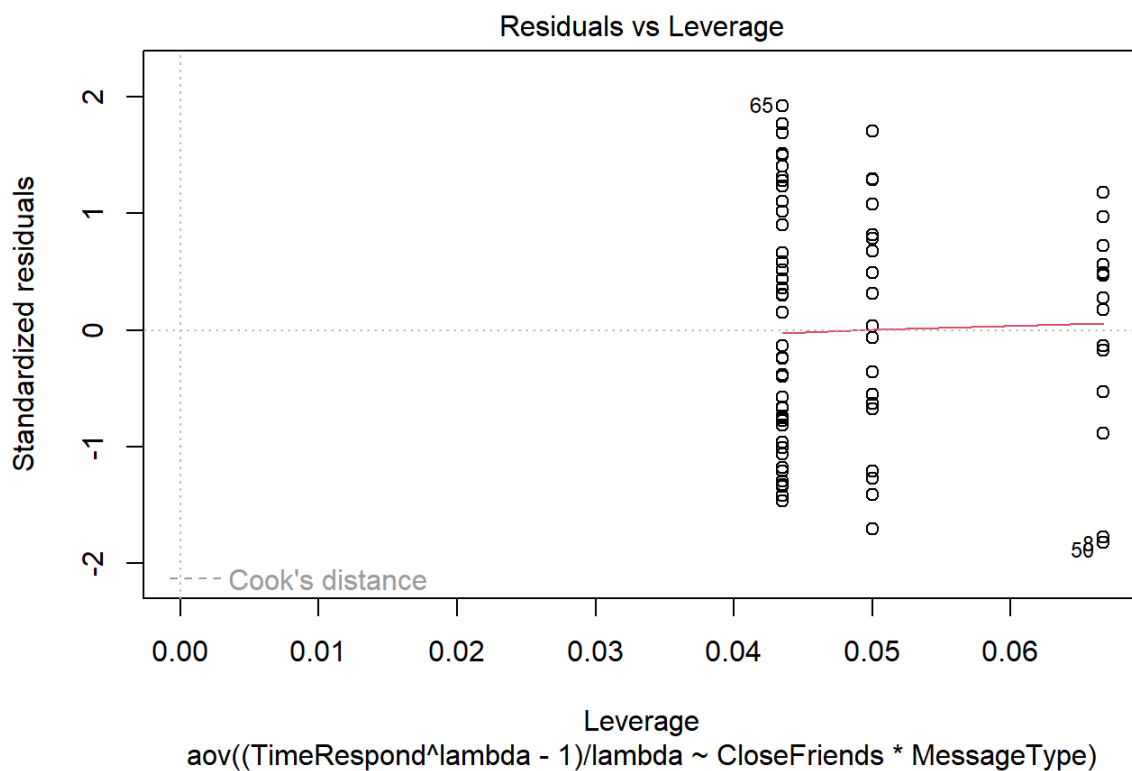
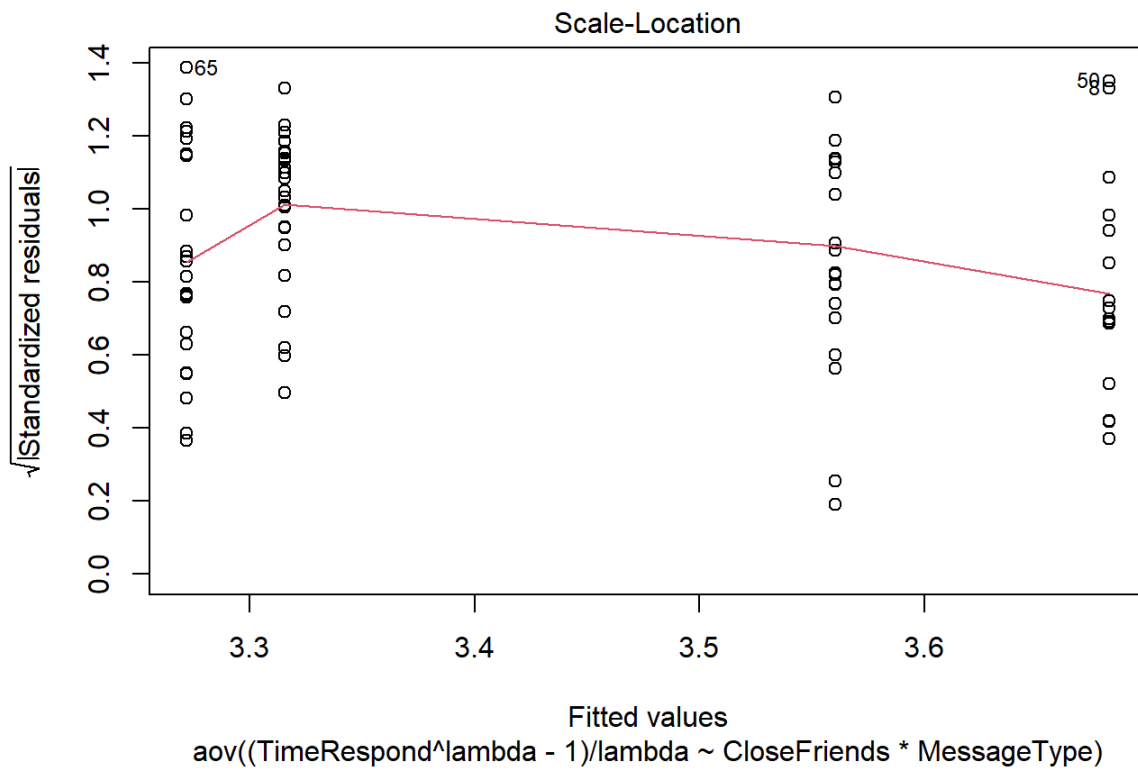
Almost zero, but will try just in case:

```
fit_anova_first_day_boxcox <- aov((TimeRespond^lambda-1)/lambda ~ CloseFriends * MessageType, data=
summary(fit_anova_first_day_boxcox)
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
CloseFriends	1	2.0	2.023	0.485	0.488
MessageType	1	0.1	0.119	0.029	0.866
CloseFriends:MessageType	1	0.0	0.030	0.007	0.933
Residuals	77	321.4	4.174		

```
plot(fit_anova_first_day_boxcox)
```





```
shapiro.test(fit_anova_first_day_boxcox$residuals)
```

Shapiro-Wilk normality test

```
data: fit_anova_first_day_boxcox$residuals
W = 0.96375, p-value = 0.02162
```

Not normal.

OTHER POSSIBLE TESTS

GLMs

GLMs don't assume normality! Just assume independent errors (homocedasticity).

Gamma GLM

```
fit_gamma_glm <- glm(TimeRespond ~ CloseFriends * MessageType, family=Gamma(link="log"), data=dato
summary(fit_gamma_glm)
```

Call:

```
glm(formula = TimeRespond ~ CloseFriends * MessageType, family = Gamma(link = "log"),
    data = datos_compressed_na)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	6.11908	0.19481	31.411	<2e-16 ***
CloseFriends1	0.12712	0.19481	0.653	0.515
MessageType1	0.07671	0.19481	0.394	0.695
CloseFriends1:MessageType1	0.06347	0.19481	0.326	0.745

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Gamma family taken to be 4.114426)

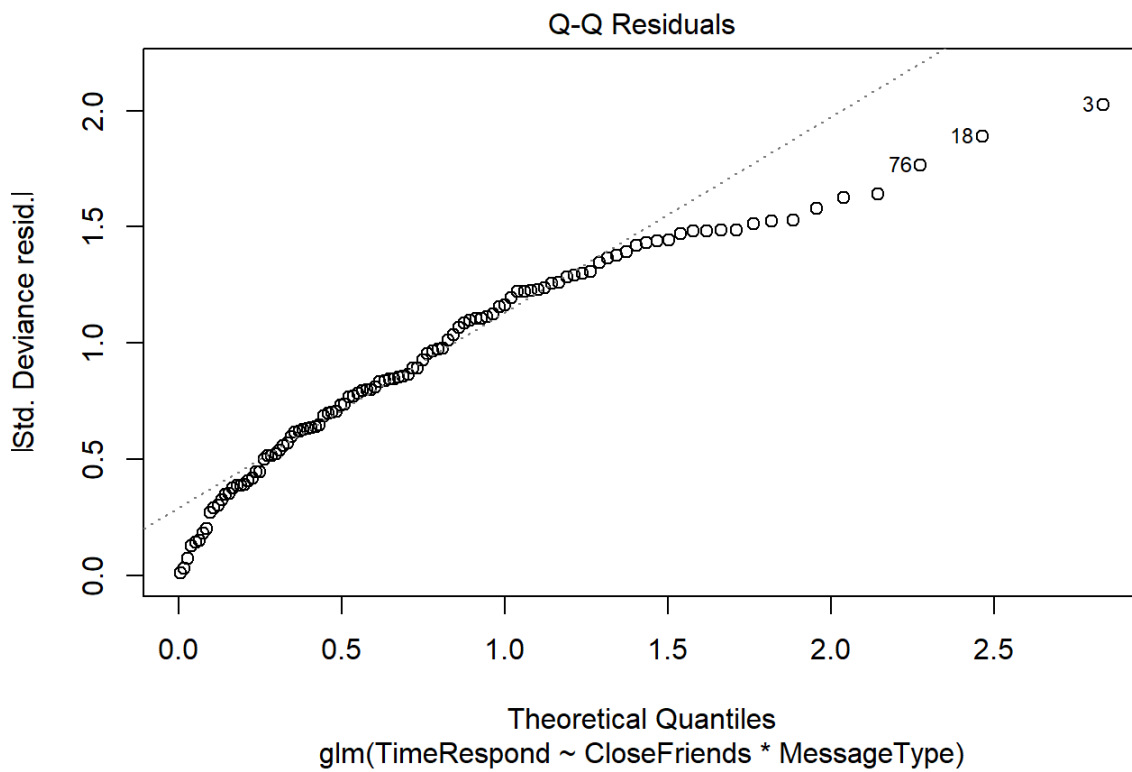
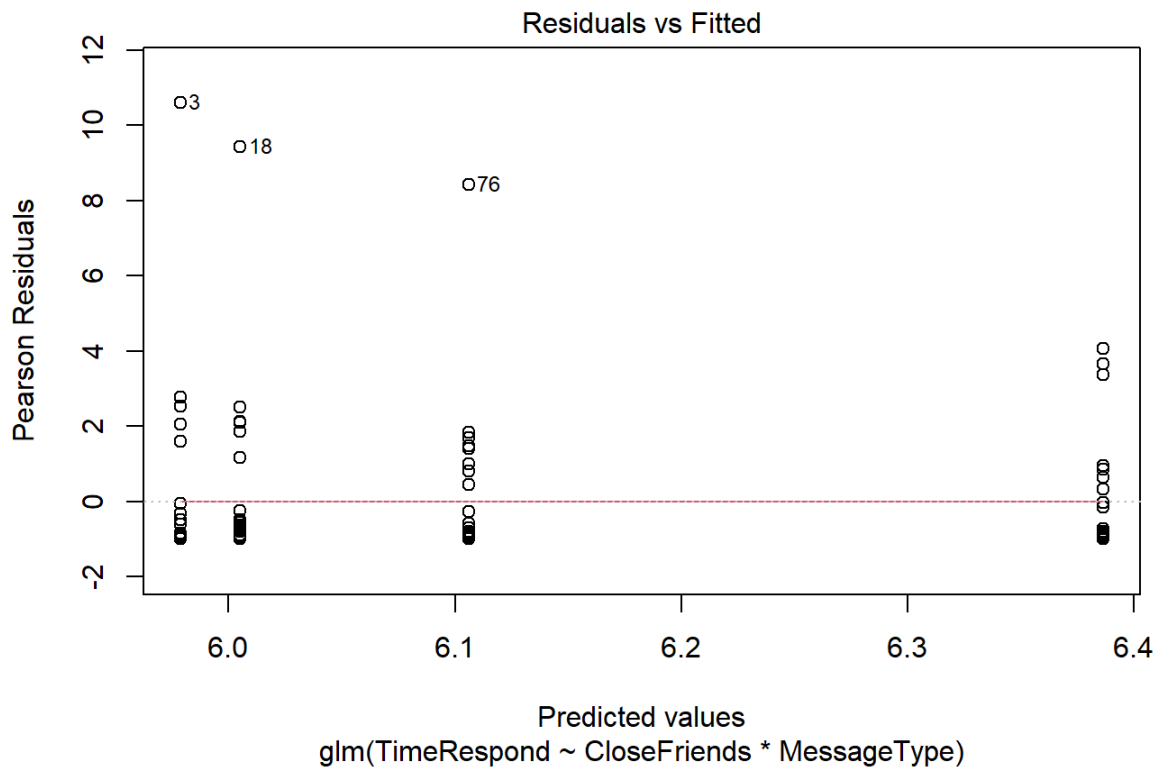
Null deviance: 440.25 on 108 degrees of freedom

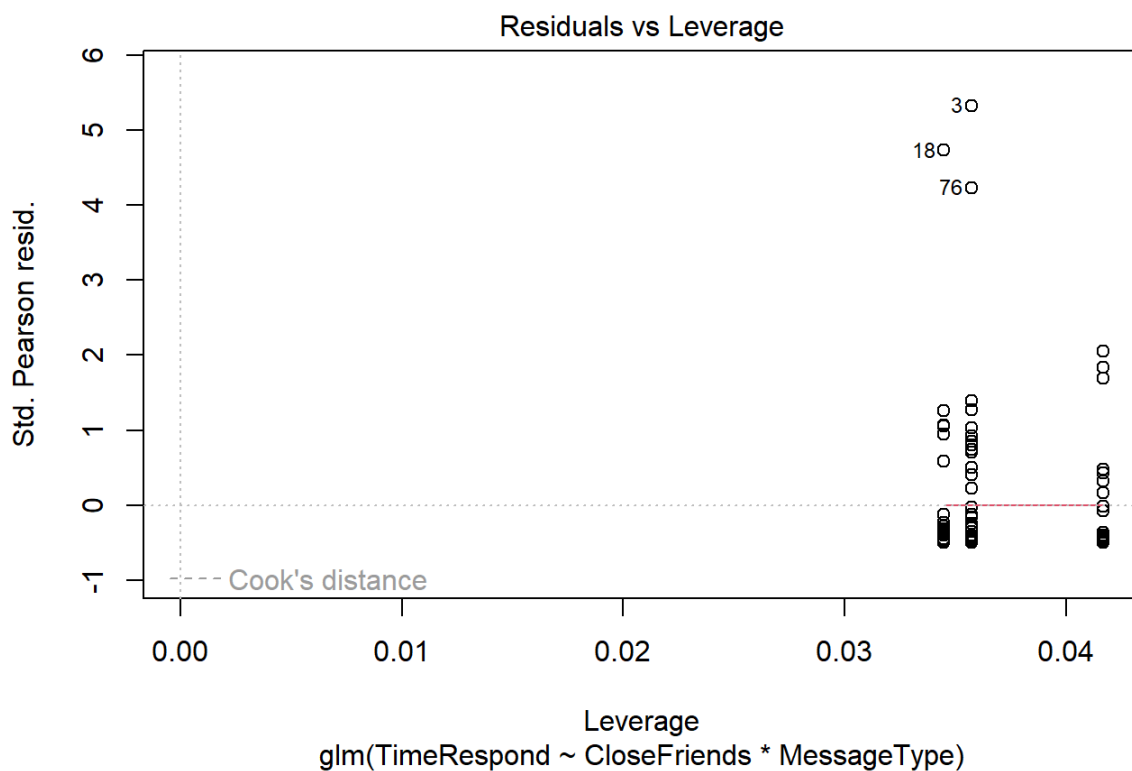
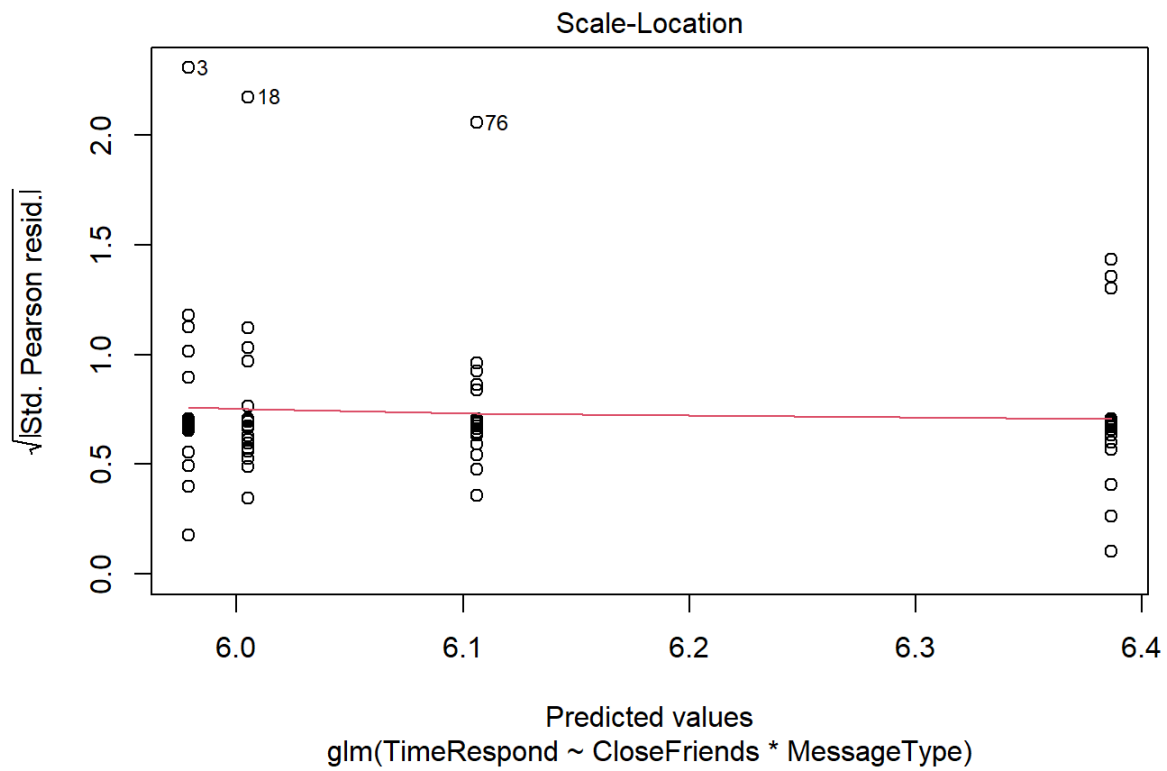
Residual deviance: 437.48 on 105 degrees of freedom

AIC: 1424.5

Number of Fisher Scoring iterations: 12

```
plot(fit_gamma_glm)
```





```
# Calculate D-squared (Deviance R-squared)
D_squared <- 1 - (fit_gamma_glm$deviance / fit_gamma_glm$null.deviance)

# Convert to percentage for easier reading
D_squared_percent <- D_squared * 100
```

```
# Print the result
cat("Deviance R-squared:", round(D_squared, 5), "\n")
```

Deviance R-squared: 0.0063

```
cat("Percentage Explained:", round(D_squared_percent, 2), "%\n")
```

Percentage Explained: 0.63 %

Gamma GLM without outliers

```
fit_gamma_glm_outliers <- glm(TimeRespond ~ CloseFriends * MessageType, family=Gamma(link="log"), d
summary(fit_gamma_glm_outliers)
```

Call:

```
glm(formula = TimeRespond ~ CloseFriends * MessageType, family = Gamma(link = "log"),
    data = datos_compressed_outliers)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	5.610506	0.162066	34.619	<2e-16 ***
CloseFriends1	0.073248	0.162066	0.452	0.652
MessageType1	0.004469	0.162066	0.028	0.978
CloseFriends1:MessageType1	-0.052808	0.162066	-0.326	0.745

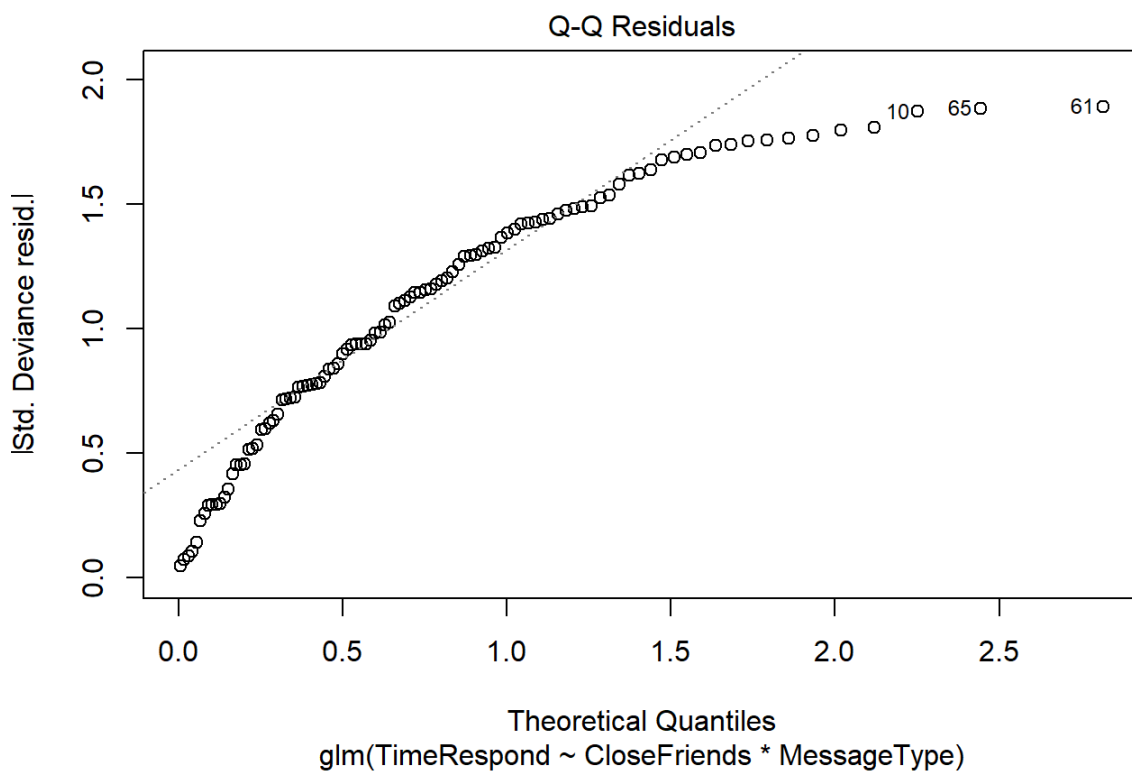
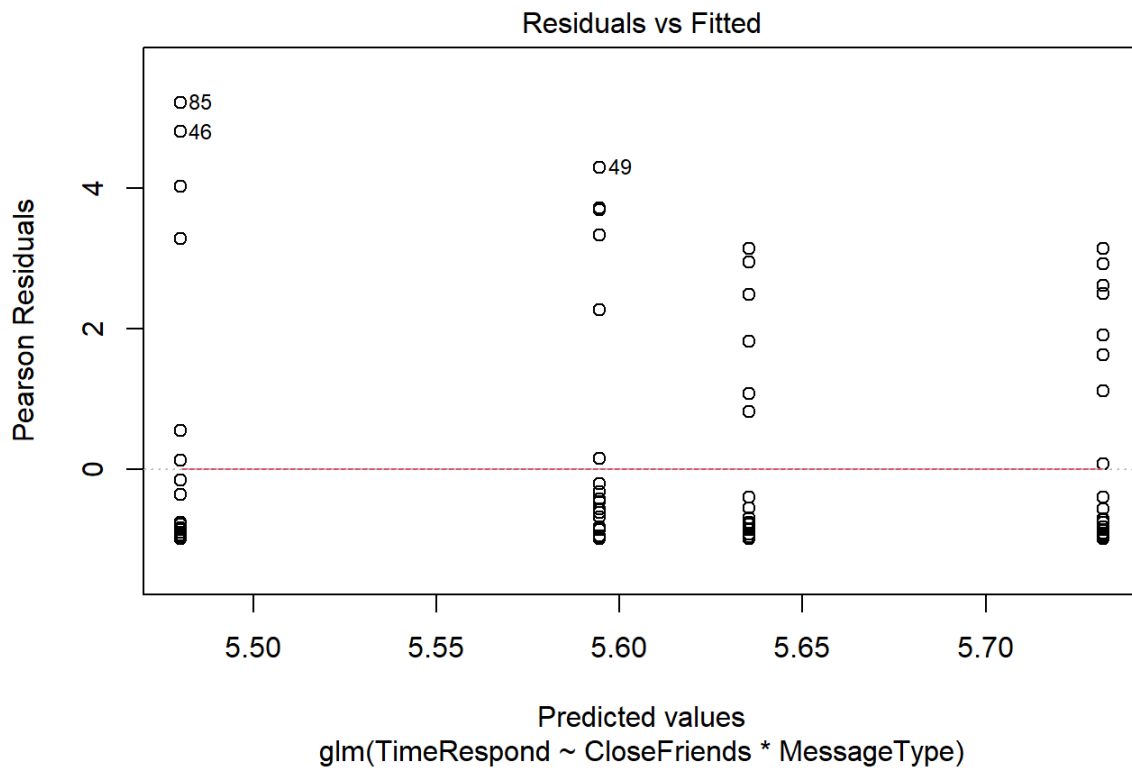
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

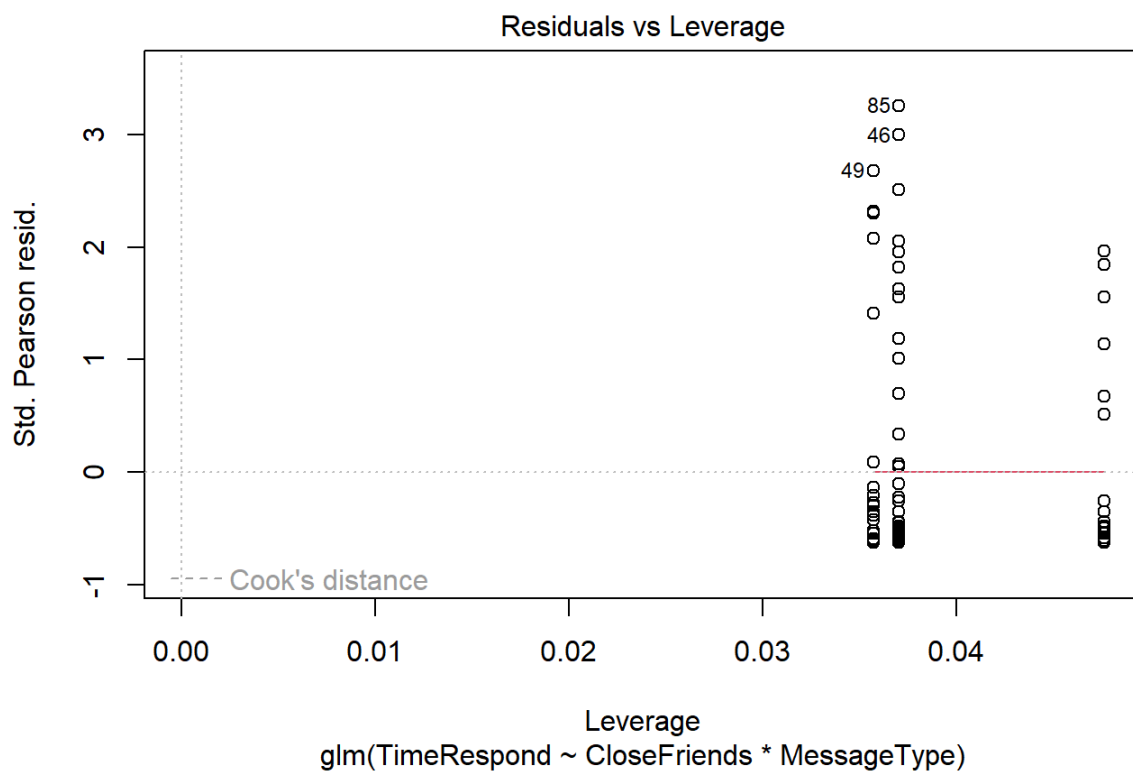
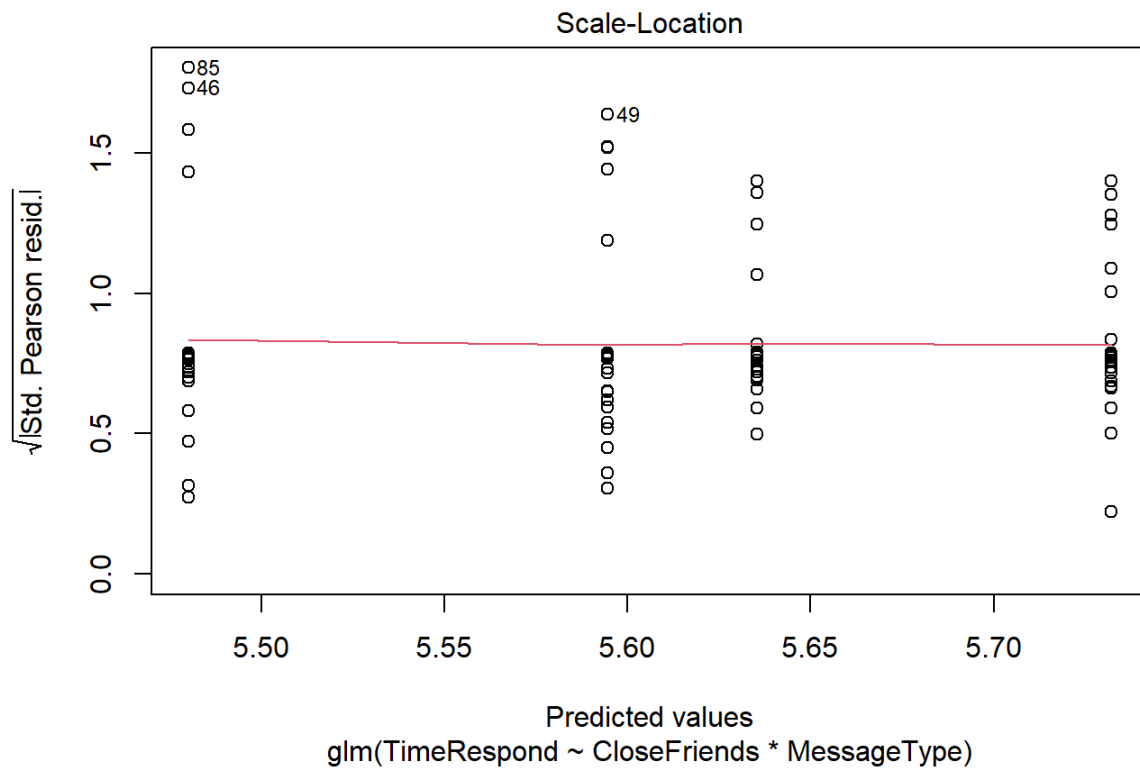
(Dispersion parameter for Gamma family taken to be 2.6698)

Null deviance: 360.04 on 102 degrees of freedom
Residual deviance: 359.17 on 99 degrees of freedom
AIC: 1277.6

Number of Fisher Scoring iterations: 10

```
plot(fit_gamma_glm_outliers)
```





```
# Calculate D-squared (Deviance R-squared)
D_squared <- 1 - (fit_gamma_glm_outliers$deviance / fit_gamma_glm_outliers$null.deviance)

# Convert to percentage for easier reading
D_squared_percent <- D_squared * 100
```

```
# Print the result
cat("Deviance R-squared:", round(D_squared, 5), "\n")
```

Deviance R-squared: 0.00243

```
cat("Percentage Explained:", round(D_squared_percent, 2), "%\n")
```

Percentage Explained: 0.24 %

Gamma GLM for first day

```
fit_gamma_glm_FD <- glm(TimeRespond ~ CloseFriends * MessageType, family=Gamma, data=datos_compress)
summary(fit_gamma_glm_FD)
```

Call:

```
glm(formula = TimeRespond ~ CloseFriends * MessageType, family = Gamma,
    data = datos_compressed_first_day)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.0167411	0.0026115	6.410	1.06e-08 ***
CloseFriends1	0.0007906	0.0026115	0.303	0.763
MessageType1	0.0005651	0.0026115	0.216	0.829
CloseFriends1:MessageType1	0.0014945	0.0026115	0.572	0.569

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Gamma family taken to be 1.826146)

Null deviance: 174.86 on 80 degrees of freedom
Residual deviance: 174.14 on 77 degrees of freedom
AIC: 819.81

Number of Fisher Scoring iterations: 7

```
# Calculate D-squared (Deviance R-squared)
D_squared <- 1 - (fit_gamma_glm_FD$deviance / fit_gamma_glm_FD$null.deviance)

# Convert to percentage for easier reading
D_squared_percent <- D_squared * 100

# Print the result
cat("Deviance R-squared:", round(D_squared, 5), "\n")
```

Deviance R-squared: 0.0041

```
cat("Percentage Explained:", round(D_squared_percent, 2), "%\n")
```

Percentage Explained: 0.41 %

Quasi Poisson GLM

```
fit_qpoisson_glm <- glm(TimeRespond ~ CloseFriends * MessageType, family=quasipoisson(link="log"),
```



```
summary(fit_qpoisson_glm)
```

Call:

```
glm(formula = TimeRespond ~ CloseFriends * MessageType, family = quasipoisson(link = "log"),  
    data = data_clean)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	6.11908	0.19098	32.040	<2e-16 ***
CloseFriends1	0.12712	0.19098	0.666	0.507
MessageType1	0.07671	0.19098	0.402	0.689
CloseFriends1:MessageType1	0.06347	0.19098	0.332	0.740

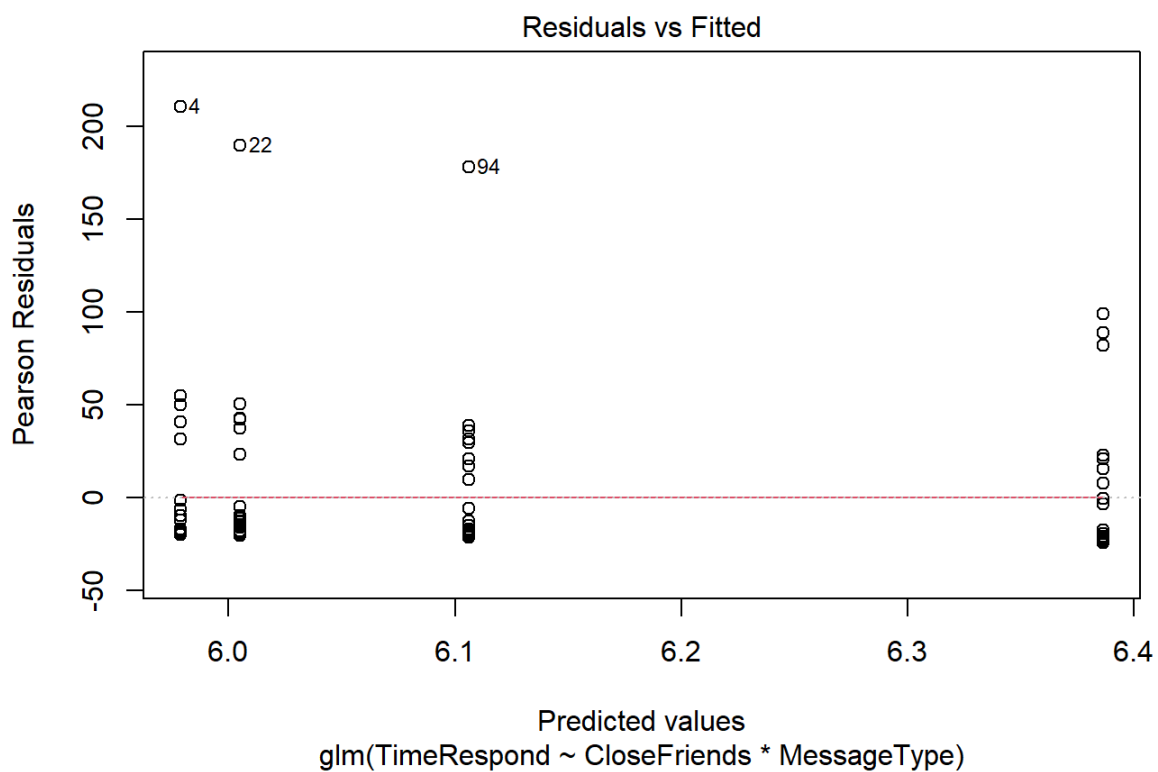
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

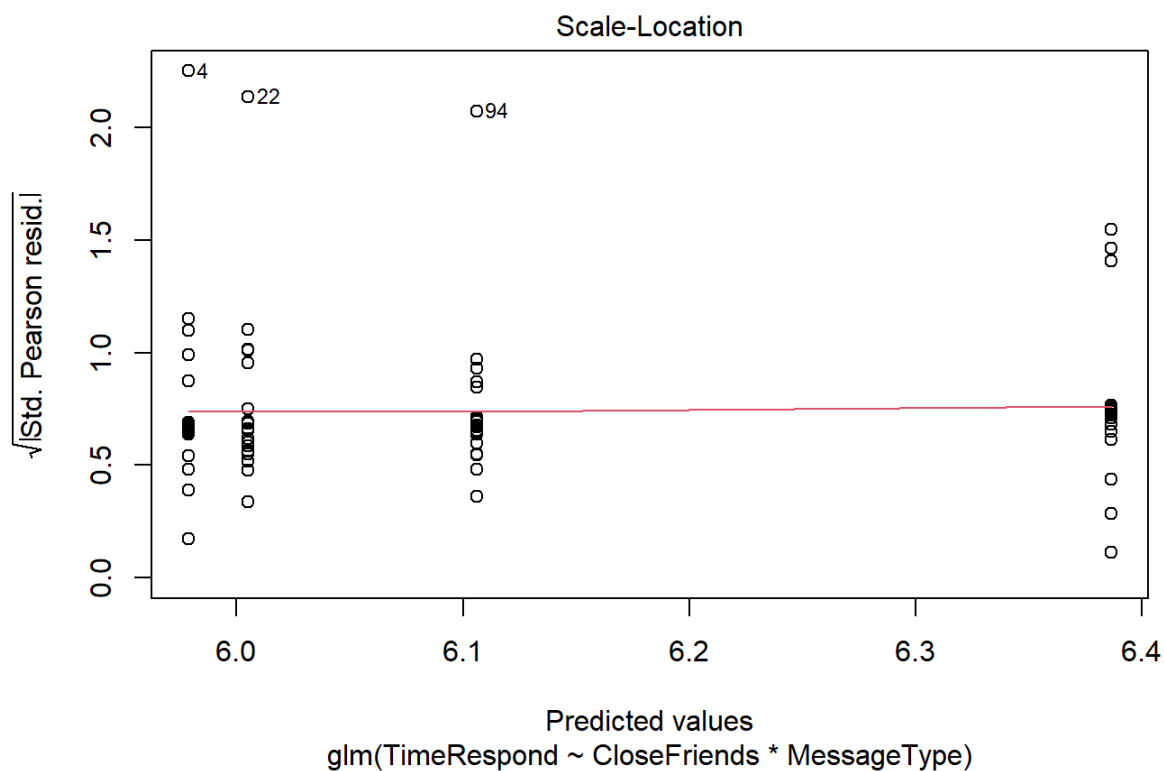
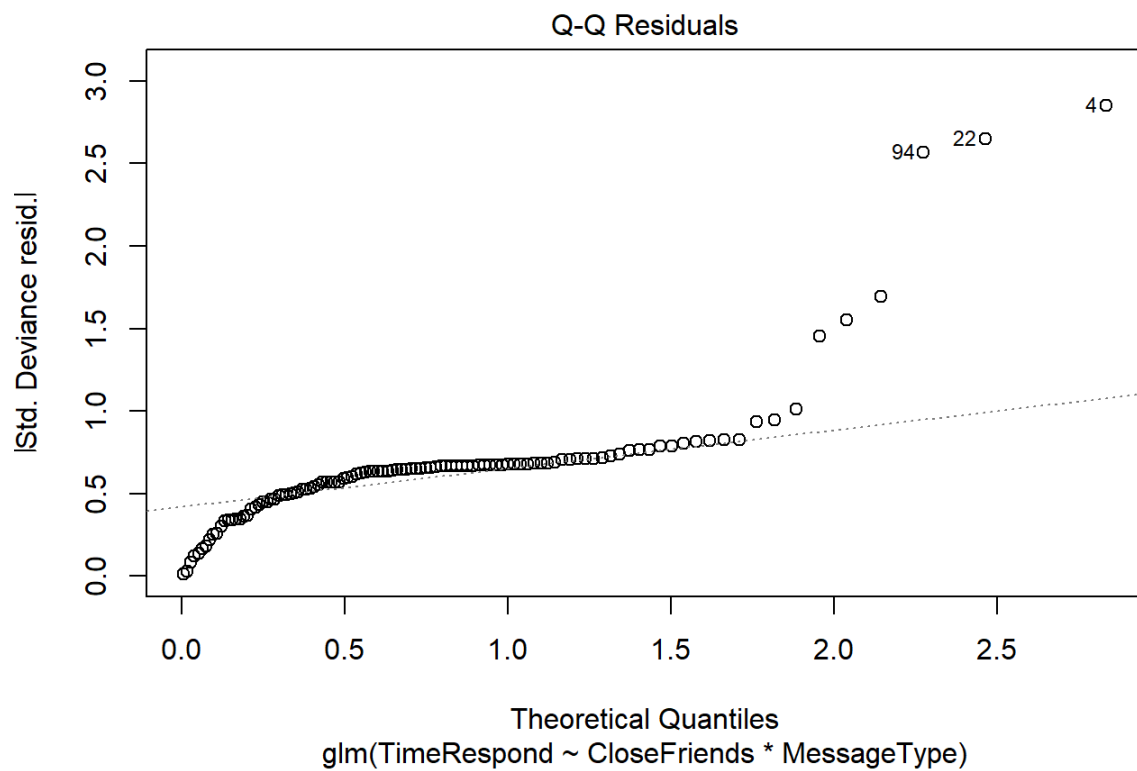
(Dispersion parameter for quasipoisson family taken to be 1794.227)

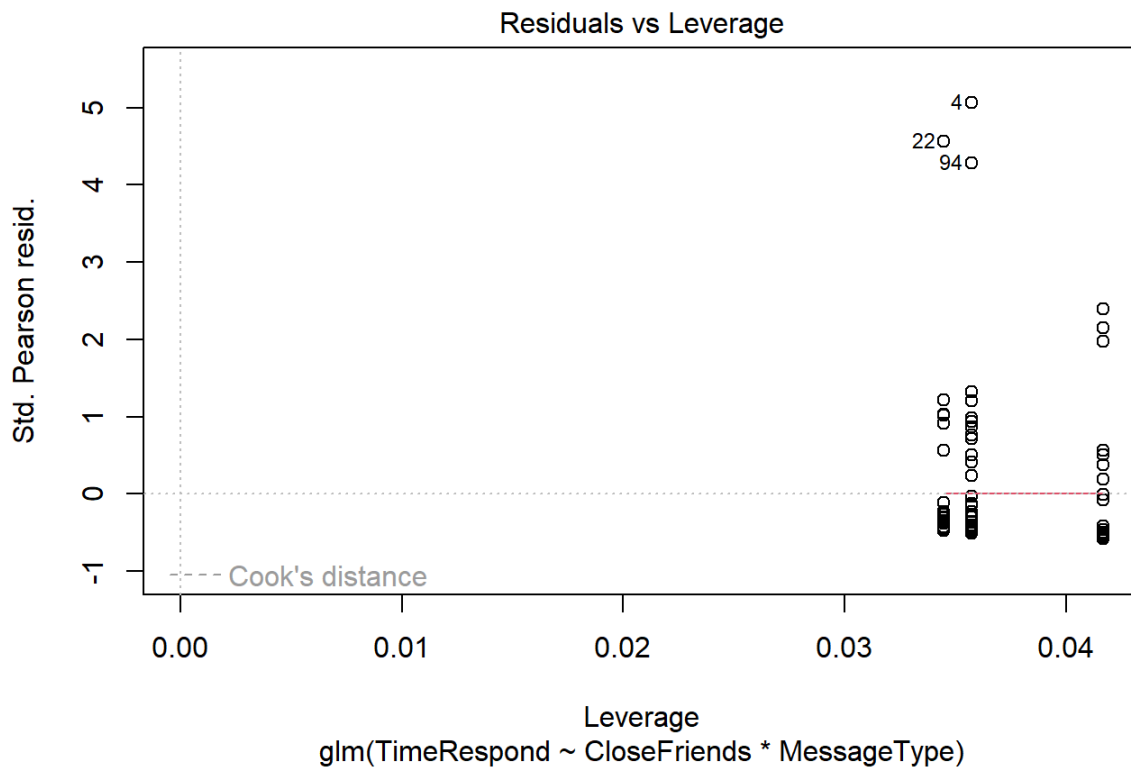
Null deviance: 117275 on 108 degrees of freedom
Residual deviance: 115953 on 105 degrees of freedom
(29 observations deleted due to missingness)
AIC: NA

Number of Fisher Scoring iterations: 6

```
plot(fit_qpoisson_glm)
```







```
# Calculate D-squared (Deviance R-squared)
D_squared <- 1 - (fit_qpoisson_glm$deviance / fit_qpoisson_glm$null.deviance)

# Convert to percentage for easier reading
D_squared_percent <- D_squared * 100

# Print the result
cat("Deviance R-squared:", round(D_squared, 5), "\n")
```

Deviance R-squared: 0.01127

```
cat("Percentage Explained:", round(D_squared_percent, 2), "%\n")
```

Percentage Explained: 1.13 %

SAMPLE SIZE ANALYSIS

```
observed_f2 <- (440.25 - 437.48) / 437.48

# Calculate required sample size for 80% power at alpha 0.05
# u = 1 (degrees of freedom for one effect)
power_analysis <- pwr.f2.test(u = 1, f2 = observed_f2, sig.level = 0.05, power = 0.80)

# Total N needed
required_N <- ceiling(power_analysis$v + power_analysis$u + 1)
print(required_N)
```

[1] 1242

observed_f2

[1] 0.006331718